

A Natural Primal-Dual Hybrid Gradient Method for Adversarial Neural Network Training on Solving Partial Differential Equations

Shu Liu, Stanley Osher, Wuchen Li

November 12, 2024

Abstract

We propose a scalable preconditioned primal-dual hybrid gradient algorithm for solving partial differential equations (PDEs). We multiply the PDE with a dual test function to obtain an inf-sup problem whose loss functional involves lower-order differential operators. The Primal-Dual Hybrid Gradient (PDHG) algorithm is then leveraged for this saddle point problem. By introducing suitable precondition operators to the proximal steps in the PDHG algorithm, we obtain an alternative natural gradient ascent-descent optimization scheme for updating the neural network parameters. We apply the Krylov subspace method (MINRES) to evaluate the natural gradients efficiently. Such treatment readily handles the inversion of precondition matrices via matrix-vector multiplication. A posterior convergence analysis is established for the time-continuous version of the proposed method. The algorithm is tested on various types of PDEs with dimensions ranging from 1 to 50, including linear and nonlinear elliptic equations, reaction-diffusion equations, and Monge-Ampère equations stemming from the L^2 optimal transport problems. We compare the performance of the proposed method with several commonly used deep learning algorithms such as physics-informed neural networks (PINNs), the DeepRitz method, weak adversarial networks (WANs), etc, for solving PDEs using the Adam and L-BFGS optimizers. The numerical results suggest that the proposed method performs efficiently and robustly and converges more stably.

Keywords— Deep learning for solving PDEs; Neural Networks; Inf-sup problem; Primal-Dual Hybrid Gradient (PDHG) algorithm; Natural Gradient; Convergence analysis; Monge-Ampère equation.

1 Introduction

Machine learning, particularly deep learning, is a fast-developing direction with modern computational technologies [1] and applications [31, 5]. Typical examples of applications often come from computer science, including creating new images, videos, and voices and generating languages. During the development of applications, machine learning also introduces several nonlinear methods, consisting of computational nonlinear models, such as neural network functions, and optimization variational formulations, such as generative adversary neural networks [31]. One cannot overestimate the applications of machine learning methods in scientific computing.

In recent years, deep learning algorithms have been developed to solve partial differential equations (PDEs). The Physics-Informed Neural Networks (PINN) method [73] employs neural networks to approximate PDE solutions by minimizing the discrepancy between observed data and the equation’s residual. The DeepRitz method [96] computes neural network surrogate solutions for PDEs using a variational approach, minimizing the associated energy functional. The Forward-Backward Stochastic Differential Equation (FBSDE) method [33] makes use of the nonlinear Feynman-Kac formula for semilinear parabolic equations to derive numerical solutions at specific time-space points. Additionally, the Weak Adversarial Network (WAN) [97] leverages the weak formulation of PDEs by multiplying the original equation with a test function and integrating by parts, resulting in an inf-sup saddle point problem for solving PDEs. The approach applies to many equations and is scalable to compute solutions of PDEs in high dimensions.

While these methods demonstrate the potential of applying machine learning techniques in solving PDEs, challenges such as hyperparameter tuning, loss function design, and convergence guarantees remain unresolved. More critically, due to the nonlinearity of neural networks, conventional optimizers such as Adam [41] or RMSProp [87] suffer from strong fluctuations and do not achieve stable convergence, which complicates the implementation of current algorithms.

In this research, we aim to tackle these challenges by adopting the adversarial training strategy and propose a designed optimizer that takes advantage of the primal-dual hybrid gradient (PDHG) algorithm [100, 16]. We utilize suitable preconditioned gradients known as natural gradients [65] to update the parameters of the neural networks. The proposed algorithm, named the Natural Primal-Dual Hybrid Gradient (NPDHG) method, performs efficiently and converges more stably than classical machine learning-based PDE solvers. In addition, we also provide a theoretical convergence guarantee for the proposed algorithm.

To illustrate the main idea, we consider the following linear equation posed with suitable boundary condition,

$$\mathcal{L}u = f \text{ on } \Omega, \quad \mathcal{B}u = g \text{ on } \partial\Omega. \quad (1)$$

Here $\Omega \subset \mathbb{R}^d$ is a bounded open region, $\partial\Omega$ denotes the boundary of Ω , $f: \Omega \rightarrow \mathbb{R}$, $g: \partial\Omega \rightarrow \mathbb{R}$ are L^2 functions. and $u: \Omega \rightarrow \mathbb{R}$ belongs to $H^2(\Omega)$. We assume $\mathcal{L}$ as a second-order elliptic operator and $\mathcal{B}$ as a linear boundary operator, which indicates the Dirichlet or Neumann boundary conditions. Here we assume that $\mathcal{L}$ can be split as $\mathcal{L} = \mathcal{M}_d^* \tilde{\mathcal{L}} \mathcal{M}_p$ with $\mathcal{M}_p, \mathcal{M}_d$ are first-order differential operators, $\tilde{\mathcal{L}}$ is a well-conditioned bounded linear operator, and $\mathcal{M}_d^*$ denotes the L^2 adjoint of $\mathcal{M}_d$. Suppose this equation admits a unique classical solution $u_* \in H^2(\Omega)$. The goal is to efficiently compute u_* .

By introducing the test functions (dual variables) $\varphi \in H_0^1(\Omega)$ and $\psi \in L^2(\partial\Omega)$ into the equation (1), we consider the following inf-sup problem with quadratic regularization terms. Here, $\epsilon > 0$ denotes the regularization coefficient,

$$\begin{aligned} \inf_u \sup_{(\varphi, \psi)} \mathcal{E}(u, (\varphi, \psi)) := & \langle \tilde{\mathcal{L}} \mathcal{M}_p u, \mathcal{M}_d \varphi \rangle_{L^2(\Omega)} - \langle f, \varphi \rangle_{L^2(\Omega)} - \frac{\epsilon}{2} \|\mathcal{M}_d \varphi\|_{L^2(\Omega)}^2 \\ & + \langle \mathcal{B}u - g, \psi \rangle_{L^2(\partial\Omega)} - \frac{\epsilon}{2} \|\psi\|_{L^2(\partial\Omega)}^2. \end{aligned} \quad (2)$$

Note that $u = u_*, \varphi = 0, \psi = 0$ form the saddle point of $\mathcal{E}$. We apply the preconditioned PDHG algorithm [39, 52] to compute inf-sup problem (2), thus solving the solution of PDE (1). The algorithm utilizes alternative proximal point steps and intermediate extrapolation to solve the inf-sup problem with selected preconditioning metrics. More specifically, the algorithm repeats the following three-line iteration

$$\begin{aligned} (\varphi_{n+1}, \psi_{n+1}) = \operatorname{argmin}_{(\varphi, \psi)} & \left\{ \frac{1}{2\tau_\varphi} (\|\mathcal{M}_d \varphi - \mathcal{M}_d \varphi_n\|_{L^2(\Omega)}^2 + \|\psi - \psi_n\|_{L^2(\partial\Omega)}^2) - \mathcal{E}(u_n, (\varphi_n, \psi_n)) \right\}, \\ \tilde{\varphi}_{n+1} = \varphi_{n+1} + \omega(\varphi_{n+1} - \varphi_n), \quad \tilde{\psi}_{n+1} = \psi_{n+1} + \omega(\psi_{n+1} - \psi_n) \\ u_{n+1} = \operatorname{argmin}_u & \left\{ \frac{1}{2\tau_u} (\|\mathcal{M}_p u - \mathcal{M}_p u_n\|_{L^2(\Omega)}^2 + \|\mathcal{B}u - \mathcal{B}u_n\|_{L^2(\partial\Omega)}^2) + \mathcal{E}(u_n, (\tilde{\varphi}_{n+1}, \tilde{\psi}_{n+1})) \right\}. \end{aligned} \quad (3)$$

Here $\tau_u, \tau_\varphi > 0$ are the step sizes of the algorithm. And $\omega > 0$ denotes the extrapolation coefficient. We briefly illustrate the motivation of preconditioning steps (3). In general, the differential operator $\mathcal{L}$ is usually ill-conditioned. As shown in [52], the convergence rate of the un-preconditioned dynamic equals $1 - \mathcal{O}(\frac{1}{\kappa^2})$ with κ denoting the condition number of the space discretization of $\mathcal{L}$. The convergence speed decreases fast as κ gets larger. To mitigate the slow convergence, we introduce preconditioning in the proximal steps of (13) (i.e., the 1st and the 3rd line of (3)).

So far, the algorithm we have developed remains at the functional level, which is generally intractable for practical implementation. To realize the proposed PDHG algorithm, we parameterize $u(\cdot)$, $\varphi(\cdot)$ and $\psi(\cdot)$ as $u_\theta(\cdot)$, $\varphi_\eta(\cdot)$ and $\psi_\xi(\cdot)$ with the tunable parameters $\theta \in \Theta_\theta \subseteq \mathbb{R}^{m_\theta}, \eta \in \Theta_\eta \subseteq \mathbb{R}^{m_\eta}$ and $\xi \in \Theta_\xi \subseteq \mathbb{R}^{m_\xi}$. A straightforward parameterization approach involves expressing these functions as linear combinations of predefined basis functions—a method traditionally employed in finite element methods. However, as the problem's dimensionality increases, such parameterization becomes computationally prohibitive due to the curse of dimensionality. Since it requires a significant number of basis functions to maintain accuracy [35].

Recent advances in deep learning have highlighted the potential of neural networks as computational tools to solve PDEs. Given their flexibilities and expressive powers, we adopt three neural network functions, such as Multi-Layer Perceptrons (MLPs, see Appendix A), to represent u_θ, φ_η , and ψ_ξ . Therefore, we reduce the original algorithm in functional spaces to a time-discrete dynamic in which the parameters θ^n, η^n, ξ^n evolve together.

We replace the implicit proximal step for updating η, ξ, θ with an explicit scheme known as the linearized PDHG algorithm. We come up with the following algorithm:

$$\begin{aligned} \begin{bmatrix} \eta^{n+1} \\ \xi^{n+1} \end{bmatrix} &= \begin{bmatrix} \eta^n \\ \xi^n \end{bmatrix} + \tau_\varphi \begin{bmatrix} M_p(\eta^n)^\dagger \nabla_\eta \mathcal{E}(u_{\theta^n}, \varphi_{\eta^n}, \psi_{\xi^n}) \\ M_{bdd}(\xi^n)^\dagger \nabla_\xi \mathcal{E}(u_{\theta^n}, \varphi_{\eta^n}, \psi_{\xi^n}) \end{bmatrix}, \\ \begin{bmatrix} \tilde{\varphi}_{n+1} \\ \tilde{\psi}_{n+1} \end{bmatrix} &= \begin{bmatrix} \varphi_{\eta^{n+1}} \\ \psi_{\xi^{n+1}} \end{bmatrix} + \omega \left(\begin{bmatrix} \varphi_{\eta^{n+1}} \\ \psi_{\xi^{n+1}} \end{bmatrix} - \begin{bmatrix} \varphi_{\eta^n} \\ \psi_{\xi^n} \end{bmatrix} \right), \\ \theta^{n+1} &= \theta^n - \tau_u M_p(\theta^n)^\dagger \nabla_\theta \mathcal{E}(u_{\theta^n}, \tilde{\varphi}_{n+1}, \tilde{\psi}_{n+1}). \end{aligned} \quad (4)$$

Here $M_p(\eta) \in \mathbb{R}^{m_\eta \times m_\eta}$, $M_{bdd}(\xi) \in \mathbb{R}^{m_\xi \times m_\xi}$, $M_p(\theta) \in \mathbb{R}^{m_\theta \times m_\theta}$ are Gram type matrices. They are derived from the bilinear form approximation of the proximal steps in the PDHG algorithm (3). Here, we denote “ $\dagger$ ” as the Moore–Penrose inverse of matrix. The precondition matrix $M_d(\eta^n)$ contains the information of the precondition operator $\mathcal{M}_p$, which is built in the original operator $\mathcal{L}$, we call $M_d(\eta^n)^\dagger \nabla_\eta \mathcal{E}(u_{\theta^n}, \varphi_{\eta^n}, \psi_{\xi^n})$ the *natural gradient* of $\mathcal{E}(u_\theta, \varphi_\eta, \psi_\xi)$ with respect to η . Similarly, we can define the natural gradient ascent and descent directions for variables ξ and θ . The algorithm alternatively updates the primal and dual parameters along the natural gradient directions. An additional extrapolation step in the functional space is introduced to enhance the convergence of the method. We denote the above updates as the **Natural Primal-Dual Hybrid Gradient (NPDHG)** algorithm. For simplicity, we also call it the **NPDG** algorithm. We refer the readers to section 2 for a detailed derivation of the algorithm.

While the NPDG algorithm is designed around linear PDEs, it effectively accommodates equations with nonlinear terms. Additionally, it can be extended to address fully nonlinear equations, such as the Monge–Ampère equation, which emerges in the context of the L^2 optimal transport (OT) problem [88, 20]. Since the optimal transport (OT) problem can be formulated as a constrained optimization problem, introducing the Lagrange multiplier method leads to a saddle point scheme. This scheme involves adversarial training with the pushforward map and the dual potential function to solve the Monge–Ampère equation, substituting both the map and potential function with neural network approximations and applying the NPDG algorithm with precondition matrices. The L^2 Gram type matrices lead to stable and efficient numerical results. Further analysis around the saddle point of the loss function suggests the other canonical preconditioning approach, where the mapping uses the L^2 Gram type matrix while the potential uses the H^1 Gram type matrix. For a detailed discussion, readers are referred to section 2.5.

In this research, we provide a posterior convergence analysis for the time-continuous version of the NPDG algorithm when applied to the linear PDE (1). Let $(\theta_t, \eta_t, \xi_t)$ be the solution obtained from the time-continuous NPDG algorithm for $0 \leq t \leq T$. Under specific conditions regarding the approximation capabilities of the tangent spaces generated by the partial derivatives of $u_{\theta_t}, \varphi_{\eta_t}$, and ψ_{ξ_t} with respect to parameters θ, η, ξ , we establish the linear convergence of the solution in the sense of

$$\|\mathcal{M}_p(u_{\theta_t} - u_*)\|_{L^2(\Omega)}^2 + \lambda \|\mathcal{B}(u_{\theta_t} - u_*)\|_{L^2(\partial\Omega)}^2 \leq C \cdot \exp(-rt) \quad \text{for } 0 \leq t \leq T.$$

Here $C > 0$ is a constant, $r > 0$ is the convergence rate depending on the PDE (1), the hyperparameters of the NPDG algorithm, and the neural network parameters. The readers are referred to section 3 for detailed discussion.

In the implementations, we apply the Monte-Carlo algorithm [15] to approximate $\mathcal{E}(u_\theta, \varphi_\eta, \psi_\xi)$; we use automatic differentiation to compute the derivatives of $\mathcal{E}(u_\theta, \varphi_\eta, \psi_\xi)$ with respect to the parameters θ, η, ξ . It is usually prohibitively expensive to explicitly evaluate $M_p(\theta), M_d(\eta), M_{bdd}(\xi)$ given that m_θ, m_η, m_ξ might be very large. To cope with this, we evaluate the pseudo-inverse in (4) via the iterative solver such as the Minimal residual method (MINRES) [70], which, instead of forming entire matrices, only requires matrix-vector multiplication. Further details of our treatment can be found in section 4.

Numerical examples of linear PDEs (1), nonlinear PDEs (26), and Monge-Ampère equations (27) in section 5 illustrate the accuracy, efficiency, and robustness of the NPDG method compared to classical methods, including the Physics-Informed Neural Network (PINN), the DeepRitz method, and the Weak Adversarial Network (WAN). Based on these numerical results, the algorithm demonstrates linear convergence for the high-dimensional PDEs tested in this section. Additionally, the proposed method achieves higher accuracy in both L^2 and H^1 norms compared to the other tested methods.

1.1 Related references

In recent years, machine learning algorithms have attracted increasing attention from the scientific computing community due to their flexibility and scalability. A considerable amount of these investigations are based on the Physics-Informed Neural Network (PINN) algorithm [73, 56]; further approaches that address the pathologies during PINN training include calibration of interior-boundary loss coefficients [90], and variable splitting techniques [8, 71]. The adaptive sampling methods [84, 85] are introduced to gain better accuracy of the neural network approximation. In addition to PINN, a series of deep learning-based algorithms are introduced for PDEs of various types, including the Deep Galerkin Method [81], Deep Ritz method [96, 57, 50], Forward-Backward Stochastic Differential Equation (FBSDE) approaches [33, 32, 37], Extreme Learning Machines [22, 68, 92], etc.

Recent research trends leverage adversarial training strategies [31, 5] to improve algorithm performance. In the Weak Adversarial Network (WAN) algorithm, discriminator neural networks are used to enhance training efficiency by employing the weak formulation of PDEs [97, 7]. Additionally, a residual-attention-based approach has been introduced in [62, 63, 4, 99] for seeking numerical solutions with higher precision.

The Primal-Dual Hybrid Gradient (PDHG) method, which is widely used in image processing problems [100, 16], has been introduced to handle nonlinear PDEs on classical numerical schemes [54, 52, 64]. Suitable preconditionings are introduced to improve the convergence of the algorithm significantly. The method is shown to converge linearly in [53].

Large-scale optimization algorithms play a crucial role in machine learning research. Stochastic gradient descent (SGD) is a widely used first-order optimization method [75, 78, 11]. One can improve the SGD's performance by incorporating momentum terms [77, 67, 83]. Various modified versions of SGD with per-parameter learning rates—such as AdaGrad [24], Adadelta [98], RMSProp [87], and Adam [41]—are popular optimizers in deep learning [72]. Additionally, second-order algorithms like the BFGS method [27], LBFGS method [49], and inexact-Newton methods [21, 12, 13, 25, 59, 76, 74] are also widely explored in machine learning research.

The natural gradient method is another critical category of second-order optimizers, initially introduced in [1] with further developments in [2, 86, 82]. An efficient, scalable variant known as the K-FAC (Kronecker-factored Approximate Curvature) method was proposed in [61]. The natural gradient method finds its application under different scenarios, including optimization involving combined loss functionals [95], PDE-constrained optimization [69], simulation and acceleration of Wasserstein gradient flows [48, 17, 91, 79, 51]. A series of research that utilizes the concept of the natural gradient to solve general time-dependent PDEs have been conducted, as detailed in [23, 14, 29, 18] and the references therein.

The natural gradient algorithm has recently been applied to training PINNs, achieving highly accurate solutions [65]. The K-FAC method is exploited in the follow-up work [19] to enable scalability in high-dimensional settings. Beyond natural gradients, the Gauss-Newton method has been introduced in [34] for computing variational PDEs. Additional preconditioning techniques for solving PDEs include the multigrid-augmented method [6], domain decomposition strategies [43], and incomplete LU preconditioning [55]. However, these methods typically need to scale more effectively to compute high-dimensional problems.

Compared to these methods, we summarize the advantages of the proposed approach in two key aspects: the primal-dual hybrid gradient algorithmic framework and the application of natural gradients in neural network functions.

► On the primal-dual framework:

- By applying integration by parts, we reduce the order of the differential operator $\mathcal{L}$ in the primal-dual formulation, lowering computational complexity when performing automatic differentiation

on the neural networks.

- The primal-dual training scheme is versatile and adaptable, making the algorithm suitable for a wide range of PDEs, including the fully nonlinear Monge-Ampère equation.
- On the primal-dual hybrid natural gradients:
 - Unlike other second-order optimization algorithms, such as L-BFGS, which are unable to handle the training involving random batches, the proposed algorithm is well-suited to data stochasticity, performing robustly under stochastic approximation.
 - To address the computation of large-scale linear systems (specifically, the pseudo-inverse of preconditioning matrices), we introduce the iterative method (MINRES). Consequently, our approach readily accommodates high-dimensional PDEs requiring neural networks with a large number of parameters. In experiments, we handle neural networks with parameter counts ranging from 20,000 to 300,000.

Generally, the proposed algorithm converges smoothly, avoiding the intense fluctuations and spikes commonly observed in the loss decay curves of classical momentum-based optimizers such as Adam and RMSProp. With appropriate preconditioning, theoretical analysis (Theorem 2) indicates linear convergence of the method. In practice, the approach performs more efficiently than classical machine learning methods and achieves higher precision in the norms L^2 and H^1 . Furthermore, as reflected in later Table 4, the method demonstrates robustness to its hyperparameters, including regularization coefficient ϵ , step sizes τ_φ , τ_u , and the extrapolation coefficient ω . Typically, a standard configuration of $\epsilon = 0.1$, $\tau_\varphi = 0.05$, $\tau_u = 0.095$, and $\omega = 1$ yields satisfactory performances.

This paper is organized as follows. In section 2, we provide a detailed derivation of the algorithm. Supplementary discussions on treating nonlinear PDEs are provided in subsection 2.5. Then, in section 3, we prove a posterior convergence result for the time-continuous version of the algorithm. Implementation details are demonstrated in section 4. We demonstrate a series of numerical examples in section 5. We also provide further materials related to algorithms and numerical examples in the Appendix.

2 Derivation of Natural Primal-Dual Hybrid Gradient (NPDHG) method

In this section, we provide a detailed derivation of the proposed method by first introducing the Primal-Dual Hybrid Gradient algorithm for root-finding problems. We then apply this algorithm to solving PDEs in the functional space. We improve the algorithm’s performance by introducing suitable preconditioning. We last discuss how we realize the algorithm by substituting the functions with neural networks and introduce the Natural Primal-Dual Hybrid Gradient (NPDHG) algorithm for adversarial training of the neural networks for solving PDEs.

2.1 Primal-Dual algorithm for root-finding problem

We first consider a root-finding problem defined on Hilbert space $\mathbb{X}$,

$$\mathcal{F}(x) = 0. \tag{5}$$

Here, we assume that $\mathcal{F} : \mathbb{X} \rightarrow \mathbb{Y}$ is a function from $\mathbb{X}$ to another Hilbert space $\mathbb{Y}$. The goal is to find a solution $x \in \mathbb{X}$. For a certain convex functional $\iota : \mathbb{Y} \rightarrow \mathbb{R}$ that satisfies $\iota(y) > 0$ iff $y \neq 0$ and $\iota(y) = 0$ whenever $y = 0$. The root-finding problem is equivalent to the following minimization problem

$$\inf_{x \in \mathbb{X}} \iota(\mathcal{F}(x)). \tag{6}$$

We denote the Legendre dual of $\iota(\cdot)$ as $\iota^*(\cdot)$ which is defined as $\iota^*(y) = \sup_{w \in \mathbb{Y}} \langle y, w \rangle_{\mathbb{Y}} - \iota(w)$. Here, we denote $\langle \cdot, \cdot \rangle_{\mathbb{Y}}$ as the inner product defined in the space $\mathbb{Y}$. Then

$$\iota(z) = \iota^{**}(z) = \sup_{y \in \mathbb{Y}} \langle z, y \rangle_{\mathbb{Y}} - \iota^*(y). \quad (7)$$

Substituting (7) into (6) yields the following saddle point problem

$$\inf_{x \in \mathbb{X}} \sup_{y \in \mathbb{Y}} \mathcal{E}(x, y) := \langle \mathcal{F}(x), y \rangle_{\mathbb{Y}} - \iota^*(y). \quad (8)$$

We now apply the PDHG algorithm to deal with the inf-sup problem (8), yielding

$$y_{n+1} = \underset{y \in \mathbb{Y}}{\operatorname{argmin}} \frac{\|y - y_n\|_{\mathbb{Y}}^2}{2\tau_y} - \mathcal{E}(x_n, y) = (\operatorname{Id} - \partial_y \mathcal{E}(x_n, \cdot))^{-1} y_n, \quad (9)$$

$$\tilde{y}_{n+1} = y_{n+1} + \omega(y_{n+1} - y_n),$$

$$x_{n+1} = \underset{x \in \mathbb{X}}{\operatorname{argmin}} \frac{\|x - x_n\|_{\mathbb{X}}^2}{2\tau_x} + \mathcal{E}(x, \tilde{y}_{n+1}) = (\operatorname{Id} + \partial_x \mathcal{E}(\cdot, \tilde{y}_{n+1}))^{-1} x_n. \quad (10)$$

Here $\tau_x, \tau_y > 0$ are the step sizes of the PDHG algorithm, $\omega > 0$ denotes the extrapolation coefficient. The proximal steps (9), (10) can be interpreted as the implicit update of the gradient ascent/descent algorithm of functional $\mathcal{E}$ as τ_x, τ_y are small enough. In practice, one can choose $\iota(\cdot) = \chi(\cdot)$, where χ is the indicator function defined as $\chi(y) = +\infty$ for $y \neq 0$ and $\chi(0) = 0$. In this case, the Legendre dual satisfies $\iota^*(\cdot) \equiv 0$. Another popular choice is $\iota(\cdot) = \frac{1}{2\epsilon} \|\cdot\|_{\mathbb{Y}}^2$ with $\iota^*(\cdot) = \frac{\epsilon}{2} \|\cdot\|_{\mathbb{Y}}^2$. Here, $\epsilon > 0$ is a tunable hyperparameter. We will mainly focus on the latter throughout the subsequent discussion of the paper.

2.2 Primal-Dual Hybrid Gradient algorithm for solving PDEs

From now on, we assume that $\Omega \subset \mathbb{R}^d$ is a bounded open set. Denote $\mathfrak{B}$ as the Borel algebra on Ω inherited from that of $\mathbb{R}^d$. We denote μ as a Borel measure on Ω . Furthermore, we let $L^2(\Omega)$ represent $L^2(\Omega, \mathfrak{B}, \mu)$; and $L^2(\Omega; \mathbb{R}^r)$ denote $L^2(\Omega, \mathfrak{B}, \mu; \mathbb{R}^r)$, the L^2 space of vector-valued functions on Ω . We denote $\mathfrak{B}_{\partial\Omega}$ as the Borel algebra on the boundary $\partial\Omega$, and $\mu_{\partial\Omega}$ as a Borel measure on $\partial\Omega$. We write $L^2(\partial\Omega)$ as $L^2(\partial\Omega, \mathfrak{B}_{\partial\Omega}, \mu_{\partial\Omega})$ in short.

Consider a linear equation defined on the Hilbert space $\mathbb{H} \subseteq L^2(\Omega)$,

$$\mathcal{L}u = f \text{ on } \Omega, \quad \text{with boundary condition(b.c.) } \mathcal{B}u = g \text{ on } \partial\Omega. \quad (11)$$

Here $\mathcal{L} : \mathbb{H} \rightarrow \mathbb{K} \subseteq L^2(\Omega)$ is a linear differential operator, and $\mathcal{B} : \mathbb{H} \rightarrow \mathbb{K}_{\partial\Omega} \subseteq L^2(\partial\Omega)$ is a linear boundary operator. We assume that $u_* \in \mathbb{H}$ is the classical solution to (11).

We now set $\mathcal{F} : \mathbb{H} \rightarrow \mathbb{K} \times \mathbb{K}_{\partial\Omega}, u \mapsto (\mathcal{L}u - f, \mathcal{B}u - g)$. By introducing the dual variable φ belonging to the Hilbert space $\mathbb{K}^{dual} \subseteq L^2(\Omega)$, and ψ from the Hilbert space $\mathbb{K}_{\partial\Omega}^{dual} \subseteq L^2(\partial\Omega)$, and setting $\mathbb{L}^2 = L^2(\Omega) \times L^2(\partial\Omega)$, we come up with the saddle point problem

$$\begin{aligned} \inf_{u \in \mathbb{H}} \sup_{\substack{\varphi \in \mathbb{K}^{dual} \\ \psi \in \mathbb{K}_{\partial\Omega}^{dual}}} \mathcal{E}(u, (\varphi, \psi)) &:= \langle \mathcal{F}(u), (\varphi, \psi) \rangle_{\mathbb{L}^2} - \frac{\epsilon}{2} \|(\varphi, \psi)\|_{\mathbb{L}^2}^2. \\ &= \langle \mathcal{L}u - f, \varphi \rangle_{L^2(\Omega)} - \frac{\epsilon}{2} \|\varphi\|_{L^2(\Omega)}^2 + \langle \mathcal{B}u - g, \psi \rangle_{L^2(\partial\Omega)} - \frac{\epsilon}{2} \|\psi\|_{L^2(\partial\Omega)}^2. \end{aligned} \quad (12)$$

It is not hard to verify that $u = u_*, \varphi = 0, \psi = 0$ form the saddle point of the inf-sup problem (12). We refer [36] for further discussion of the saddle point structure of related inf-sup formulations. We propose the

following PDHG algorithm to deal with the inf-sup problem (12),

$$\begin{aligned}
\begin{bmatrix} \varphi_{n+1} \\ \psi_{n+1} \end{bmatrix} &= \underset{(\varphi, \psi) \in \mathbb{K}^{dual} \times \mathbb{K}_{\partial\Omega}^{dual}}{\operatorname{argmin}} \left\{ \frac{1}{2\tau_\varphi} (\|\varphi - \varphi_n\|_{L^2(\Omega)}^2 + \|\psi - \psi_n\|_{L^2(\partial\Omega)}^2) - \mathcal{E}_0(u_n, (\varphi_n, \psi_n)) \right\}, \\
\begin{bmatrix} \tilde{\varphi}_{n+1} \\ \tilde{\psi}_{n+1} \end{bmatrix} &= \begin{bmatrix} \varphi_{n+1} \\ \psi_{n+1} \end{bmatrix} + \omega \left(\begin{bmatrix} \varphi_{n+1} \\ \psi_{n+1} \end{bmatrix} - \begin{bmatrix} \varphi_n \\ \psi_n \end{bmatrix} \right), \\
u_{n+1} &= \underset{u \in \mathbb{H}}{\operatorname{argmin}} \left\{ \frac{1}{2\tau_u} (\|u - u_n\|_{L^2(\Omega)}^2 + \|\mathcal{B}u - \mathcal{B}u_n\|_{L^2(\partial\Omega)}^2) + \mathcal{E}_0(u_n, (\tilde{\varphi}_{n+1}, \tilde{\psi}_{n+1})) \right\}.
\end{aligned} \tag{13}$$

To develop an intuitive understanding of why the algorithm (13) has difficulties in approaching the PDE solution, we consider the square region Ω and discretize it into N_x^d lattices. We apply the finite difference scheme to discretize (11) into grids. Solving the PDE yields a linear equation $Ax - b = 0$. Here, A is the matrix obtained upon discretizing $\mathcal{L}$. Roughly speaking, $A \in \mathbb{R}^{N_x^d \times N_x^d}$ is self-adjoint and non-singular, $x \in \mathbb{R}^{N_x^d}$ denotes the numerical solution of the PDE on the grid points, $b \in \mathbb{R}^{N_x^d}$ is the vector encoding f and its boundary condition. The proposed PDHG algorithm yields

$$\begin{aligned}
y_{n+1} &= \underset{y}{\operatorname{argmin}} \frac{\|y - y_n\|^2}{2\tau_y} - (Ax_n - b)^\top y, \\
\tilde{y}_{n+1} &= 2y_{n+1} - y_n, \\
x_{n+1} &= \underset{x}{\operatorname{argmin}} \frac{\|x - x_n\|^2}{2\tau_x} + (Ax - b)^\top \tilde{y}_{n+1}.
\end{aligned}$$

Here, we set $\epsilon = 0$ and $\omega = 1$ to simplify the discussion. And $\|\cdot\|$ denotes the ℓ_2 norm of $\mathbb{R}^N$. It is not hard to verify that the above algorithm is equivalent to the following update:

$$\begin{bmatrix} \hat{x}_{n+1} \\ y_{n+1} \end{bmatrix} = \underbrace{\begin{bmatrix} I - 2\tau_x\tau_y A^\top A & -\tau_x A^\top \\ \tau_y A & I \end{bmatrix}}_{\text{denote as } \Gamma} \begin{bmatrix} \hat{x}_n \\ y_n \end{bmatrix}.$$

Here, we denote x_* as the solution to $Ax - b = 0$ and $\hat{x}_n = x_n - x_*$. The convergence rate of the PDHG algorithm depends on the spectrum radius $\rho(\Gamma)$ of Γ . The minimum value of $\rho(\Gamma)$ equals $\sqrt{1 - \frac{c}{\kappa^2}}$, where $c \in [1, \frac{4}{3}]$ and κ denotes the condition number of A [52]. If $\mathcal{L}$ equals the Laplace operator Δ , the matrix A obtained via central difference scheme takes condition number $\kappa = \mathcal{O}(N_x^2)$ [47]. This indicates that the convergence rate of the PDHG method is $\sqrt{1 - \frac{c}{\kappa^2}} = 1 - \mathcal{O}(\frac{1}{N_x^2})$, which becomes very inefficient as N_x increases.

2.3 Preconditioning of the primal-dual damping algorithm

The discussion in section 1 suggests that we should introduce preconditioning to the original algorithm (13). As mentioned previously, we assume that $\mathcal{L}$ can be split as $\mathcal{L} = \mathcal{M}_d^* \tilde{\mathcal{L}} \mathcal{M}_p$ with $\mathcal{M}_d^*, \tilde{\mathcal{L}}, \mathcal{M}_p$ are linear differential operators whose domain and image spaces are denoted as follows.

$$\begin{array}{ccccccc}
L^2(\Omega; \mathbb{R}^r) & & L^2(\Omega; \mathbb{R}^r) & & & & \\
\cup & & \cup & & & & \\
L^2(\Omega) \supseteq \mathbb{H} & \xrightarrow{\mathcal{M}_p} & \tilde{\mathbb{H}} & \xrightarrow{\tilde{\mathcal{L}}} & \tilde{\mathbb{K}} & \xrightarrow{\mathcal{M}_d^*} & \mathbb{K} \subseteq L^2(\Omega)
\end{array}$$

Here we assume $\tilde{\mathbb{H}}, \tilde{\mathbb{K}}$ are Hilbert spaces. Moreover, suppose $\mathcal{M}_d : \mathbb{K}^{dual} \rightarrow \tilde{\mathbb{K}}^{dual} \subseteq L^2(\Omega; \mathbb{R}^r)$ is a linear operator with $\tilde{\mathbb{K}}^{dual}$ as a Hilbert space. We assume that $\mathcal{M}_d^*$ is the “adjoint” of $\mathcal{M}_d$ in the sense of

$$\langle \mathcal{M}_d^* \mathbf{u}, \varphi \rangle_{L^2(\Omega)} = \langle \mathbf{u}, \mathcal{M}_d \varphi \rangle_{L^2(\Omega; \mathbb{R}^r)}, \quad \forall \varphi \in \mathbb{K}^{dual}, \mathbf{u} \in \tilde{\mathbb{K}}.$$

Now recall that $u_* \in \mathbb{H}$ is the solution to (11). For any $u \in \mathbb{H}, \varphi \in \mathbb{K}^{dual}$, we have

$$\begin{aligned}\langle \mathcal{L}u - f, \varphi \rangle_{L^2(\Omega)} &= \langle \mathcal{L}(u - u_*), \varphi \rangle_{L^2(\Omega)} = \langle \mathcal{M}_d^* \tilde{\mathcal{L}} \mathcal{M}_p(u - u_*), \varphi \rangle_{L^2(\Omega)} \\ &= \langle \tilde{\mathcal{L}} \mathcal{M}_p(u - u_*), \mathcal{M}_d \varphi \rangle_{L^2(\Omega; \mathbb{R}^r)}.\end{aligned}\quad (14)$$

Example 2.1. Taking the negative Laplace operator $\mathcal{L} = -\Delta$ as an example, we can split $-\Delta = -\nabla \cdot \nabla$. By setting $\mathbb{H} = H^2(\Omega)$, $\tilde{\mathbb{H}} = \tilde{\mathbb{K}} = H^1(\Omega, \mathbb{R}^d)$, $\mathbb{K} = L^2(\Omega)$, and $\mathbb{K}^{dual} = H_0^1(\Omega)$, $\tilde{\mathbb{K}}^{dual} = L^2(\Omega; \mathbb{R}^d)$, and choosing $\mathcal{M}_d = \mathcal{M}_p = \nabla$ and $\tilde{\mathcal{L}} = \text{Id}$, we obtain

$$\begin{aligned}\langle -\Delta u - f, \varphi \rangle_{L^2(\Omega)} &= \langle -\Delta(u - u_*), \varphi \rangle_{L^2(\Omega)} \\ &= - \int_{\partial\Omega} \frac{\partial(u - u_*)}{\partial \mathbf{n}} \varphi \, d\sigma + \int_{\Omega} \nabla(u - u_*) \cdot \nabla \varphi \, d\mu \\ &= \langle \nabla(u - u_*), \nabla \varphi \rangle_{L^2(\Omega; \mathbb{R}^d)},\end{aligned}$$

for any $u \in \mathbb{H} = H^2(\Omega)$, $\varphi \in \mathbb{K}^{dual} = H_0^1(\Omega)$.

Example 2.2. Consider the elliptic operator $\mathcal{L} = \text{Id} - \Delta$, where Id is an identity operator. By setting the functional spaces the same as the previous example, while letting $\tilde{\mathbb{H}} = H^2(\Omega) \times H^1(\Omega; \mathbb{R}^d)$, and $\mathbb{K}^{dual} = H^1(\Omega) \times L^2(\Omega; \mathbb{R}^d)$, we can split the elliptic operator as

$$\text{Id} - \Delta = \begin{bmatrix} \text{Id} & -\nabla \cdot \end{bmatrix} \begin{bmatrix} \text{Id} & \\ & \text{Id} \end{bmatrix} \begin{bmatrix} \text{Id} \\ \nabla \end{bmatrix} = \begin{bmatrix} \text{Id} \\ \nabla \end{bmatrix}^* \begin{bmatrix} \text{Id} & \\ & \text{Id} \end{bmatrix} \begin{bmatrix} \text{Id} \\ \nabla \end{bmatrix} = \mathcal{M}_d^* \tilde{\mathcal{L}} \mathcal{M}_p.$$

We have

$$\begin{aligned}\langle (\text{Id} - \Delta)u - f, \varphi \rangle_{L^2(\Omega)} &= \langle (\text{Id} - \Delta)(u - u_*), \varphi \rangle_{L^2(\Omega)} \\ &= \langle u - u_*, \varphi \rangle_{L^2(\Omega)} + \langle \nabla(u - u_*), \nabla \varphi \rangle_{L^2(\Omega; \mathbb{R}^d)} \\ &= \left\langle \begin{bmatrix} u - u_* \\ \nabla(u - u_*) \end{bmatrix}, \begin{bmatrix} \varphi \\ \nabla \varphi \end{bmatrix} \right\rangle_{L^2(\Omega; \mathbb{R}^{1+d})}\end{aligned}$$

for any $u \in H^2(\Omega)$, $\varphi \in H_0^1(\Omega)$.

More examples can be found in section 5. Similar to (14), recall that $\mathcal{B}$ is linear, for any $u \in \mathbb{H}, \psi \in \mathbb{K}_{\partial\Omega}^{dual}$, we have

$$\langle \mathcal{B}u - g, \psi \rangle_{L^2(\partial\Omega)} = \langle \mathcal{B}(u - u_*), \psi \rangle_{L^2(\partial\Omega)}.$$

As mentioned in section 1, we substitute u, φ in the proximal steps of (13) with $\mathcal{M}_p(u - u_*)$, $\mathcal{B}(u - u_*)$ and $\mathcal{M}_d \varphi$. Correspondingly, we use the following modified functional $\mathcal{E} : \mathbb{H} \times \mathbb{K}^{dual} \times \mathbb{K}_{\partial\Omega}^{dual} \rightarrow \mathbb{R}$ in the algorithm:

$$\begin{aligned}\mathcal{E}(u, \varphi, \psi) &= \langle \tilde{\mathcal{L}} \mathcal{M}_p(u - u_*), \mathcal{M}_d \varphi \rangle_{L^2(\Omega; \mathbb{R}^r)} + \langle \mathcal{B}(u - u_*), \psi \rangle_{L^2(\partial\Omega)} - \frac{\epsilon}{2} (\|\mathcal{M}_d \varphi\|_{L^2(\Omega; \mathbb{R}^r)}^2 + \|\psi\|_{L^2(\Omega)}^2) \\ &= \langle \tilde{\mathcal{L}} \mathcal{M}_p u, \mathcal{M}_d \varphi \rangle_{L^2(\Omega; \mathbb{R}^r)} - \langle \mathcal{L} u_*, \varphi \rangle_{L^2(\Omega)} + \langle \mathcal{B}(u - u_*), \psi \rangle_{L^2(\partial\Omega)} - \frac{\epsilon}{2} (\|\mathcal{M}_d \varphi\|_{L^2(\Omega; \mathbb{R}^r)}^2 + \|\psi\|_{L^2(\Omega)}^2) \\ &= \langle \tilde{\mathcal{L}} \mathcal{M}_p u, \mathcal{M}_d \varphi \rangle_{L^2(\Omega; \mathbb{R}^r)} - \langle f, \varphi \rangle_{L^2(\Omega)} + \langle \mathcal{B}u - g, \psi \rangle_{L^2(\partial\Omega)} - \frac{\epsilon}{2} (\|\mathcal{M}_d \varphi\|_{L^2(\Omega; \mathbb{R}^r)}^2 + \|\psi\|_{L^2(\Omega)}^2).\end{aligned}\quad (15)$$

We now treat $(\mathcal{M}_p(u - u_*), \mathcal{B}(u - u_*))$, together with $(\mathcal{M}_d \varphi, \psi)$, as the new primal and dual variables of the algorithm. By doing so, we substitute $(u, \mathcal{B}u)$, (φ, ψ) in the proximal steps (the 1st and 3rd line) of (13) with $(\mathcal{M}_p(u - u_*), \mathcal{B}(u - u_*))$, $(\mathcal{M}_d \varphi, \psi)$. Therefore, we come up with the following preconditioned version of the PDHG algorithm. This treatment is also known as the G-prox PDHG algorithm introduced in [39].

$$\begin{aligned}\begin{bmatrix} \varphi_{n+1} \\ \psi_{n+1} \end{bmatrix} &= \underset{(\varphi, \psi) \in \mathbb{K}^{dual} \times \mathbb{K}_{\partial\Omega}^{dual}}{\text{argmin}} \left\{ \frac{1}{2\tau_\varphi} (\|\mathcal{M}_d \varphi - \mathcal{M}_d \varphi_n\|_{L^2(\Omega; \mathbb{R}^r)}^2 + \|\psi - \psi_n\|_{L^2(\partial\Omega)}^2) - \mathcal{E}(u_n, \varphi, \psi) \right\}, \\ \begin{bmatrix} \tilde{\varphi}_{n+1} \\ \tilde{\psi}_{n+1} \end{bmatrix} &= \begin{bmatrix} \varphi_{n+1} \\ \psi_{n+1} \end{bmatrix} + \omega \left(\begin{bmatrix} \varphi_{n+1} \\ \psi_{n+1} \end{bmatrix} - \begin{bmatrix} \varphi_n \\ \psi_n \end{bmatrix} \right), \\ u_{n+1} &= \underset{u \in \mathbb{H}}{\text{argmin}} \left\{ \frac{1}{2\tau_u} (\|\mathcal{M}_p u - \mathcal{M}_p u_n\|_{L^2(\Omega; \mathbb{R}^r)}^2 + \|\mathcal{B}u - \mathcal{B}u_n\|_{L^2(\partial\Omega)}^2) + \mathcal{E}(u, \tilde{\varphi}_{n+1}, \tilde{\psi}_{n+1}) \right\},\end{aligned}\quad (16)$$

Remark 1. It is worth noting that the space $\mathbb{H}$ mentioned earlier can be extended to a broader functional space. For instance, as illustrated in Example 2.1 with $\mathcal{L} = -\Delta$, the domain of $\mathcal{M}_d = \nabla$ can be extended to $H^1(\Omega)$, allowing the range for the variable u to also be expand to this functional space. Since our primary objective in this research is to compute the classical solution u_* of the equation, it is reasonable to confine our discussion to $\mathbb{H}$. Moreover, in implementing the algorithm, as illustrated in sections 5.1 and 5.2, we model $u(\cdot)$ as a Multi-Layer Perceptron with tanh or Softplus activation functions, which belong to $\mathbb{H} = H^2(\Omega)$. Thus, we restrict our analysis to $\mathbb{H}$ in this study, with the extension of our theory to broader functional spaces planned as future work.

2.4 Natural Primal-Dual Hybrid Gradient (NPDHG) algorithm for neural networks

At the beginning of this section, we briefly introduce the idea of the Natural Gradient method [1, 2, 60].

2.4.1 Natural Gradient method

For a wide range of machine learning problems, we assume that the loss function $J(\theta) = \mathcal{J}(u_\theta)$ where $\mathcal{J} : \mathbb{U} \rightarrow \mathbb{R}$ denotes the loss functional and u_θ is the parametrized function on the metric space $\mathbb{U}$ with the parameter $\theta \in \Theta \subset \mathbb{R}^m$ to be determined. The essential idea of the natural gradient algorithm is to conduct gradient descent on u_θ as an entity in the functional space rather than on the parameter θ . This can be realized by considering the proximal algorithm

$$\inf_{u_\theta \in \mathbb{U}} \frac{d^2(u_\theta, u_{\theta^n})}{2\tau} + J(\theta). \quad (17)$$

The preconditioning matrix $G(\theta)$ can thus be obtained by investigating the infetsimal distance $d^2(u_\theta, u_{\theta^n}) \approx (\theta - \theta^n)^\top G(\theta)(\theta - \theta^n)$, where $d(\cdot, \cdot)$ is a distance function enriches the Hessian information of the loss functional $\mathcal{J}$. By sending $\tau \rightarrow 0$, the implicit scheme (17) reduces to the natural gradient flow

$$\dot{\theta}_t = -G(\theta)^{-1} \nabla_\theta J(\theta).$$

As a result, viewing from the parameter space, the natural gradient algorithm can be realized by applying $G(\theta)$ -preconditioned gradient descent steps to loss function $J(\theta)$. We refer the interested readers to [2] for a comprehensive illustration of the Natural Gradient methods.

Let us continue the discussion on the derivation of NPDG algorithm. We substitute $u(\cdot)$, $\varphi(\cdot)$, $\psi(\cdot)$ with neural networks $u_\theta(\cdot)$, $\varphi_\eta(\cdot)$ and $\psi_\xi(\cdot)$ with tunable parameters $\theta \in \Theta_\theta \subseteq \mathbb{R}^{m_\theta}$, $\eta \in \Theta_\eta \subseteq \mathbb{R}^{m_\eta}$, $\xi \in \Theta_\xi \subseteq \mathbb{R}^{m_\xi}$. Here, we assume that the parameter spaces $\Theta_\theta, \Theta_\eta, \Theta_\xi$ are open sets of the Euclidean space. Then, algorithm (16) becomes

$$\begin{aligned} \begin{bmatrix} \eta^{n+1} \\ \xi^{n+1} \end{bmatrix} &= \underset{\varphi \in \mathbb{R}^{m_\eta}, \psi \in \mathbb{R}^{m_\xi}}{\operatorname{argmin}} \left\{ \frac{1}{2\tau_\varphi} (\|\mathcal{M}_d \varphi_\eta - \mathcal{M}_d \varphi_{\eta^n}\|_{L^2(\Omega; \mathbb{R}^r)}^2 + \|\psi_\xi - \psi_{\xi^n}\|_{L^2(\partial\Omega)}^2) - \mathcal{E}(u_{\theta^n}, \varphi_\eta, \psi_\xi) \right\}, \\ \begin{bmatrix} \tilde{\varphi}_{n+1} \\ \tilde{\psi}_{n+1} \end{bmatrix} &= \begin{bmatrix} \varphi_{\eta^{n+1}} \\ \psi_{\xi^{n+1}} \end{bmatrix} + \omega \left(\begin{bmatrix} \varphi_{\eta^{n+1}} \\ \psi_{\xi^{n+1}} \end{bmatrix} - \begin{bmatrix} \varphi_{\eta^n} \\ \psi_{\xi^n} \end{bmatrix} \right), \\ \theta^{n+1} &= \underset{\theta \in \mathbb{R}^{m_\theta}}{\operatorname{argmin}} \left\{ \frac{1}{2\tau_u} (\|\mathcal{M}_p u_\theta - \mathcal{M}_p u_{\theta^n}\|_{L^2(\Omega; \mathbb{R}^r)}^2 + \|\mathcal{B} u_\theta - \mathcal{B} u_{\theta^n}\|_{L^2(\partial\Omega)}^2) + \mathcal{E}(u_\theta, \tilde{\varphi}_{n+1}, \tilde{\psi}_{n+1}) \right\}, \end{aligned} \quad (18)$$

Let us take a closer look at the first line of (18). Since φ and ψ are separable in $\mathcal{E}$, it is not hard to verify that the updating rule of η^{n+1} can be formulated as

$$\eta^{n+1} = \underset{\eta \in \mathbb{R}^{m_\eta}}{\operatorname{argmin}} \left\{ \frac{1}{2\tau_\varphi} \|\mathcal{M}_d \varphi_\eta - \mathcal{M}_d \varphi_{\eta^n}\|_{L^2(\Omega; \mathbb{R}^r)}^2 - \mathcal{E}(u_\theta, \varphi_\eta, \psi_{\xi^n}) \right\}. \quad (19)$$

As suggested in section 1, we approximate $\mathcal{M}_d \varphi_\eta(x) - \mathcal{M}_d \varphi_{\eta^n}(x)$ with

$$\frac{\partial}{\partial \eta} (\mathcal{M}_d \varphi_{\eta^n}(x) \cdot (\eta - \eta^n)) := \frac{\partial}{\partial \eta} \mathcal{M}_d \varphi_{\eta^n}(x) (\eta - \eta^n).$$

Here, we denote $\frac{\partial}{\partial \eta} \mathcal{M}_d \varphi_\eta(x) \in \mathbb{R}^{r \times m_\eta}$ as the Jacobian matrix of $\mathcal{M}_d \varphi_\eta(\cdot)$ with respect to the parameter η at η^n . We therefore have

$$\begin{aligned} \|\mathcal{M}_d \varphi_\eta - \mathcal{M}_d \varphi_{\eta^n}\|_{L^2(\Omega; \mathbb{R}^r)}^2 &\approx \int_{\Omega} \left\| \frac{\partial}{\partial \eta} \mathcal{M}_d \varphi_{\eta^n}(x) (\eta - \eta^n) \right\|^2 d\mu(x) \\ &= \sum_{i=1}^{m_\eta} \sum_{j=1}^{m_\eta} \left\langle \frac{\partial}{\partial \eta_i} \mathcal{M}_d \varphi_{\eta^n}, \frac{\partial}{\partial \eta_j} \mathcal{M}_d \varphi_{\eta^n} \right\rangle_{L^2(\Omega; \mathbb{R}^r)} (\eta_i - \eta_i^n)(\eta_j - \eta_j^n) \\ &= (\eta - \eta^n)^\top M_d(\eta^n) (\eta - \eta^n), \end{aligned} \quad (20)$$

where we denote

$$M_d(\eta^n) = \int_{\Omega} \frac{\partial}{\partial \eta} \mathcal{M}_d \varphi_{\eta^n}(x)^\top \frac{\partial}{\partial \eta} \mathcal{M}_d \varphi_{\eta^n}(x) d\mu(x), \quad (21)$$

as an $\mathbb{R}^{m_\varphi \times m_\varphi}$ symmetric, semi-positive definite Gram matrix that encodes the information of $\mathcal{M}_d$.

Replacing the proximal term in (19) with the quadratic term (20) yields

$$\frac{1}{2\tau_\varphi} \Delta \eta^\top M_d(\eta^n) \Delta \eta - \widehat{E}(\theta^n, \eta^n + \Delta \eta, \xi^n). \quad (22)$$

Here we denote $\Delta \eta = \eta - \eta^n$ and $\widehat{E}(\theta^n, \eta, \xi) = \mathcal{E}(u_{\theta^n}, \varphi_\eta, \psi_\xi)$ for shorthand. By linearizing $\widehat{E}(\theta^n, \eta, \xi^n)$ at $\eta = \eta^n$, minimizing the quantity (22) is equivalent to minimizing

$$\begin{aligned} &\frac{1}{2} \Delta \eta^\top M_d(\eta^n) \Delta \eta - \tau_\varphi (\widehat{E}(\theta^n, \eta^n + \Delta \eta, \xi^n) - \widehat{E}(\theta^n, \eta^n, \xi^n)) \\ &\approx \frac{1}{2} \Delta \eta^\top M(\eta^n) \Delta \eta - \tau_\varphi \nabla_\eta \widehat{E}(\theta^n, \eta^n, \xi^n)^\top \Delta \eta + \mathcal{O}(\tau_\varphi \Delta \eta^2). \end{aligned}$$

We further omit the term $\mathcal{O}(\tau_\varphi \Delta \eta^2)$ to obtain

$$\min_{\Delta \eta \in \mathbb{R}^{m_\eta}} \left\{ \frac{1}{2} \Delta \eta^\top M_d(\eta^n) \Delta \eta - \tau_\varphi \nabla_\eta \widehat{E}(\theta^n, \eta^n, \psi_n)^\top \Delta \eta \right\}. \quad (23)$$

This is a least-square problem with the optimal solution denoted as

$$\Delta \eta = \tau_\varphi \cdot M_d(\eta^n)^\dagger \nabla_\eta \widehat{E}(\theta^n, \eta^n, \xi^n).$$

The resulting formula suggests that we explicitly update η along the gradient ascent direction preconditioned by the Gram matrix $M_d(\eta^n)$,

$$\eta^{n+1} = \eta^n + \tau_\varphi \cdot M_d(\eta^n)^\dagger \nabla_\eta \widehat{E}(\theta^n, \eta^n, \xi^n).$$

By doing so, we exchange some of the numerical stability enjoyed by the proximal step for computational feasibility and efficacy.

Remark 2. It is worth mentioning that the Moore-Penrose inverse used here is generally not necessary. According to Lemma 1 proved in the next section, we have $\nabla_\eta \widehat{E}(\theta^n, \eta^n, \psi_n) \in \text{Ran}(M_d(\eta^n))$. Thus, in order to determine a solution $\mathbf{v}$ to (23), one only need to guarantee that $M_d(\eta^n) \mathbf{v} = \nabla_\eta \widehat{E}(\theta^n, \eta^n, \psi_n)$. Consider any pseudo-inverse matrix $M_p^+(\eta^n)$ such that $M_p(\eta^n) M_p^+(\eta^n)$ maps the column vectors of $M_p(\eta^n)$ to themselves, i.e., $M_p(\eta^n) M_p^+(\eta^n) M_p(\eta^n) = M_p(\eta^n)$. Then, $\mathbf{v} = M_p^+(\eta^n) \nabla_\eta \widehat{E}(\theta^n, \eta^n, \psi_n)$ will be a solution to (23). Thus, we can also set

$$\Delta \eta = M_p^+(\eta^n) \nabla_\eta \widehat{E}(\theta^n, \eta^n, \psi_n).$$

In this research, we pick $M_p^+(\eta^n)$ as the unique Moore-Penrose inverse to simplify our discussion.

Moreover, we utilize the same idea to update the parameters ξ and θ , such that

$$\begin{aligned}\xi^{n+1} &= \xi^n + \tau_\varphi \cdot M_{bdd}(\xi^n)^\dagger \nabla_\xi \widehat{E}(\theta^n, \eta^n, \xi^n), \\ \theta^{n+1} &= \theta^n - \tau_u \cdot M_p(\theta^n)^\dagger \nabla_\theta \mathcal{E}(u_{\theta^n}, \tilde{\varphi}_{n+1}, \tilde{\psi}_{n+1}),\end{aligned}$$

where $\tilde{\varphi}_{n+1} = \varphi_{\eta^{n+1}} + \omega(\varphi_{\eta^{n+1}} - \varphi_{\eta^n})$, $\tilde{\psi}_{n+1} = \psi_{\xi^{n+1}} + \omega(\psi_{\xi^{n+1}} - \psi_{\xi^n})$ are obtained via the extrapolation. The Gram type matrices $M_p(\theta)$, $M_{bdd}(\xi)$ are computed as

$$M_p(\theta) = \int_{\Omega} \frac{\partial}{\partial \theta} \mathcal{M}_p u_\theta(x)^\top \frac{\partial}{\partial \theta} \mathcal{M}_p u_\theta(x) d\mu + \int_{\partial\Omega} \frac{\partial}{\partial \theta} \mathcal{B} u_\theta(y)^\top \frac{\partial}{\partial \theta} \mathcal{B} u_\theta(y) d\mu_{\partial\Omega}. \quad (24)$$

$$M_{bdd}(\xi) = \int_{\partial\Omega} \frac{\partial \psi_\xi(y)}{\partial \xi}^\top \frac{\partial \psi_\xi(y)}{\partial \xi} d\mu_{\partial\Omega}, \quad (25)$$

This yields our NPDG algorithm (4).

In practice, choosing the stepsize τ_φ (as well as τ_ψ , τ_u) ranging from 10^{-2} to 10^{-1} usually yields stable and efficient performance of this explicit scheme. The study on the optimal choice of the stepsizes, as well as the application of more meticulous line search strategies will serve as the future research directions.

2.5 Nonlinear Equations

A similar treatment can be applied to the nonlinear equation taking the form of

$$\mathcal{L}u + \mathcal{N}u = f, \quad \text{on } \Omega, \quad \mathcal{B}u = g \quad \text{on } \partial\Omega, \quad (26)$$

where $\mathcal{L}$, $\mathcal{N}$ denote the linear and nonlinear operators, respectively. $\mathcal{B}$ is the boundary operator. $f : \Omega \rightarrow \mathbb{R}$, $g : \partial\Omega \rightarrow \mathbb{R}$. Suppose that $\mathcal{L}$ splits as $\mathcal{L} = \mathcal{M}_d^* \tilde{\mathcal{L}} \mathcal{M}_p$. We then multiply the equation and its boundary condition with the dual variables φ, ψ and derive the functional

$$\begin{aligned}\mathcal{E}(u, \varphi, \psi) &= \langle \tilde{\mathcal{L}} \mathcal{M}_p u, \mathcal{M}_d \varphi \rangle_{L^2(\Omega; \mathbb{R}^r)} + \langle \mathcal{N}(u), \varphi \rangle_{L^2(\Omega)} - \langle f, \varphi \rangle_{L^2(\Omega)} + \langle \mathcal{B}u, \psi \rangle_{L^2(\partial\Omega)} \\ &\quad - \frac{\epsilon}{2} \left(\|\mathcal{M}_d \varphi\|_{L^2(\Omega; \mathbb{R}^r)}^2 + \|\psi\|_{L^2(\partial\Omega)}^2 \right).\end{aligned}$$

We can now apply algorithm (4) with preconditioning matrices M_p, M_d, M_{bdd} mentioned above in (24), (21), (25) to solve the equation. Related numerical examples and more detailed descriptions of our treatment can be found in section 5.3 and 5.4.

2.5.1 Monge-Ampère equation

The algorithm can readily handle some fully nonlinear equation that possesses a saddle point formulation, such as the Monge-Ampère equation.

$$|\det(D^2 u(x))| = \frac{\rho_0(x)}{\rho_1(\nabla u(x))}, \quad \rho_0 dx - a.e., \quad u \text{ is convex on } \mathbb{R}^d. \quad (27)$$

Here, ρ_0, ρ_1 are probability density functions. $D^2 u$ denotes the Hessian matrix of the potential function u . This equation takes an equivalent form of

$$\nabla u \# \mu_0 = \mu_1, \quad u \text{ is convex on } \mathbb{R}^d,$$

where μ_0, μ_1 are probability distributions. We assume μ_0, μ_1 are absolutely continuous with respect to the Lebesgue measure on $\mathbb{R}^d$, with density functions ρ_0, ρ_1 . And $\#$ denotes the “pushforward” of probability distribution μ_0 by the map ∇u in the sense of

$$\int_{\mathbb{R}^d} h(\nabla u(x)) d\mu_0(x) = \int_{\mathbb{R}^d} h(y) d\mu_1(y), \quad \text{for any measurable function } h \text{ defined on } \mathbb{R}^d.$$

There is already adequate research on the classical numerical methods for the Monge-Ampère equation. We refer the readers to [9, 28, 10, 66] and the references therein for further discussion.

In this research, we aim to propose a mesh-free algorithm based on the data samples drawn from ρ_0 and ρ_1 to evaluate $\nabla u(\cdot)$ of the equation. We should first point out that the Monge-Ampère equation is closely related to the following Optimal Transport (OT) problem (also known as the Monge problem)[89][20],

$$\min_{\substack{T \in \mathcal{M}(\mathbb{R}^d, \mathbb{R}^d) \\ T_{\#}\mu_0 = \mu_1}} \int_{\mathbb{R}^d} \frac{1}{2} \|x - T(x)\|^2 d\mu_0(x). \quad (28)$$

Here $\mathcal{M}(\mathbb{R}^d, \mathbb{R}^d)$ denotes the space of measurable maps from $\mathbb{R}^d$ to $\mathbb{R}^d$. We aim at computing for the optimal map T that transport the probability distribution ρ_0 to ρ_1 by minimizing the L^2 transportation cost. One can show that the optimal map T_* of (28) exists uniquely as long as μ_0, μ_1 possesses densities ρ_0, ρ_1 , and there exists a convex function $u : \mathbb{R}^d \rightarrow \mathbb{R}$ such that $T_*(x) = \nabla u(x)$ for μ_0 -a.e. $x \in \mathbb{R}^d$. Furthermore, if μ_0, μ_1 are supported on bounded smooth open sets $X, Y \subset \mathbb{R}^d$, and ρ_0, ρ_1 are bounded away from zero and infinity on X and Y , then the potential u solves the Monge-Ampère equation (27).

Given the connection between the Monge-Ampère equation and the OT problem, we mainly focus on computing (28) instead of (27). The goal is to compute for the OT map T_* (or ∇u). Notice that (28) is a constrained optimization problem. By denoting $\mathcal{C}_b(\mathbb{R}^d)$ as the space of bounded continuous functions, it is natural to introduce the Lagrange multiplier $\varphi \in \mathcal{C}_b(\mathbb{R}^d)$ (also known as the Kantorovich potential of the OT problem) to the constraint $T_{\#}\rho_0 = \rho_1$, and obtain

$$\mathcal{E}(T, \varphi) = \int_{\mathbb{R}^d} \frac{1}{2} \|x - T(x)\|^2 d\mu_0(x) + \int_{\mathbb{R}^d} \varphi(T(x)) d\mu_0(x) - \int_{\mathbb{R}^d} \varphi(y) d\mu_1(y). \quad (29)$$

Upon solving (28), we consider the sup-inf saddle point problem

$$\sup_{\varphi \in \mathcal{C}_b(\mathbb{R}^d)} \inf_{T \in \mathcal{M}(\mathbb{R}^d, \mathbb{R}^d)} \mathcal{E}(T, \varphi). \quad (30)$$

It is shown in [26] that, as long as μ_0, μ_1 are compactly supported, and μ_0 is absolutely continuous w.r.t. Lebesgue measure, the saddle point (T_*, φ_*) of (30) exists, and the map $T_*(\cdot)$ equals to the OT map $T_*(\cdot)$ μ_0 -almost surely.

In the computation, we substitute T, φ with neural networks T_θ, φ_η . A natural way of preconditioning this problem is to set $\mathcal{M}_p = \text{Id}$ and $\mathcal{M}_d = \text{Id}$ for $M_p(\theta), M_d(\eta)$, i.e.,

$$\begin{aligned} M_p(\theta) &= \int_{\mathbb{R}^d} \frac{\partial T_\theta(x)}{\partial \theta}^\top \frac{\partial T_\theta(x)}{\partial \theta} \rho_0(x) dx, \\ M_d(\eta; \theta) &= \int_{\mathbb{R}^d} \frac{\partial \varphi_\eta(T_\theta(x))}{\partial \eta} \frac{\partial \varphi_\eta(T_\theta(x))}{\partial \eta}^\top \rho_0(x) dx. \end{aligned} \quad (31)$$

However, a more canonical choice is to set $\mathcal{M}_p = \text{Id}$ and $\mathcal{M}_d = \nabla$. To motivate this preconditioning technique, we carry out the following calculation. Suppose (T_*, φ_*) is the saddle point of the above problem (30). As $T_*(\cdot) = T_*(\cdot)$ μ_0 -almost surely; one can show that $T_{\#}\mu_0 = \mu_1$, and

$$T_*(x) - x + \nabla \varphi_*(T_*(x)) = 0, \quad \mu_0 - a.s. \quad (32)$$

Now assume that (T, φ) is close to the optimal solution (T_*, φ_*) . We have

$$\begin{aligned}
\mathcal{E}(T, \varphi) &= \int_{\mathbb{R}^d} \frac{1}{2} \|T(x) - T_*(x) + T_*(x) - x\|^2 \rho_0(x) dx + \int_{\mathbb{R}^d} (\varphi(T(x)) - \varphi(T_*(x))) \rho_0(x) dx \\
&= \int_{\mathbb{R}^d} \left(\frac{1}{2} \|T_*(x) - x\|^2 + \langle T(x) - T_*(x), T_*(x) - x \rangle + \frac{1}{2} \|T(x) - T_*(x)\|^2 \right) \rho_0(x) dx \\
&\quad + \int_{\mathbb{R}^d} (\langle \nabla \varphi(T(x)), T(x) - T_*(x) \rangle + \frac{1}{2} \langle T(x) - T_*(x), \nabla^2 \varphi(\xi)(T(x) - T_*(x)) \rangle) \rho_0(x) dx \\
&= \int_{\mathbb{R}^d} (\langle T(x) - T_*(x), T_*(x) - x \rangle + \langle \nabla \varphi(T(x)), T(x) - T_*(x) \rangle) \rho_0(x) dx \\
&\quad + \int_{\mathbb{R}^d} \frac{1}{2} \|T_*(x) - x\|^2 \rho_0(x) dx + \mathcal{O}(\|T - T_*\|^2) \\
&= \int_{\mathbb{R}^d} \langle T_*(x) - x + \nabla \varphi(T(x)), T(x) - T_*(x) \rangle \rho_0(x) dx + \text{Const} + \mathcal{O}(\|T - T_*\|^2). \tag{33}
\end{aligned}$$

Here we denote $\xi = (1 - \theta)T(x) + \theta T_*(x)$ with $0 \leq \theta \leq 1$.

Recall the optimality relation (32). We can reformulate the first term of (33) as

$$\begin{aligned}
&\int_{\mathbb{R}^d} \langle x - \nabla \varphi_*(T_*(x)) - x + \nabla \varphi(T(x)), T(x) - T_*(x) \rangle \rho_0(x) dx \\
&= \int_{\mathbb{R}^d} \langle \nabla \varphi(T(x)) - \nabla \varphi_*(T_*(x)), T(x) - T_*(x) \rangle \rho_0(x) dx. \tag{34}
\end{aligned}$$

Now (34) suggests that as (T, φ) approaches the optimal solution (T_*, φ_*) , the loss functional $\mathcal{E}(T, \varphi)$ is roughly the $L^2(\rho_0)$ inner product between $\nabla \varphi(T(\cdot)) - \nabla \varphi_*(T_*(\cdot))$ and $T(\cdot) - T_*(\cdot)$. This suggests setting $\mathcal{M}_p = \text{Id}, \mathcal{M}_d = \nabla$ as the preconditioning of (30), i.e.,

$$\begin{aligned}
M_p(\theta) &= \int_{\mathbb{R}^d} \frac{\partial T_\theta(x)}{\partial \theta}^\top \frac{\partial T_\theta(x)}{\partial \theta} \rho_0(x) dx, \\
M_d(\eta; \theta) &= \int_{\mathbb{R}^d} \sum_{k=1}^d \frac{\partial}{\partial \eta} [\partial_{x_k} \varphi_\eta(T_\theta(x))] \frac{\partial}{\partial \eta} [\partial_{x_k} \varphi_\eta(T_\theta(x))]^\top \rho_0(x) dx. \tag{35}
\end{aligned}$$

Applying (4) to the adversarial training of T_θ, φ_η leads to a faster, and more robust algorithm for approaching the saddle point (T_*, φ_*) , which also yields the desired OT map (solution to Monge Ampère equation) T_* (∇u). We refer the readers to section 5.5 for details on implementation and numerical examples.

Remark 3. *It is worth mentioning that the idea of optimizing φ with respect to the H^1 metric has already been introduced in [38], in which the authors introduce a back-and-forth algorithm with the H^1 -natural gradients to deal with the Kantorovich dual problem of (28).*

3 Convergence Analysis of the NPDHG flow

In this section, we are inspired by the research conducted in [53] to provide a posterior convergence analysis on the time-continuous version of the NPDHG algorithm (4) as $\tau_p, \tau_d \rightarrow 0$ and $\omega \tau_d \rightarrow \gamma > 0$.

Recall that (4) can be reformulated as

$$\begin{aligned}
&\left(\begin{pmatrix} \eta^{n+1} \\ \xi^{n+1} \end{pmatrix} - \begin{pmatrix} \eta^n \\ \xi^n \end{pmatrix} \right) / \tau_\varphi = \begin{pmatrix} M_p(\eta^n)^\dagger \nabla_\eta \mathcal{E}(u_{\theta^n}, \varphi_{\eta^n}, \psi_{\xi^n}) \\ M_{dd}(\xi^n)^\dagger \nabla_\xi \mathcal{E}(u_{\theta^n}, \varphi_{\eta^n}, \psi_{\xi^n}) \end{pmatrix}, \\
&\begin{pmatrix} \tilde{\varphi}_{n+1} \\ \tilde{\psi}_{n+1} \end{pmatrix} = \begin{pmatrix} \varphi_{\eta^{n+1}} \\ \psi_{\xi^{n+1}} \end{pmatrix} + \omega \tau_\varphi \left(\begin{pmatrix} \varphi_{\eta^{n+1}} \\ \psi_{\xi^{n+1}} \end{pmatrix} - \begin{pmatrix} \varphi_{\eta^n} \\ \psi_{\xi^n} \end{pmatrix} \right) / \tau_\varphi, \\
&\frac{\theta^{n+1} - \theta^n}{\tau_u} = -M_p(\theta^n)^\dagger \nabla_\theta \mathcal{E}(u_{\theta^n}, \tilde{\varphi}_{n+1}, \tilde{\psi}_{n+1}).
\end{aligned}$$

By replacing the finite differences by the time derivatives, we verify that (4) converges to

$$\begin{aligned}\dot{\eta}_t &= M_d(\eta_t)^\dagger \nabla_{\eta} \mathcal{E}(u_{\theta_t}; \varphi_{\eta_t}, \psi_{\xi_t}), \\ \dot{\xi}_t &= M_{bdd}(\xi_t)^\dagger \nabla_{\xi} \mathcal{E}(u_{\theta_t}, \varphi_{\eta_t}, \psi_{\xi_t}), \\ \dot{\theta}_t &= -M_p(\theta_t)^\dagger \nabla_{\theta} \mathcal{E}(u_{\theta_t}; \tilde{\varphi}_t, \tilde{\psi}_t),\end{aligned}\tag{36}$$

where we denote

$$\begin{pmatrix} \tilde{\varphi}_t \\ \tilde{\psi}_t \end{pmatrix} = \begin{pmatrix} \varphi_{\eta_t} \\ \psi_{\xi_t} \end{pmatrix} + \gamma \begin{pmatrix} \dot{\varphi}_{\eta_t} \\ \dot{\psi}_{\xi_t} \end{pmatrix},\tag{37}$$

as $\tau_u, \tau_\varphi \rightarrow 0$ and $\omega\tau_\varphi \rightarrow \gamma$. We call the above time-continuous dynamic (36) as the **NPDHG flow**. In this section, we analyze the convergence of the numerical solution $\{u_{\theta_t}\}$ along (36).

3.1 Natural gradient induces orthogonal projections of Fréchet derivatives

Before our discussion, we need the following lemma. Similar results have already been proved in several references, including [51, 69, 94, 65, 101]. We restate the lemma here for the sake of completeness.

Lemma 1. *Given a certain Hilbert space $\mathbb{X}$, we consider a Fréchet differentiable functional $\mathcal{F} : \mathbb{X} \rightarrow \mathbb{R}$. Suppose $\Theta \subseteq \mathbb{R}^m$ denotes the parameter space, we consider a parametrized family of functions $\{u_\theta\}_{\theta \in \Theta}$ which belong to $\mathbb{X}$. We denote $D_u \mathcal{F}(u) \in (\mathbb{X})^* = \mathbb{X}$ as the Fréchet derivative at u . Assume that u_θ is differentiable with respect to θ and $\frac{\partial u_\theta}{\partial \theta_i} \in \mathbb{X}$ for arbitrary $1 \leq i \leq m$, $\theta \in \Theta$. We define the $m \times m$ Gram matrix $M(\theta)$ as*

$$(M(\theta))_{ij} = \left\langle \frac{\partial u_\theta}{\partial \theta_i}, \frac{\partial u_\theta}{\partial \theta_j} \right\rangle_{\mathbb{X}}, \quad 1 \leq i, j \leq m.$$

Furthermore, we denote $F(\theta) = \mathcal{F}(u_\theta)$. Then one can show that

- $\nabla_\theta F(\theta) \in \text{Ran}(M(\theta))$,
- For any $\mathbf{v} \in \mathbb{R}^m$ such that $M(\theta)\mathbf{v} = \nabla_\theta F(\theta)$, we can show that $\mathbf{v}$ is the solution to the following least square problem¹.

$$\mathbf{v} \in \underset{\zeta \in \mathbb{R}^m}{\text{argmin}} \left\{ \left\| D_u \mathcal{F}(u_\theta) - \frac{\partial u_\theta}{\partial \theta} \zeta \right\|_{\mathbb{X}}^2 \right\} = \underset{\zeta \in \mathbb{R}^m, \zeta_1, \dots, \zeta_m \in \mathbb{R}}{\text{argmin}} \left\{ \left\| D_u \mathcal{F}(u_\theta) - \sum_{i=1}^m \zeta_i \frac{\partial u_\theta}{\partial \theta_i} \right\|_{\mathbb{X}}^2 \right\}.$$

One can also verify that

$$D_u \mathcal{F}(u_\theta) - \frac{\partial u_\theta}{\partial \theta} \mathbf{v}$$

as a vector in $\mathbb{X}$, is orthogonal (w.r.t. inner product defined on $\mathbb{X}$) to the subspace spanned by $\{\frac{\partial u_\theta}{\partial \theta_1}, \dots, \frac{\partial u_\theta}{\partial \theta_m}\}$. Or equivalently, $\frac{\partial u_\theta}{\partial \theta} \mathbf{v}$ is the orthogonal projection of $D_u \mathcal{F}(u_\theta)$ on $\text{span}\{\frac{\partial u_\theta}{\partial \theta_1}, \dots, \frac{\partial u_\theta}{\partial \theta_m}\}$.

We defer the proof of this lemma to Appendix B.

We should mention that the Moore-Penrose inverse $M(\theta)^\dagger \nabla_\theta F(\theta)$ yields a solution to the least square problem mentioned above. For the convenience of our future discussion, we denote the orthogonal projection (w.r.t. inner product on $\mathbb{X}$) onto $\text{span}\{\frac{\partial u_\theta}{\partial \theta_1}, \dots, \frac{\partial u_\theta}{\partial \theta_m}\}$ as $\Pi_{\partial_\theta u_\theta} : \mathbb{X} \rightarrow \mathbb{X}$, we thus have

$$\frac{\partial u_\theta}{\partial \theta} M(\theta)^\dagger \nabla_\theta F(\theta) = \Pi_{\partial_\theta u_\theta} [D_u \mathcal{F}(u_\theta)].$$

Correspondingly, we denote the orthogonal projection onto the orthogonal complement of $\text{span}\{\frac{\partial u_\theta}{\partial \theta_1}, \dots, \frac{\partial u_\theta}{\partial \theta_m}\}$ as $\Pi_{\partial_\theta u_\theta^\perp} : \mathbb{X} \rightarrow \mathbb{X}$, we have,

$$D_u \mathcal{F}(u_\theta) - \frac{\partial u_\theta}{\partial \theta} M(\theta)^\dagger \nabla_\theta F(\theta) = \Pi_{\partial_\theta u_\theta^\perp} [D_u \mathcal{F}(u_\theta)].$$

¹It is worth mentioning that for fixed x , $\frac{\partial u_\theta(x)}{\partial \theta}$ is a $k \times m$ matrix.

3.2 Convergence analysis of the time-continuous version of the algorithm for linear PDEs

At first, we assume that $\Omega \subset \mathbb{R}^d$ is a bounded open set. Denote $\mathfrak{B}$ as the Borel algebra on Ω inherited from that of $\mathbb{R}^d$. We denote μ as a Borel measure on Ω . Furthermore, $L^2(\Omega)$ denotes $L^2(\Omega, \mathfrak{B}, \mu)$; $L^2(\Omega; \mathbb{R}^r)$ denotes $L^2(\Omega, \mathfrak{B}, \mu; \mathbb{R}^r)$ for simplicity. We suppose the boundary $\partial\Omega \in C^1$. We denote $\mathfrak{B}_{\partial\Omega}$ as the Borel algebra on $\partial\Omega$, and $\mu_{\partial\Omega}$ as a Borel measure on $\partial\Omega$. We denote $L^2(\partial\Omega)$ as $L^2(\partial\Omega, \mathfrak{B}_{\partial\Omega}, \mu_{\partial\Omega})$ for shorthand.

Recall that we consider the linear equation (11) defined on $\mathbb{H}$. We assume u_* as a real solution to (11). We will adopt the notations used in previous section 2.2 and 2.3. In our discussion, we always assume that the operator $\tilde{\mathcal{L}}$ is bounded from both above and below in the sense of

$$0 < L_0 \leq \inf_{\mathbf{u} \in \tilde{\mathbb{H}}} \frac{\|\tilde{\mathcal{L}}\mathbf{u}\|_{L^2(\Omega; \mathbb{R}^r)}}{\|\mathbf{u}\|_{L^2(\Omega; \mathbb{R}^r)}} \leq \sup_{\mathbf{u} \in \tilde{\mathbb{H}}} \frac{\|\tilde{\mathcal{L}}\mathbf{u}\|_{L^2(\Omega; \mathbb{R}^r)}}{\|\mathbf{u}\|_{L^2(\Omega; \mathbb{R}^r)}} \leq L_1 < \infty. \quad (38)$$

We denote $L_1 \vee 1 = \max\{L_1, 1\}$ and $L_0 \wedge 1 = \min\{L_0, 1\}$, and

$$\tilde{\kappa} = \frac{L_1 \vee 1}{L_0 \wedge 1} \quad (39)$$

for shorthand.

Suppose that we perform the NPDHG flow up to a time T . We denote $\alpha, \beta_1, \beta_2 \in [0, 1]$ as coefficients quantifying the approximation power of the subspaces spanned by $\left\{ \left(\frac{\partial \mathcal{M}_p u_{\theta_t}}{\partial \theta_k}, \sqrt{\lambda} \frac{\partial \mathcal{B} u_{\theta_t}}{\partial \theta_k} \right) \right\}_{1 \leq k \leq m_\theta}$, $\left\{ \frac{\partial \mathcal{M}_d \varphi_{\eta_t}}{\partial \eta_k} \right\}_{1 \leq k \leq m_\eta}$, and $\left\{ \frac{\partial \psi_\xi}{\partial \xi_k} \right\}_{1 \leq k \leq m_\xi}$ for $t \in [0, T]$. To be more specific, α is a constant satisfying

$$\begin{aligned} \min_{\substack{\zeta \in \mathbb{R}^{m_\theta} \\ \zeta_1, \dots, \zeta_{m_\theta} \in \mathbb{R}}} \left\{ \left\| \sum_{k=1}^{m_\theta} \zeta_k \frac{\partial \mathcal{M}_p u_{\theta_t}}{\partial \theta_k} - \mathcal{M}_p(u_{\theta_t} - u_*) \right\|_{L^2(\Omega; \mathbb{R}^r)}^2 + \left\| \sum_{k=1}^{m_\theta} \zeta_k \sqrt{\lambda} \frac{\partial \mathcal{B} u_{\theta_t}}{\partial \theta_k} - \sqrt{\lambda} \mathcal{B}(u_{\theta_t} - u_*) \right\|_{L^2(\partial\Omega)}^2 \right\} \\ \leq \alpha^2 (\|\mathcal{M}_p(u_{\theta_t} - u_*)\|_{L^2(\Omega; \mathbb{R}^r)}^2 + \|\sqrt{\lambda} \mathcal{B}(u_{\theta_t} - u_*)\|_{L^2(\partial\Omega)}^2), \quad \text{for all } t \in [0, T]. \end{aligned}$$

Recall that $\mathbb{L}^2 = L^2(\Omega; \mathbb{R}^r) \times L^2(\partial\Omega)$, we denote the subspace

$$\partial_\theta \mathbf{U}_\theta = \text{span} \left\{ \left(\frac{\partial \mathcal{M}_p u_\theta}{\partial \theta_k}, \sqrt{\lambda} \frac{\partial \mathcal{B} u_\theta}{\partial \theta_k} \right) \right\}_{k=1}^{m_\theta} \subset \mathbb{L}^2.$$

Then, α quantifies the relative $\mathbb{L}^2$ norm of the $\partial_\theta \mathbf{U}_{\theta_t}$ -orthogonal component of $(\mathcal{M}_p(u_{\theta_t} - u_*), \mathcal{B}(u_{\theta_t} - u_*))$ on $t \in [0, T]$. Similarly, we denote the subspace

$$\partial_{\eta, \xi} \Phi_{\eta, \xi} = \text{span} \left\{ \left(\frac{\partial \mathcal{M}_d \varphi_\eta}{\partial \eta_k}, 0 \right) \right\}_{k=1}^{m_\eta} \oplus \text{span} \left\{ \left(0, \sqrt{\lambda} \frac{\partial \psi_\xi}{\partial \xi_k} \right) \right\}_{k=1}^{m_\xi} \subset \mathbb{L}^2.$$

Then, β_1, β_2 denote the relative $\mathbb{L}^2$ norms of $\partial_{\eta, \xi} \Phi_{\eta_t, \xi_t}$ -orthogonal component of $(\tilde{\mathcal{L}} \mathcal{M}_p u_{\theta_t}, \sqrt{\lambda} \mathcal{B} u_{\theta_t})$ and $(\mathcal{M}_d \varphi_{\eta_t}, \sqrt{\lambda} \psi_{\xi_t})$, respectively. The detailed definitions of α, β_1, β_2 can be found later in (55), (56) and (57).

The following Theorem analyzes the convergence of the numerical solution u_{θ_t} solved from (36) on $[0, T]$.

Theorem 2 (A posterior convergence analysis of NPDHG flow). *Suppose $\{(\theta_t, \eta_t, \xi_t)\}$ solves the NPDHG flow (36) on $[0, T]$. Recall that α, β_1, β_2 quantifies the approximation quality of neural networks $u_{\theta_t}, \varphi_{\eta_t}, \psi_{\xi_t}$ through $[0, T]$, and $\tilde{\kappa}$ denotes the condition number (39). Suppose $\alpha + \beta_1 < \frac{1}{\tilde{\kappa}^2}$, $\beta_2 < 1$, if we further assume that the hyperparameters of the NPDHG flow $\gamma, \epsilon > 0$ satisfies*

$$\left(\frac{1}{\tilde{\kappa}^2} - (\alpha + \beta_1) \right) \cdot (1 - \beta_2) > \frac{((1 + \beta_1)\gamma\epsilon + \beta_2 + \alpha|1 - \gamma\epsilon|)^2}{4\gamma\epsilon}. \quad (40)$$

Then there exists a constant $r > 0$, such that

$$\|\mathcal{M}_p(u_{\theta_t} - u_*)\|_{L^2(\Omega; \mathbb{R}^r)}^2 + \lambda \|\mathcal{B}(u_{\theta_t} - u_*)\|_{L^2(\partial\Omega)}^2 \leq 2 \exp(-rt) \cdot C_0 \quad \text{for } 0 \leq t \leq T.$$

Here $C_0 \geq 0$ is a constant depending on the initial value $(\theta_0, \eta_0, \xi_0)$ of the NPDHG flow. We note that $r > 0$ is the convergence rate depending on the equation (11), the hyperparameter γ , and the neural network parameters. The explicit form of r is provided in (58).

The proof of Theorem 2 is provided in Appendix B.

It is also worth mentioning that as t increases, u_{θ_t} will approach the real solution u_* ; however, as the approximation gets better, the error term $(\mathcal{M}_p(u_{\theta_t} - u_*), \mathcal{B}(u_{\theta_t} - u_*))$ also erects orthogonally away from the exploration space $\partial_\theta \mathcal{U}_{\theta_t}$. Consequently, the quantity α will approach 1, so as $\beta_1 \rightarrow 1$. That may prevent condition $\alpha + \beta_1 < \frac{1}{\tilde{\kappa}^2}$ at a certain time t along the NPDHG flow (recall that $\tilde{\kappa} > 1$). As a result, the convergence analysis presented in Theorem 2 is only applicable on a finite time interval. We leave the analysis of the largest time T that guarantees the convergence of $\{u_{\theta_t}\}$ as the future research direction.

Remark 4. If we assume that $u_\theta(\cdot), \varphi_\eta(\cdot), \psi_\xi(\cdot)$ are linear combinations of basis functions, i.e.,

$$u_\theta(x) = \sum_{k=1}^{m_\theta} \theta_k u_k(x), \quad \varphi_\eta(x) = \sum_{k=1}^{m_\eta} \eta_k \varphi_k(x), \quad \psi_\xi(x) = \sum_{k=1}^{m_\xi} \xi_k \psi_k(x),$$

with θ, η, ξ serving as the coefficients of the basis functions and u_k, φ_k, ψ_k being given basis functions in their corresponding spaces. If u_* can further be represented by linear combination of $\{u_k\}_{k=1}^{m_\theta}$, we will have $\alpha = \beta_2 = 0$. And the inequality (40) reduces to

$$\gamma\epsilon \leq \frac{4(\tilde{\kappa}^{-2} - \beta_1)}{(1 + \beta_1)^2}.$$

4 Algorithm

In this section, we provide a detailed description of how we implement the NPDG algorithm (4). We take the linear PDE (11) as an illustrative example.

4.1 Loss functional and the precondition matrices

Recall our discussions in section 2.3. We introduce the pair of dual neural networks (φ_η, ψ_ξ) to equation (11) and consider the loss functional

$$\begin{aligned} \mathcal{E}(u, \varphi, \psi) = & \left(\int_{\Omega} \tilde{\mathcal{L}} \mathcal{M}_p u(x) \cdot \mathcal{M}_d \varphi(x) - f(x) \varphi(x) \, d\mu(x) - \frac{\epsilon}{2} \int_{\Omega} \mathcal{M}_d \varphi(x) \cdot \mathcal{M}_d \varphi(x) \, d\mu(x) \right) \\ & + \lambda \left(\int_{\partial\Omega} (\mathcal{B}u(y) - g(y)) \cdot \psi(y) \, d\mu_{\partial\Omega}(y) - \frac{\epsilon}{2} \int_{\partial\Omega} \psi(y) \cdot \psi(y) \, d\mu_{\partial\Omega}(y) \right). \end{aligned}$$

We denote $\mu, \mu_{\partial\Omega}$ as the uniform probability distribution on Ω and $\partial\Omega$. Sometimes, it also helps if we add the L^2 boundary loss

$$\|\mathcal{B}u - g\|_{L^2(\mu_{\partial\Omega})}^2 = \int_{\partial\Omega} (\mathcal{B}u - g)^2 \, d\mu_{\partial\Omega}$$

into $\mathcal{E}(u, \varphi, \psi)$, and consider

$$\tilde{\mathcal{E}}(u, \varphi, \psi) = \mathcal{E}(u, \varphi, \psi) + \lambda \|\mathcal{B}u - g\|_{L^2(\mu_{\partial\Omega})}^2.$$

We also recall that the precondition matrices $M_p(\theta) \in \mathbb{R}^{m_\theta \times m_\theta}, M_d(\eta) \in \mathbb{R}^{m_\eta \times m_\eta}, M_{bd}(\xi) \in \mathbb{R}^{m_\xi \times m_\xi}$ are defined in (24), (21) and (25).

4.2 Monte-Carlo approximation

We apply the Monte Carlo algorithm to approximate the loss function $\mathcal{E}(u_\theta; \varphi_\eta, \psi_\xi)$ throughout our computation. Assume that $\{\mathbf{X}_i\}_{1 \leq i \leq N_{in}}$, $\{\mathbf{Y}_j\}_{1 \leq j \leq N_{bdd}}$ are samples uniformly drawn from the domain Ω and its boundary $\partial\Omega$, respectively. Under the scenario of linear PDE (11), we compute

$$\begin{aligned} \mathcal{E}(u_\theta; \varphi_\eta, \psi_\xi) \approx & \frac{1}{N_{in}} \sum_{i=1}^{N_{in}} \tilde{\mathcal{L}} \mathcal{M}_p u(\mathbf{X}_i) \cdot \mathcal{M}_d \varphi(\mathbf{X}_i) - f(\mathbf{X}_i) \varphi(\mathbf{X}_i) - \frac{\epsilon}{2} \mathcal{M}_d \varphi_\eta(\mathbf{X}_i) \cdot \mathcal{M}_d \varphi_\eta(\mathbf{X}_i) \\ & + \lambda \left(\frac{1}{N_{bdd}} \sum_{j=1}^{N_{bdd}} (\mathcal{B}u(\mathbf{Y}_j) - g(\mathbf{Y}_j)) \cdot \psi_\xi(\mathbf{Y}_j) - \frac{\epsilon}{2} \psi_\xi(\mathbf{Y}_j) \cdot \psi_\xi(\mathbf{Y}_j) \right). \end{aligned}$$

Here, “ $\cdot$ ” denotes the multiplication of scalars or the inner product of vectors. For example, if $\mathcal{L} = \Delta$, $\mathcal{B}$ is the restriction of u on $\partial\Omega$, and we set $\mathcal{M}_p = \mathcal{M}_d = \nabla$, $\tilde{\mathcal{L}} = \text{Id}$, then

$$\begin{aligned} \tilde{\mathcal{L}} \mathcal{M}_p u(\mathbf{X}_i) \cdot \mathcal{M}_d \varphi(\mathbf{X}_i) &= \nabla u(\mathbf{X}_i) \cdot \nabla \varphi(\mathbf{X}_i), \quad \mathcal{M}_d \varphi_\eta(\mathbf{X}_i) \cdot \mathcal{M}_d \varphi_\eta(\mathbf{X}_i) = \|\nabla \varphi(\mathbf{X}_i)\|^2, \\ (\mathcal{B}u(\mathbf{Y}_j) - g(\mathbf{Y}_j)) \cdot \psi_\xi(\mathbf{Y}_j) - \frac{\epsilon}{2} \psi_\xi(\mathbf{Y}_j) \cdot \psi_\xi(\mathbf{Y}_j) &= (u(\mathbf{Y}_j) - g(\mathbf{Y}_j)) \psi_\xi(\mathbf{Y}_j) - \frac{\epsilon}{2} \psi_\xi^2(\mathbf{Y}_j). \end{aligned}$$

For general nonlinear PDE, the loss function $\mathcal{E}(u_\theta; \varphi_\eta, \psi_\xi)$ can also be approximated via the Monte-Carlo algorithm.

Furthermore, it is also straightforward to evaluate the preconditioning matrices $M_p(\theta)$ via Monte Carlo method, for example $M_p(\theta)$ can be computed as ²

$$\begin{aligned} M_p(\theta) &\approx \frac{1}{N_{in}} \sum_{i=1}^{N_{in}} \frac{\partial}{\partial \theta} (\mathcal{M}_p u_\theta(\mathbf{X}_i))^\top \frac{\partial}{\partial \theta} (\mathcal{M}_p u_\theta(\mathbf{X}_i)) + \frac{\lambda}{N_{in}} \sum_{j=1}^{N_{bdd}} \frac{\partial}{\partial \theta} u_\theta(\mathbf{Y}_j)^\top \frac{\partial}{\partial \theta} u_\theta(\mathbf{Y}_j), \\ \text{i.e., } (M_p(\theta))_{ij} &= \frac{1}{N_{in}} \sum_{i=1}^{N_{in}} \frac{\partial \mathcal{M}_p u_\theta(\mathbf{X}_i)}{\partial \theta_i} \cdot \frac{\partial \mathcal{M}_p u_\theta(\mathbf{X}_i)}{\partial \theta_j} + \frac{\lambda}{N_{bdd}} \sum_{j=1}^{N_{bdd}} \frac{\partial u_\theta(\mathbf{Y}_j)}{\partial \theta_i} \frac{\partial u_\theta(\mathbf{Y}_j)}{\partial \theta_j}, \quad \forall 1 \leq i, j \leq m. \end{aligned}$$

It is worth mentioning that we use the same set of samples for computation of both the loss function and the preconditioning matrices.

4.3 Inverting the preconditioning matrices via Krylov iterative solver

We then solve the least square problem (23) for $\mathbf{v}$. As mentioned in Remark 2, this is equivalent to solve the linear equation

$$M_p(\theta_k) \mathbf{v} = \frac{\partial}{\partial \theta} \mathcal{E}(u_{\theta_k}; \varphi_{\eta_k}, \psi_{\xi_k}). \quad (41)$$

However, this may suffer from the limitation on scalability: The method always computes and records the entire preconditioning matrix $M_p(\theta_k)$ at each optimization step. For neural networks such as Multi-Layer Perceptron, $M_p(\theta)$ is generally non-sparse, which suggests that forming this $m \times m$ matrix will occupy immense memory space of the computing resources as the number of parameters of the neural networks increases. For example, in numerical experiment 5.2, we deal with MLP $u_\theta(\cdot)$ with $d_{in} = 50, d_h = 256, d_{out} = 1$ and $n_l = 6$, this neural network contains $m_\theta = 279090$ parameters. Forming such $m_\theta \times m_\theta$ matrix is generally infeasible.

As a mitigation, instead of the direct evaluation of the preconditioning matrices, we apply the MINRES algorithm, which is an iterative solver, to solve (41). The MINRES iterative solver only requires matrix-vector multiplications that can readily avoid the direct formation of the preconditioning matrices. Similar

²Recall that we denote $\frac{\partial}{\partial \theta} \mathcal{M}_p u_\theta(\mathbf{X}_i)$ as the Jacobian of $\mathcal{M}_p u_\theta$ at $\mathbf{X}_i$. For example, if one sets $\mathcal{M}_p = \nabla$, then $\frac{\partial}{\partial \theta} \mathcal{M}_p u_\theta(\mathbf{X}_i)$ is a $d \times m$ matrix. Similarly, we assume $\frac{\partial u_\theta(\mathbf{Y}_j)}{\partial \theta}$ is $1 \times d$.

treatment is also utilized in [21, 59, 76, 74] and the references therein in optimization problems. The same technique is also used in [94, 40] to handle the computation of Wasserstein geometric flows.

We briefly describe how we evaluate $M_p(\theta)\mathbf{v}$ for arbitrary vector $\mathbf{v} \in \mathbb{R}^m$ under the deep learning framework. Given neural network $u_\theta(\cdot) : \mathbb{R}^d \rightarrow \mathbb{R}$ with parameter $\theta \in \mathbb{R}^m$, we make a copy $u_{\theta'}^{\text{copy}}(\cdot)$ by inheriting the architecture of $u_\theta(\cdot)$ and by setting $\theta' = \theta$. We apply auto-differentiation to evaluate $\{\mathcal{M}_p u_\theta(\mathbf{X}_i)\}_{i=1}^{N_{in}}$ and $\{\mathcal{M}_p u_{\theta'}^{\text{copy}}(\mathbf{X}_i)\}_{i=1}^{N_{in}}$, we also evaluate $\{u_\theta(\mathbf{Y}_j)\}_{j=1}^{N_{bdd}}$, $\{u_{\theta'}^{\text{copy}}(\mathbf{Y}_j)\}_{j=1}^{N_{bdd}}$. Then we compute the scalar

$$\Gamma(\theta', \theta) = \frac{1}{N_{in}} \sum_{i=1}^{N_{in}} \mathcal{M}_p u_{\theta'}^{\text{copy}}(\mathbf{X}_i) \cdot \mathcal{M}_p u_\theta(\mathbf{X}_i) + \frac{\lambda}{N_{bdd}} \sum_{j=1}^{N_{bdd}} u_{\theta'}^{\text{copy}}(\mathbf{Y}_j) u_\theta(\mathbf{Y}_j). \quad (42)$$

Now, by applying auto-differentiation again, we take the partial derivative of $\Gamma_{in}(\theta', \theta)$ w.r.t. θ , and making an inner product with $\mathbf{v}$, this yields $\partial_\theta \Gamma_{in}(\theta', \theta)^\top \mathbf{v}$. Finally, taking the partial derivative w.r.t. θ' yields

$$\begin{aligned} \partial_{\theta'}(\partial_\theta \Gamma(\theta', \theta)\mathbf{v}) &= \frac{\partial}{\partial \theta'} \left(\left(\frac{1}{N_{in}} \sum_{i=1}^{N_{in}} \mathcal{M}_p u_{\theta'}^{\text{copy}}(\mathbf{X}_i) \cdot \frac{\partial}{\partial \theta} (\mathcal{M}_p u_\theta(\mathbf{X}_i)) + \frac{\lambda}{N_{bdd}} \sum_{j=1}^{N_{bdd}} u_{\theta'}^{\text{copy}}(\mathbf{Y}_j) \frac{\partial}{\partial \theta} u_\theta(\mathbf{Y}_j) \right) \mathbf{v} \right) \\ &\stackrel{u^{\text{copy}}=u, \theta'=\theta}{=} \left(\frac{1}{N_{in}} \sum_{i=1}^{N_{in}} \frac{\partial}{\partial \theta} (\mathcal{M}_p u_\theta(\mathbf{X}_i))^\top \frac{\partial}{\partial \theta} (\mathcal{M}_p u_\theta(\mathbf{X}_i)) + \frac{\lambda}{N_{bdd}} \sum_{j=1}^{N_{bdd}} \frac{\partial}{\partial \theta} u_\theta(\mathbf{Y}_j) \frac{\partial}{\partial \theta} u_\theta(\mathbf{Y}_j) \right) \mathbf{v} \\ &= M_p(\theta)\mathbf{v}. \end{aligned}$$

This suggests an effective way of evaluating $M_p(\theta)\mathbf{v}$ without forming $M_p(\theta)$ explicitly. We summarize this in the following Algorithm 1.

Algorithm 1 Evaluating $M_p(\theta)\mathbf{v}$

Input: Preconditioning operators $\mathcal{M}_p, \mathcal{M}_d$. Neural network $u_\theta(\cdot)$, samples $\{\mathbf{X}_i\}_{i=1}^{N_{in}} \subset \Omega$, $\{\mathbf{Y}_j\}_{j=1}^{N_{bdd}} \subset \partial\Omega$, vector $\mathbf{v} \in \mathbb{R}^m$.

- 1: Make a copy $u_{\theta'}^{\text{copy}}(\cdot)$ of the given $u_\theta(\cdot)$ with $\theta' = \theta$.
- 2: Evaluate $\Gamma(\theta', \theta)$ as defined in (42).
- 3: Apply auto-differentiation to evaluate $\partial_\theta \Gamma(\theta', \theta)\mathbf{v}$.
- 4: Apply auto-differentiation to evaluate $\mathbf{u} = \partial_{\theta'}(\partial_\theta \Gamma(\theta', \theta)\mathbf{v})$.

Return: $\mathbf{u}$

Similarly, the matrix-vector multiplication involving the preconditioning matrices $M_d(\eta), M_{bdd}(\xi)$ can be computed by using the same technique.

Remark 5. Calculating $M_p(\theta)\mathbf{v}$ can be further simplified by using the finite-difference approximation, which may lead to faster speed and lower memory cost. This technique has been conducted in several Hessian-free optimization algorithms [59, 42, 74]. This possible improvement will serve as the future research directions.

4.4 Sketch of main algorithm

We summarize the proposed method in Algorithm 2.

5 Numerical Examples

In this section, we apply the proposed Natural Primal-Dual Hybrid Gradient (NPDHG) algorithm to various types of PDEs, including linear and nonlinear, static, and time-dependent equations. We denote our method as the NPDG algorithm for simplicity.

Throughout numerical experiments, we set neural networks as Multi-Layer Perceptron (MLP). That is the fully connected neural network with the input dimension d_{in} , the hidden dimension d_{hidden} , the output dimension d_{out} , and the number of layers n_{MLP} . We denote such MLP with activation function f as $MLP_f(d_{in}, d_{hidden}, d_{out}, n_{MLP})$. Readers are referred to Appendix A for further details on MLP.

Algorithm 2 Natural Primal-Dual Hybrid Gradient method (NPDHG)

Input: The equation $F(u, \nabla u, \nabla^2 u, \dots) = 0$ on Ω with (if any) boundary condition $\mathcal{B}u = g$ on $\partial\Omega$. Preconditioning operators $\mathcal{M}_p, \mathcal{M}_d$. The functional $\mathcal{E}(u, \varphi, \psi)$. Stepsizes $\tau_u, \tau_\varphi, \tau_\psi$ of the NPDG algorithm; extrapolation coefficient ω ; Total iteration number of the NPDG algorithm N_{iter} . Number of samples drawn from Ω and $\partial\Omega$: N_{in}, N_{bdd} . Max iteration number n_{MINRES} and tolerance of relative residual tol_{MINRES} of the MINRES algorithm.

- 1: Initialize the primal neural network $u_\theta(\cdot)$, dual neural network(s) $\varphi_\eta(\cdot)$ and $\psi_\xi(\cdot)$ if the equation is equipped with boundary condition(s).
- 2: **for** $iter = 1$ to N_{iter} **do**
- 3: Set $\eta_0 = \eta, \xi_0 = \xi$
- 4: Apply Monte-Carlo algorithm and auto-differentiation to evaluate $(\mathbf{w}_\varphi^\top, \mathbf{w}_\psi^\top)^\top = \partial_{(\eta, \xi)} \mathcal{E}(u_\theta, \varphi_\eta, \psi_\xi)$
- 5: Apply the MINRES algorithm $(n_{\text{MINRES}}, tol_{\text{MINRES}})$ together with Algorithm 1 to solve

$$M_d(\eta) \mathbf{v}_\varphi = \mathbf{w}_\varphi, M_{bdd}(\xi) \mathbf{v}_\psi = \mathbf{w}_\psi$$

- 6: Update $\eta = \eta + \tau_\varphi \mathbf{v}_\varphi, \xi = \xi + \tau_\psi \mathbf{v}_\psi$ ▷ Natural gradient ascent
- 7: Set $\tilde{\varphi} = \varphi_\eta + \omega(\varphi_\eta - \varphi_{\eta_0}), \tilde{\psi} = \psi_\xi + \omega(\psi_\xi - \psi_{\xi_0})$ ▷ Extrpolaion in *functional* space
- 8: Apply Monte-Carlo algorithm and auto-differentiation to evaluate $\mathbf{w}_u = \partial_\theta \mathcal{E}(u_\theta, \tilde{\varphi}, \tilde{\psi})$
- 9: Apply MINRES algorithm $(n_{\text{MINRES}}, tol_{\text{MINRES}})$ together with Algorithm 1 to solve

$$M_p(\theta) \mathbf{v}_u = \mathbf{w}_u$$

- 10: Update $\theta = \theta - \tau_u \mathbf{v}_u$ ▷ Natural gradient descent
 - 11: **end for**
- Return:** $u_\theta(\cdot)$
-

We compare the proposed algorithm with a series of commonly used deep-learning solvers, namely, Physics-Informed Neural Network (PINN) [73], Deep Ritz method [96], and primal-dual-type algorithms for PDEs/optimal transport [97] [26]. We apply Adam [41, 72] and (or) L-BFGS [49, 72] algorithms to PINN. When we use the L-BFGS method, we choose $lr = 1.0$ as the default. The L-BFGS method does not perform stably with the Deep Ritz and primal-dual type methods. We will only apply the Adam algorithm to these two methods. To keep the comparison fair, we keep the same neural network architecture for all the methods tested. We justify the computational efficiency of the proposed methods by summarizing the GPU-time costs of each method for different PDEs with various dimensions in Table 4. The robustness of the proposed method is reflected in the semi-log plots of the relative L^2 -loss for different equations. Necessary plots are also provided to visualize the numerical results produced by the proposed method.

The associated Python code will be available upon request.

5.1 Poisson's equation (10D, 50D)

We consider the following Poisson's equation defined on the region $\Omega = [0, 1]^d$.

$$-\Delta u = f, \text{ on } \Omega, \quad u = g, \text{ on } \partial\Omega. \quad (43)$$

where we define $f(x) = \sum_{k=1}^d \frac{\pi^2}{4} \sin(\frac{\pi}{2} x_k)$, and $u = \sum_{k=1}^d \sin(\frac{\pi}{2} x_k)$ on $\partial\Omega$. The exact solution of this equation is

$$u_*(x) = \sum_{k=1}^d \sin(\frac{\pi}{2} x_k).$$

In this example, by multiplying the dual functions φ and ψ to the equation $-\Delta u = f$, and its boundary condition $u|_{\partial\Omega} = g$, we introduce the loss functional $\mathcal{E} : H^2(\Omega) \times H_0^1(\Omega) \times L^2(\partial\Omega) \rightarrow \mathbb{R}$ as

$$\begin{aligned}\mathcal{E}(u; \varphi, \psi) &= \int_{\Omega} (-\Delta u(x) - f(x))\varphi(x) d\mu(x) - \frac{\epsilon}{2} \int_{\Omega} |\nabla \varphi(x)|^2 d\mu(x) + \lambda \left(\int_{\partial\Omega} (u - g)\psi d\mu_{\partial\Omega} - \frac{\epsilon}{2} \int_{\partial\Omega} \psi^2 d\mu_{\partial\Omega} \right) \\ &= \int_{\Omega} \nabla u(x) \cdot \nabla \varphi(x) - f(x)\varphi(x) d\mu(x) - \frac{\epsilon}{2} \int_{\Omega} |\nabla \varphi(x)|^2 dx + \lambda \left(\int_{\partial\Omega} (u - g)\psi d\mu_{\partial\Omega} - \frac{\epsilon}{2} \int_{\partial\Omega} \psi^2 d\mu_{\partial\Omega} \right).\end{aligned}$$

The second equality in the above derivation is due to the fact that $\int_{\Omega} -\Delta u \varphi dx = -\int_{\partial\Omega} \frac{\partial u}{\partial \mathbf{n}} \varphi d\sigma + \int_{\Omega} \nabla u \cdot \nabla \varphi dx$, and $\varphi = 0$ on $\partial\Omega$. In practice, we discover that it is helpful to add the $L^2(\partial\Omega)$ loss functional to $\mathcal{E}(u, \varphi, \psi)$. Thus, we obtain

$$\tilde{\mathcal{E}}(u; \varphi, \psi) = \mathcal{E}(u; \varphi, \psi) + \lambda \|u - g\|_{L^2(\mu_{\partial\Omega})}^2.$$

In short, we use the functional $\tilde{\mathcal{E}}$. Let us first consider $d = 10$. We substitute u, φ, ψ with MLPs with tanh as activation functions,

$$u_{\theta} = \text{MLP}_{\tanh}(d, 256, 1, 4), \quad \varphi_{\eta} = \text{MLP}_{\tanh}(d, 256, 1, 4) \cdot \zeta, \quad \psi_{\xi} = \text{MLP}_{\tanh}(d, 64, 1, 4).$$

Here, we multiply the MLP with the truncation function

$$\zeta(x) = \min_{1 \leq k \leq d} \{x_k, 1 - x_k\},$$

in order to enforce $\varphi_{\eta} \in H_0^1(\Omega)$. Furthermore, based on the definition of $\mathcal{E}$, we set

$$\mathcal{M}_p = \mathcal{M}_d = \nabla$$

as discussed in section 3. And recall the definition (24), (21) and (25), we define the preconditioning matrices in the proposed NPDG algorithm as

$$\begin{aligned}M_p(\theta) &= \int_{\Omega} \frac{\partial}{\partial \theta} (\nabla_x u_{\theta}(x)) \frac{\partial}{\partial \theta} (\nabla_x u_{\theta}(x))^{\top} d\mu(x) + \lambda \int_{\partial\Omega} \frac{\partial u_{\theta}(y)}{\partial \theta} \frac{\partial u_{\theta}(y)}{\partial \theta}^{\top} d\mu_{\partial\Omega}(y) \\ M_d(\eta) &= \int_{\Omega} \frac{\partial}{\partial \theta} (\nabla_x \varphi_{\eta}(x)) \frac{\partial}{\partial \theta} (\nabla_x \varphi_{\eta}(x))^{\top} d\mu(x), \quad M_{bdd}(\xi) = \lambda \int_{\partial\Omega} \frac{\partial}{\partial \xi} \psi_{\xi}(y) \frac{\partial}{\partial \xi} \psi_{\xi}(y)^{\top} d\mu_{\partial\Omega}(y).\end{aligned}$$

In this example, we pick $N_{in} = 2000$ and $N_{bdd} = 80d = 800$. We choose $\lambda = 10$ and $\epsilon = 1$ in the loss function. For the hyper parameters of the algorithm, we set the extrapolation coefficient $\omega = 1$, and the stepsizes $\tau_u = 0.5 \cdot 10^{-1}$, $\tau_{\varphi} = \tau_{\psi} = 0.95 \cdot 10^{-1}$. We set the maximum iteration number $n_{\text{MINRES}} = 1000$ for the MINRES algorithm. We test the thresholds $tol_{\text{MINRES}} = 10^{-3}, 10^{-4}$ in the algorithm. We compare the algorithms with the PINN, DeepRitz, and WAN methods. The detailed settings for these three methods are provided in Table 2. We run each method for 150 seconds and make semi-log plots of relative error vs. computational time for all the methods tested. Throughout this research, we consider the relative L^2 error of u_{θ} and ∇u_{θ} . The error plots are presented in Figure 1. The plot of MINRES iteration numbers at each NPDG step is also provided in Figure 1

We investigate the effectiveness of our natural(preconditioned)-gradient method by comparing it with the same algorithm using flat gradients. That is, we replace line 6, line 10 in Algorithm 2 by $\eta = \eta + \tau_{\varphi} \mathbf{w}_{\varphi}$, $\xi = \xi + \tau_{\psi} \mathbf{w}_{\psi}$, and $\theta = \theta - \tau_u \mathbf{w}_u$. This is demonstrated in Figure 2, 2a. In the same plot, it is also observed that the extrapolation step (line 7 of Algorithm 2) will slightly enhance the convergence of the proposed algorithm. Furthermore, choosing suitable preconditioning matrices compatible with the mathematical nature of the PDE is crucial for the proposed method. In Figure 2, 2b, we compare our treatment with the NPDG algorithm with $M_p(\theta), M_d(\eta)$ obtained by setting $\mathcal{M}_p = \mathcal{M}_d = \text{Id}$. As reflected in the plot, unreasonable preconditioning may lead to instabilities in the optimization procedure.

In addition, we also test the same example with $d = 50$. We set

$$u_{\theta} = \text{MLP}_{\tanh}(d, 256, 1, 6), \quad \varphi_{\eta} = \text{MLP}_{\tanh}(d, 256, 1, 6) \cdot \zeta, \quad \psi_{\xi} = \text{MLP}_{\tanh}(d, 128, 1, 6).$$

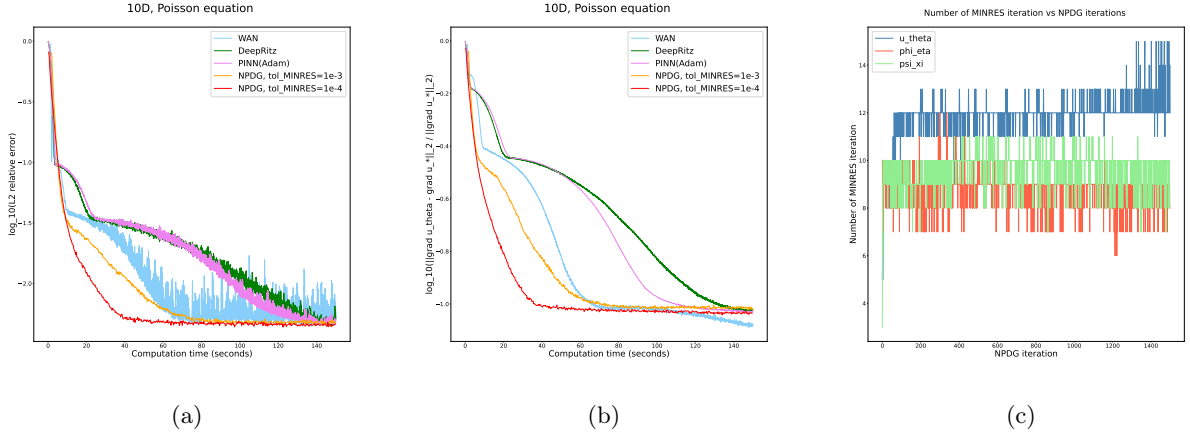


Figure 1: (10D Poisson equation) **Left**: Semi-log plot of relative L2 error ($\frac{\|u_\theta - u_*\|_{L^2(\mu)}}{\|u_*\|_{L^2(\mu)}}$) vs. computational time (seconds); **Middle**: Semi-log plot of $\frac{\|\nabla u_\theta - \nabla u_*\|_{L^2(\mu)}}{\|\nabla u_*\|_{L^2(\mu)}}$ vs. computational time (seconds). The values of $\|u_*\|_{L^2(\mu)}$ and $\|\nabla u_*\|_{L^2(\mu)}$ are provided in Table 3. **Right**: Numbers of iterations required by the MINRES algorithm for updating θ, η, ξ at each NPDG step vs. NPDG iteration.

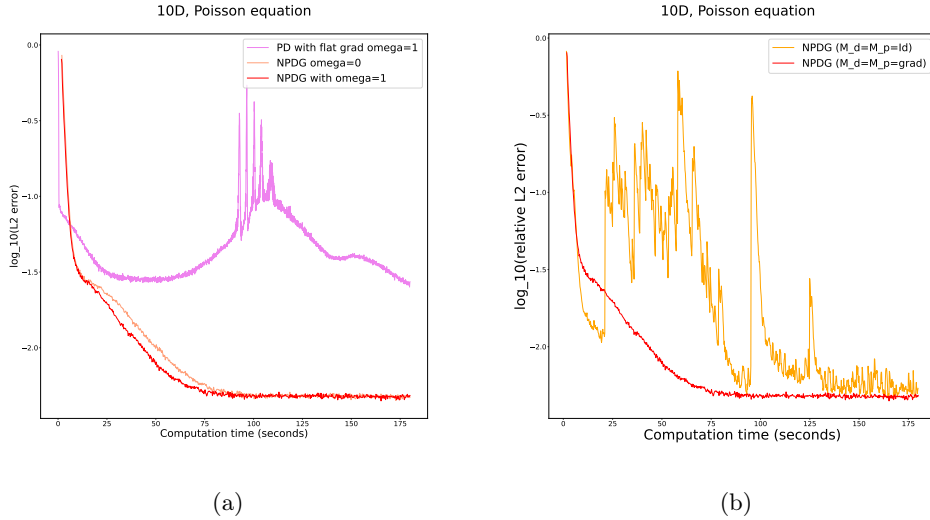


Figure 2: (10D Poisson equation) **Left**: Comparison with the same algorithm using flat gradients instead (pink), and with the same algorithm without extrapolation ($\omega = 0$) (light red); **Right**: Comparison with our NPDG method, but using $M_p(\theta), M_d(\eta)$ obtained by $M_p = M_d = \text{Id}$ as our preconditioning (orange). All the plots in these two figures are relative L2 error vs. computational time (seconds).

We choose the tolerance $tol_{\text{MINRES}} = 10^{-4}$ to ensure higher accuracy in computing the natural gradient. We keep all the remaining hyperparameters unchanged. Figure 3 presents the associated numerical results. The loss plot 3c suggests that our proposed NPDG algorithm converges faster and more stably compared with the algorithms based on the Adam optimizer. We also record the GPU time spent by each method to achieve a certain accuracy for various dimensions $d = 5, 10, 20, 50$. Details are provided in Table 4 of the Appendix D. the proposed method performs more efficiently than the other methods as the dimension d increases.

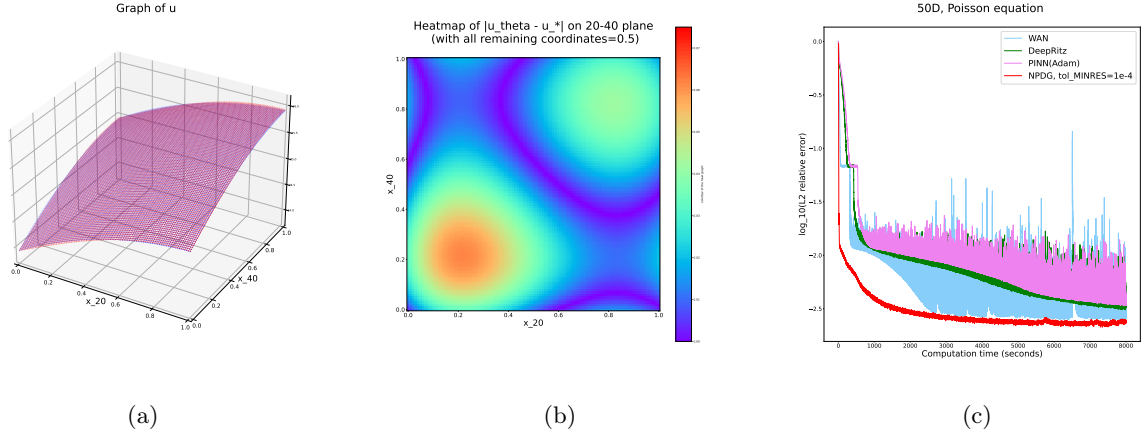


Figure 3: (50D Poisson equation) **Left**: Graph of learned u_θ and real solution u_* on the 20-40 coordinate plane (with remaining coordinates equal to $\frac{1}{2}$) in $\mathbb{R}^{50}$; **Middle**: Heatmap of $|u_\theta(x) - u_*(x)|$ on the same plane. **Right**: Semi-log plot of relative L2 error vs. computational time (seconds). The values of $\|u_*\|_{L^2(\mu)}$ and $\|\nabla u_*\|_{L^2(\mu)}$ are provided in Table 3.

5.2 Elliptic equation with variable coefficients (10D, 20D, 50D)

We consider the following elliptic equation with a variable coefficient

$$-\nabla \cdot (\kappa(x) \nabla u(x)) = f(x), \quad u(y) = g(y) \text{ on } \partial\Omega. \quad (44)$$

Here we assume $\Omega = [-1, 1]^d$ with even dimension d . We set

$$\kappa(x) = \frac{x^\top \Lambda x + 1}{2}, \quad \text{with } \Lambda = \text{diag}(\lambda_0, \lambda_1, \dots, \lambda_0, \lambda_1),$$

where $\lambda_0, \lambda_1 > 0$ appeared alternatively for $\frac{d}{2}$ times, and choose

$$f(x) = -\frac{\text{Tr}(\Lambda^{-1})}{2}(x^\top \Lambda x + 1) - \|x\|^2, \quad \text{and } g(y) = \frac{1}{2}y^\top \Lambda^{-1}y, \quad y \in \partial\Omega.$$

The solution to this equation is $u_*(x) = \frac{1}{2}x^\top \Lambda^{-1}x$.

Similar to the previous example, we introduce φ, ψ to the equation and its boundary condition. Integration by parts yields the functional $\mathcal{E} : H^2(\Omega) \times H_0^1(\Omega) \times L^2(\Omega) \rightarrow \mathbb{R}$:

$$\begin{aligned} \mathcal{E}(u, \varphi, \psi) = & \int_{\Omega} \kappa(x) \nabla \varphi(x) \cdot \nabla u(x) - f(x) \varphi(x) \, d\mu(x) - \frac{\epsilon}{2} \int_{\Omega} \|\nabla \varphi(x)\|^2 \, d\mu(x) \\ & + \lambda \left(\int_{\partial\Omega} (u - g) \psi \, d\mu_{\partial\Omega} - \frac{\epsilon}{2} \int_{\partial\Omega} \psi^2 \, d\mu_{\partial\Omega} \right). \end{aligned}$$

Similarly, we add the boundary loss function to $\mathcal{E}(u, \varphi, \psi)$ to obtain

$$\tilde{\mathcal{E}}(u; \varphi, \psi) = \mathcal{E}(u; \varphi, \psi) + \lambda \|u - g\|_{L^2(\mu_{\partial\Omega})}^2.$$

We use $\tilde{\mathcal{E}}$ in the computation. We set

$$\mathcal{M}_p = \mathcal{M}_d = \nabla$$

for the preconditioning matrices $M_p(\theta)$, $M_d(\eta)$, $M_{bdd}(\xi)$ as defined in (24), (21) and (25). We test this example with $d = 10, 20, 50$. We substitute u, φ, ψ with MLPs with $\text{softplus}(\cdot)$ as activation functions. Here, $\text{softplus}(\cdot)$

is a smooth approximation of the ReLU function defined as³

$$\text{softplus}(x) = \frac{1}{\beta} \log(1 + \exp(\beta x))$$

with $\beta = \frac{1}{4}$. We summarize the neural net architecture of our experiments in Table 1. Similar to our treatment for the Poisson’s equation, we multiply φ_η by the truncation function $\zeta(\cdot)$ to enforce $\varphi_\eta \in H_0^1(\Omega)$.

	Primal & Dual Neural Networks			N_{in}, N_{bdd}	$\tau_u, \tau_\varphi, \tau_\psi$	MINRES tol
	u_θ (MLP _{softplus})	φ_η (MLP _{softplus} · ζ)	ψ_ξ MLP _{softplus}			
$d = 10$	$(d, 256, 1, 4)$	$(d, 256, 1, 4)$	$(d, 128, 1, 4)$	4000, 40d	0.1, 0.19, 0.19	$0.5 \cdot 10^{-3}$
$d = 20$					0.05, 0.095, 0.095	
$d = 50$	$(d, 256, 1, 6)$	$(d, 256, 1, 6)$	$(d, 128, 1, 6)$	6000, 40d		10^{-4}

Table 1: Basic setting of our experiments on computing (44).

In this example, for all dimensions $d = 10, 20, 50$, we set $\lambda = 10$ and $\epsilon = 1$ in the loss function; we set the extrapolation coefficient $\omega = 1$. The stepsizes $\tau_u, \tau_\varphi, \tau_\psi$, the number of samples N_{in}, N_{bdd} , as well as the tolerance of MINRES used in all tests, are also summarized in Table 1. We improve the tolerance of the MINRES algorithm from 10^{-3} to 10^{-4} as the dimension d increases to 50. We run the proposed method for 500 and 1500 seconds for 10-D and 20-D problems respectively. For the 50-D problem, we perform the proposed method for 36000 iterations. For all $d = 10, 20, 50$, we compare the algorithm with the PINN, DeepRitz, and WAN methods. The detailed settings for these three methods are provided in Table 2. We make semi-log/log-log plots of relative error vs. computational time for all methods. The error plots are presented in Figure 5. The plots justify the linear convergence of the proposed method. Compared with the other algorithms based on Adam optimizers, the proposed method performs more stably and achieves higher accuracy in this example. We also record the GPU time spent by each method to achieve a certain accuracy. One can find the details in Table 4 of Appendix D. It turns out that only the proposed method can achieve an accuracy such that $\frac{\|u_\theta - u_*\|_{L^2(\mu)}}{\|u_*\|_{L^2(\mu)}} \leq 0.005$.

For $d = 20$, we visualize the solution u_θ learned by the NPDG algorithm by plotting the graph of u_θ on the 9 – 10 plane while fixing the remaining coordinates to 0 and 0.5 for $d = 20$ in Figure 5. The associated heatmaps of $|u_\theta(x) - u_*(x)|$ on the 9 – 10 plane are also provided in Figure 5. To investigate the accuracy of u_θ over the entire space of Ω , we separate $\Omega = \bigcup_{l=1}^{50} \Omega_l$ into 50 square shells with gradually increasing sizes,

$$\Omega_l := \{x = (x_1, \dots, x_d)^\top \in \mathbb{R}^d | (l-1)/50 \leq |x_k| < l/50, 1 \leq i \leq d\}.$$

We plot the average L^2 error of u_θ computed via different methods on Ω_l with respect to the size $l/50$ of each square shell Ω_l in Figure 4, 5e.

Different MINRES tolerances: Slightly improving (i.e., decreasing) the tolerance tol_{MINRES} of the MINRES algorithm yields more accurate directions of the natural gradients and enhances the convergence of the NPDG algorithm. However, selecting tol_{MINRES} too small makes the algorithm sensitive with respect to data stochasticity and thus may introduce instability to the method. This is reflected in Figure 6a and 6b.

Comparing with L-BFGS optimizer: We apply the L-BFGS optimizer to PINN and compare its convergence speed with the proposed method. L-BFGS utilizes the second-order information from the loss function in optimization. However, L-BFGS is known to be unstable in stochastic setting—using random batches is not a feasible strategy for L-BFGS method. In this example, we fix the Monte-Carlo samples in the algorithm and optimize the PINN loss function with L-BFGS method. For $d = 20$, as shown in Figure 6c, our NPDG algorithm with $tol_{\text{MINRES}} = 10^{-4}$ converges faster than the L-BFGS method. Moreover, the L-BFGS method faces instability even without data stochasticity. As demonstrated in Figure 6d, the L-BFGS method always blows up given a long enough running time for dimensions $d = 20$ and $d = 50$.

³In PyTorch, for numerical stability, the implementation of $\text{softplus}(\cdot)$ reverts to the linear function when $x > \frac{\text{threshold}}{\beta}$. The default value for the threshold equals 20.

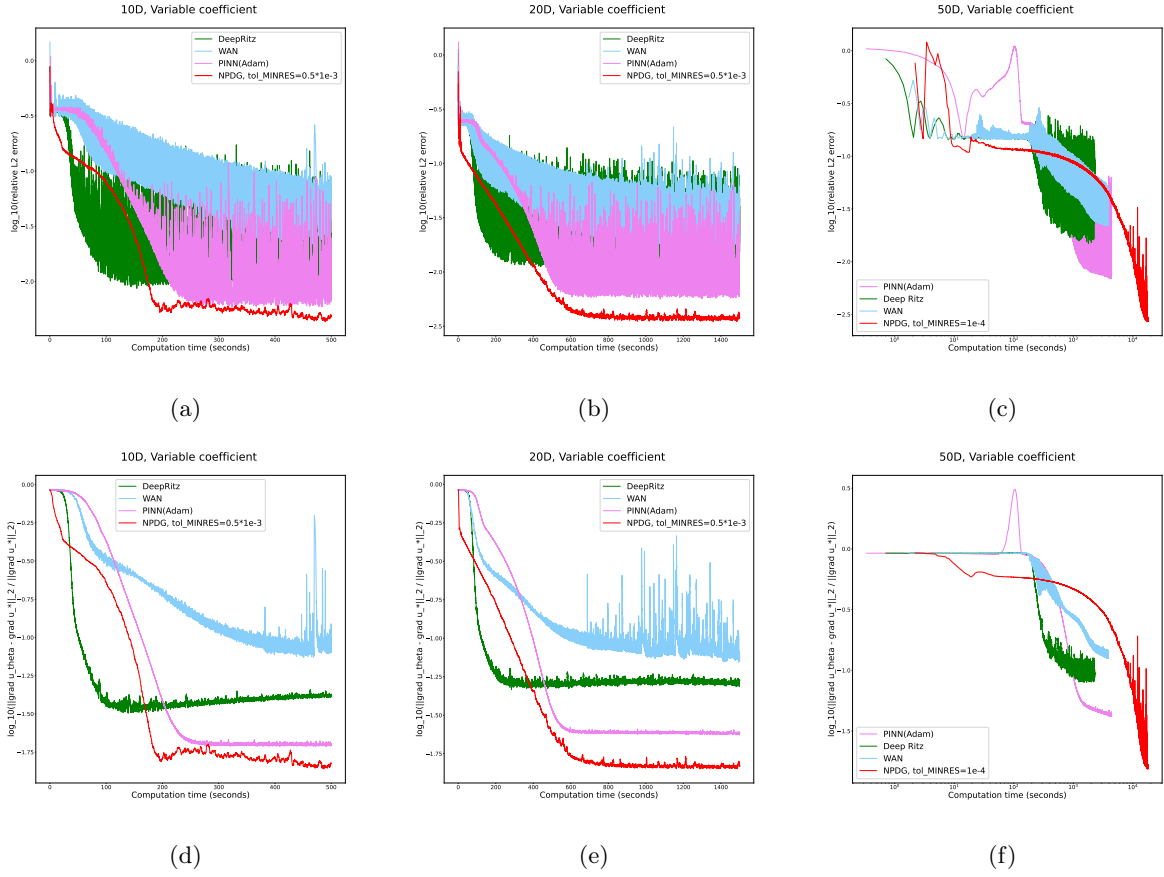


Figure 4: **Left** column (4a) (4d): Semi-log plot (up) of relative L2 error vs. computational time(seconds) and semi-log plot (down) of $\frac{\|\nabla u_\theta - \nabla u_*\|_{L^2(\mu)}}{\|\nabla u_*\|_{L^2(\mu)}}$ vs. computational time. Dimension $d = 10$; **Middle** column (4b) (4e): The same plots for $d = 20$; **Right** column (4c) (4f): The same plots (but in Log-log form) for $d = 50$. The values of $\|u_*\|_{L^2(\mu)}$ and $\|\nabla u_*\|_{L^2(\mu)}$ are provided in Table 3.

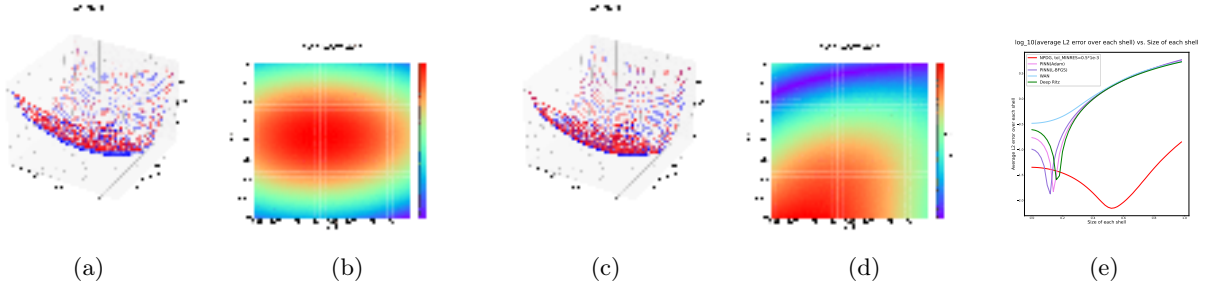


Figure 5: Plots for $d = 20$. 5a: Graph of u_θ obtained by NPDG method (blue) with real solution (red) plotted on 9 – 10 plane with remaining coordinates fixed to 0; 5b Heatmap of error $|u_\theta(x) - u_*(x)|$ plotted on 9 – 10 plane with remaining coordinates fixed to 0. 5c, 5d: Same plots plotted on 9 – 10 plane with remaining coordinates fixed to 0.5; 5e: Semi-log plot of $\log_{10} \left(\frac{1}{|\Omega_l|} \|\nabla u_\theta - \nabla u_*\|_{L^2(\Omega_l)} \right)$ vs. size of each square shell Ω_l .

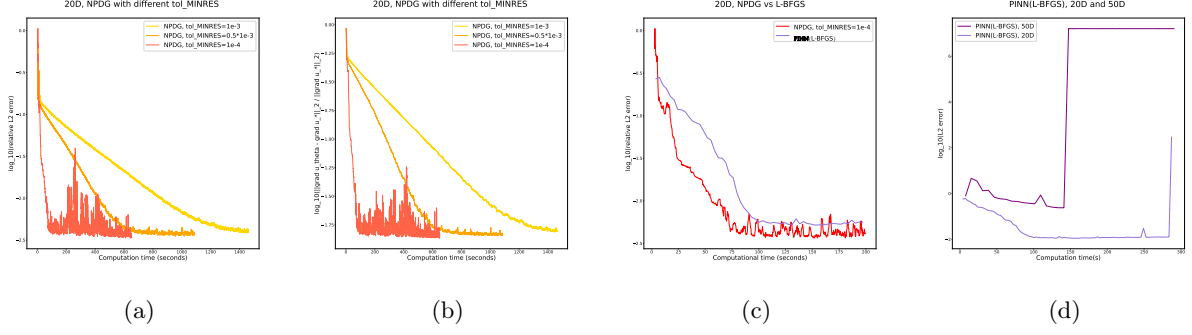


Figure 6: Figures 6a, 6b: Plots of relative error vs. computation time(seconds) with different $tol_{MINRES} = 10^{-4}, 0.5 \cdot 10^{-3}, 10^{-3}$. Figure 6c: Plot of relative L^2 error vs. computation time(seconds), we compare NPDG with $tol_{MINRES} = 10^{-4}$ to PINN using L-BFGS optimizer. Figure 6d: Long-time behavior of L-BFGS optimizer when applied to 20D and 50D problems.

5.3 Nonlinear elliptic equation (5D)

We consider the following nonlinear elliptic equation equipped with Dirichlet boundary condition on a d -dimensional ball with radius $R = 3$

$$B_{d,R} = \{x \in \mathbb{R}^d \mid \|x\| \leq R\}.$$

$$\frac{1}{2} \|\nabla u(x)\|^2 + V(x) = \Delta u(x), \quad u|_{\partial B_{d,R}} = 0. \quad (45)$$

Here we set

$$V(x) = -\frac{\pi^2}{8} \sin^2\left(\frac{\pi}{2}r\right) - \frac{\pi^2}{4} \cos\left(\frac{\pi}{2}r\right) - \frac{\pi(d-1)}{2r} \sin\left(\frac{\pi}{2}r\right)$$

with $r = \|x\|$. The solution to this equation is the radial function

$$u_*(x) = \cos\left(\frac{\pi}{2}r\right).$$

Similar to the previous examples, we introduce φ, ψ to the equation and its boundary condition. We obtain $\mathcal{E} : H^2(\Omega) \times H_0^1(\Omega) \times L^2(\Omega) \rightarrow \mathbb{R}$ as:

$$\begin{aligned} \mathcal{E}(u, \varphi, \psi) = & \left(\int_{\Omega} \nabla \varphi(x) \cdot \nabla u(x) + \frac{1}{2} \|\nabla u(x)\|^2 \varphi(x) + V(x) \varphi(x) \, d\mu(x) - \frac{\epsilon}{2} \int_{\Omega} \|\nabla \varphi(x)\|^2 \, d\mu(x) \right) \\ & + \lambda \left(\int_{\partial\Omega} u\psi \, d\mu_{\partial\Omega} - \frac{\epsilon}{2} \int_{\partial\Omega} \psi^2 \, d\mu_{\partial\Omega} \right). \end{aligned}$$

And we consider solving $\inf_u \sup_{\varphi, \psi} \{\tilde{\mathcal{E}}(u, \varphi, \psi)\}$ in this computation, where

$$\tilde{\mathcal{E}}(u; \varphi, \psi) = \mathcal{E}(u; \varphi, \psi) + \lambda \|u\|_{L^2(\mu_{\partial\Omega})}^2.$$

It is still unclear what the optimal way is to precondition the nonlinear term in this equation. Our treatment only focuses on the linear part Δu . Thus we set

$$\mathcal{M}_p = \mathcal{M}_d = \nabla$$

for the preconditioning matrices $M_p(\theta), M_d(\eta), M_{bdd}(\xi)$.

We test this example with $d = 5$, we set

$$u_{\theta} = \text{MLP}_{\tanh}(d, 256, 1, 4), \quad \varphi_{\eta} = \text{MLP}_{\tanh}(d, 256, 1, 4), \quad \psi_{\xi} = \text{MLP}_{\tanh}(d, 128, 1, 4).$$

Similar to the previous examples, we choose $\lambda = 10$ and $\omega = 1$ for the NPDG algorithm. We apply Monte-Carlo method to evaluate the loss function, in order to sample uniformly from $B_{d,R}$, we first randomly sample N_{in} points $\rho_1, \dots, \rho_{N_{in}}$ from the interval $[0, R]$ following the density function $p(\rho) = \frac{d+1}{R} \left(\frac{\rho}{R}\right)^d, \rho \in [0, R]^4$. Then we sample N_{in} points $\mathbf{w}_1, \dots, \mathbf{w}_{N_{in}}$ from the standard Gaussian distribution $\mathcal{N}(0, I_d)$. Thus, we obtain N_{in} sample points in $B_{d,R}$ by forming $x_i = \rho_i \frac{\mathbf{w}_i}{\|\mathbf{w}_i\| + e_0}, 1 \leq i \leq N_{in}$. We add $e_0 = 10^{-8}$ to prevent zero denominators. We run the proposed method for $N_{iter} = 10000$ iterations.

In this example, we also test the PINN(Adam/L-BFGS) and WAN methods. The hyperparameters for these methods are provided in Table 2. Log-log plots of the relative error vs. the computation time among the methods are provided in Figure 7. We plot the graph of u_θ obtained by the algorithm on the 1 – 2

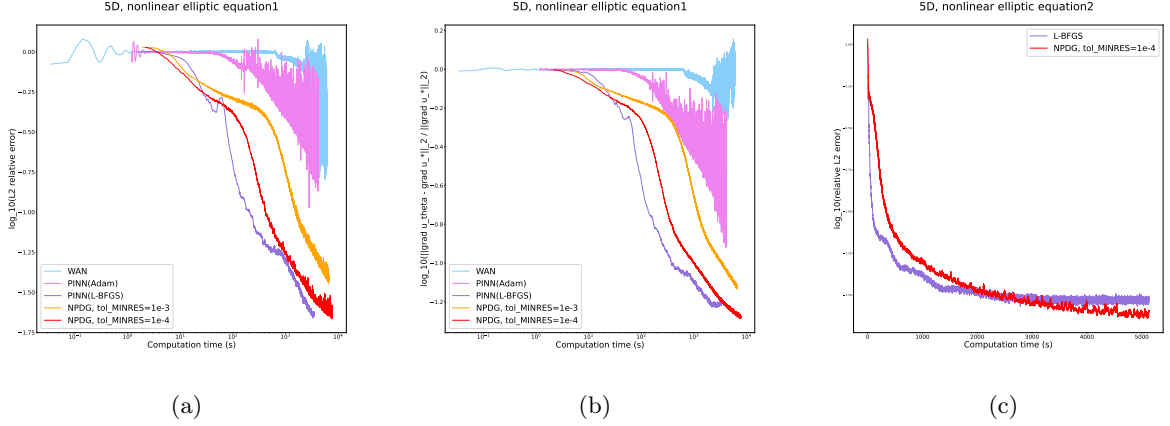


Figure 7: Equation (45): **Left**: Log-log plot of relative L2 error vs. computational time (seconds); **Middle**: Log-log plot of $\frac{\|\nabla u_\theta - \nabla u_*\|_{L^2(\mu)}}{\|\nabla u_*\|_{L^2(\mu)}}$ vs. computational time (seconds). The values of $\|u_*\|_{L^2(\mu)}$ and $\|\nabla u_*\|_{L^2(\mu)}$ are provided in Table 3. Equation (46): **Right**: Semi-log plot of relative L2 error vs. computational time.

coordinate plane in Figure 8a. We also plot the heat maps of the error function $|u_\theta(\cdot) - u_*(\cdot)|$ on various coordinate planes in Figures 8b-8e. Similar to previous examples, we record the GPU times spent by different

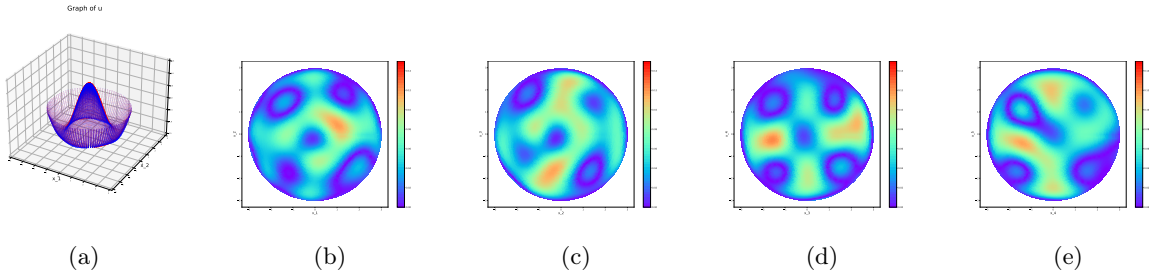


Figure 8: Figure 8a: Graph of u_θ on the 1 – 2 coordinate plane (that is, the plane spanned by the first and second components with the remaining coordinates fixed to 0). The parameter θ is obtained by the NPDG method after 10000 iterations; Figures 8b-8e: Heatmaps of $|u_\theta(\cdot) - u_*(\cdot)|$ plotted on 1 – 2, 2 – 3, 3 – 4, 4 – 5 coordinate planes.

methods for achieving certain accuracy in Table 4 of Appendix D. Furthermore, we also consider the following

⁴This can be done by first sampling n_ρ points $r_1, \dots, r_{n_\rho}$ uniformly from $[0, 1]$ and then transform each r_i to $\rho_i = r_i^{\frac{1}{d}} \cdot R^{1-\frac{1}{d}}$ for $1 \leq i \leq n_\rho$.

equation on the same region $B_{d,R}$ ($d = 5$, $R = 3$) with a weaker nonlinear term,

$$\frac{\epsilon_0}{2} \|\nabla u(x)\|^2 + \Delta u(x) = V(x), \quad u|_{\partial B_{d,R}} = 0. \quad (46)$$

Here we set $\epsilon = \frac{1}{10}$ and

$$V(x) = \frac{\epsilon_0 \pi^2}{8} \sin^2\left(\frac{\pi}{2}r\right) - \frac{\pi^2}{4} \cos\left(\frac{\pi}{2}r\right) - \frac{\pi(d-1)}{2r} \sin\left(\frac{\pi}{2}r\right).$$

The solution to this equation is still $u_*(x) = \cos(\frac{\pi}{2}r)$. We apply the NPDG algorithm with exactly the same neural network architecture and hyperparameters as in (45) to solve equation (46). We also test the L-BFGS optimizer to minimize the PINN loss of equation (46). Figure 7c indicates that the proposed method achieves performance that is compatible with L-BFGS in this example.

5.4 Allen-Cahn equation

We have discussed several examples of time-independent PDEs. We now briefly show how the proposed method is applied to resolve the time-implicit, semi-discrete schemes of the time-dependent equations. In this section, we primarily focus on the 1D and 2D Allen-Cahn equations to illustrate the main idea. Future research will explore additional approaches, such as adaptive sampling techniques [93] and extensions to higher dimensions.

We consider the Allen-Cahn equation on a bounded domain Ω posed with the homogeneous Neumann boundary condition on time interval $[0, T]$.

$$\frac{\partial u(x, t)}{\partial t} = \epsilon_0 \Delta u(x, t) - \frac{1}{\epsilon_0} W'(u), \quad \frac{\partial u}{\partial \mathbf{n}} = 0 \text{ on } \partial\Omega, \quad u(\cdot, 0) = u_0(\cdot).$$

Here we define the double-well potential function $W(u) = \frac{1}{4}(1 - u^2)^2$, with $W'(u) = u^3 - u$. In this research, we focus on resolving the time-implicit, semi-discrete numerical scheme of this equation. We divide the time interval into N_t subintervals and consider

$$\frac{u^t(x) - u^{t-1}(x)}{h_t} = \epsilon_0 \Delta u^t(x) - \frac{1}{\epsilon_0} W'(u^t(x)), \quad \frac{\partial u^t}{\partial \mathbf{n}} = 0 \text{ on } \partial\Omega,$$

sequentially for $1 \leq t \leq N_t$ with $u^0(\cdot)$ set as $u_0(\cdot)$. That is, we need to solve the N_t consecutive elliptic equations with a cubic term as shown below,

$$u^t(x) - \epsilon_0 h_t \Delta u^t(x) + \frac{h_t}{\epsilon_0} ((u^t(x))^3 - u^t(x)) = u^{t-1}(x), \quad \frac{\partial u^t}{\partial \mathbf{n}} = 0 \text{ on } \partial\Omega, \quad 1 \leq t \leq N_t. \quad (47)$$

We can tame the nonlinear term $W'(u) = u^3 - u$ by subtracting its linear approximation at the equilibrium state $\bar{u} = \pm 1$, i.e., we consider $R(u) = W'(u) - (W'(\bar{u}) + W''(\bar{u})(u - \bar{u}))$. We then absorb the linear term $W''(\bar{u})u$ of $W'(\bar{u}) + W''(\bar{u})(u - \bar{u})$ to the linear portion of (47) to obtain

$$\underbrace{\left(\left(1 + \frac{h_t W''(\bar{u})}{\epsilon_0}\right) \text{Id} - h_t \epsilon_0 \Delta \right) u}_{\mathcal{D}} + \frac{h_t}{\epsilon_0} R(u) = u^{t-1} - \underbrace{\frac{h_t}{\epsilon_0} (W'(\bar{u}) - W''(\bar{u})\bar{u})}_{\text{Const}}.$$

It is reasonable to precondition on the linear differential operator $\mathcal{D}$ for this equation. We introduce the operators

$$\mathcal{M}_p = \mathcal{M}_d : u \mapsto \begin{pmatrix} \sqrt{1 + h_t W''(\bar{u})/\epsilon_0} u \\ \sqrt{\epsilon_0 h_t} \nabla u \end{pmatrix}.$$

It is not difficult to verify that $\langle \mathcal{M}_p u, \mathcal{M}_d \varphi \rangle_{L^2} = \langle \mathcal{D}u, \varphi \rangle_{L^2}$ for arbitrary $\varphi \in H_0^1(\Omega)$. Thus, we introduce $\varphi \in H_0^1(\Omega), \psi \in L^2(\partial\Omega)$ for the equation and its boundary condition and design the loss functional

$$\begin{aligned} \mathcal{E}(u; \varphi, \psi | u^{t-1}) &= \langle \mathcal{D}u + \frac{h_t}{\epsilon_0} R(u) - u^{t-1} + \text{Const}, \varphi \rangle_{L^2(\Omega)} - \frac{\epsilon}{2} \|\mathcal{M}_d \varphi\|_{L^2(\mu)}^2 + \lambda \left(\left\langle \frac{\partial u}{\partial \mathbf{n}}, \psi \right\rangle_{L^2(\partial\Omega)} - \frac{\epsilon}{2} \|\psi\|_{L^2(\mu_{\partial\Omega})}^2 \right) \\ &= \int_{\Omega} (u(x) - u^{t-1}(x) + \frac{h_t}{\epsilon_0} (u^3(x) - u(x))) \varphi(x) + \epsilon_0 h_t \nabla u(x) \cdot \nabla \varphi(x) \, d\mu(x) \\ &\quad - \frac{\epsilon}{2} \left(\left(1 - \frac{h_t}{\epsilon_0} W''(\bar{u}) \right) \int_{\Omega} \varphi^2(x) \, d\mu(x) - \epsilon_0 h_t \int_{\Omega} \|\nabla \varphi(x)\|^2 \, d\mu(x) \right) \\ &\quad + \lambda \left(\int_{\partial\Omega} \frac{\partial u(y)}{\partial \mathbf{n}(y)} \psi(y) \, d\mu_{\partial\Omega}(y) - \frac{\epsilon}{2} \int_{\partial\Omega} \psi^2(y) \, d\mu_{\partial\Omega}(y) \right). \end{aligned}$$

In practice, we found that it makes the optimization more stable if we add the PINN loss function as a regularization term to $\mathcal{E}(u; \varphi, \psi)$, i.e., we denote the PINN loss

$$\mathcal{E}_{PINN}(u | u^{t-1}) = \int_{\Omega} \left| u(x) - u^{t-1}(x) - \epsilon_0 h_t \Delta u(x) + \frac{h_t}{\epsilon_0} (u^3(x) - u(x)) \right|^2 \, d\mu(x) + \lambda \int_{\partial\Omega} \left| \frac{\partial u(y)}{\partial \mathbf{n}} \right|^2 \, d\mu_{\partial\Omega}(y),$$

and consider

$$\tilde{\mathcal{E}}(u; \varphi, \psi | u^{t-1}) = \mathcal{E}(u; \varphi, \psi | u^{t-1}) + \mathcal{E}_{PINN}(u; \varphi, \psi | u^{t-1}).$$

In the implementation, we substitute u, φ, ψ with neural networks with tanh as activation functions,

$$u_{\theta} = \text{MLP}_{\tanh}(d, 128, 1, 5), \quad \varphi_{\eta} = \text{MLP}_{\tanh}(d, 128, 1, 5) \cdot \zeta, \quad \psi_{\xi} = \text{MLP}_{\tanh}(d, 64, 1, 5).$$

We set the precondition matrices as below:

$$\begin{aligned} M_p(\theta) &= \left(1 + \frac{h_t W''(\bar{u})}{\epsilon_0} \right) \int_{\Omega} \frac{\partial u_{\theta}(x)}{\partial \theta} \frac{\partial u_{\theta}(x)}{\partial \theta}^{\top} \, d\mu(x) + h_t \epsilon_0 \int_{\Omega} \frac{\partial}{\partial \theta} (\nabla_x u_{\theta}(x)) \frac{\partial}{\partial \theta} (\nabla_x u_{\theta}(x))^{\top} \, d\mu(x) \\ &\quad + \lambda \int_{\partial\Omega} \frac{\partial}{\partial \theta} (\partial_{\mathbf{n}} u_{\theta}(x)) \frac{\partial}{\partial \theta} (\partial_{\mathbf{n}} u_{\theta}(x))^{\top} \, d\mu_{\partial\Omega}(y), \\ M_p(\eta) &= \left(1 + \frac{h_t W''(\bar{u})}{\epsilon_0} \right) \int_{\Omega} \frac{\partial \varphi_{\eta}(x)}{\partial \theta} \frac{\partial \varphi_{\eta}(x)}{\partial \theta}^{\top} \, d\mu(x) + h_t \epsilon_0 \int_{\Omega} \frac{\partial}{\partial \theta} (\nabla_x \varphi_{\eta}(x)) \frac{\partial}{\partial \theta} (\nabla_x \varphi_{\eta}(x))^{\top} \, d\mu(x), \\ M_{bdd}(\xi) &= \lambda \int_{\partial\Omega} \frac{\partial}{\partial \xi} \psi_{\xi}(y) \frac{\partial}{\partial \xi} \psi_{\xi}(y)^{\top} \, d\mu_{\partial\Omega}(y). \end{aligned}$$

1D example We first test the algorithm on the 1D example with $\Omega = [0, 2]$, $\epsilon_0 = 0.1$ and the initial data $u_0(x) = (1 - \cos(\pi(x-1))) \cos(\pi(x-1))$. We set $T = 1$, $N_t = 10$. In this example, we treat the distribution $\mu_{\partial\Omega} = \frac{1}{2}(\delta_0 + \delta_2)$ with δ_x denotes the Dirac measure⁵ concentrated on the point $x \in \mathbb{R}$.

For the algorithm, we set $\lambda = 10$, $\omega = 1$, $N_{in} = 2000$, $N_{bdd} = 2$ (since $\partial\Omega = \{0, 2\}$, we assign one sample for each end point). We remian the stepsizes unchanged as $\tau_u = 0.5 \times 10^{-1}$, $\tau_{\varphi} = 0.95 \times 10^{-1}$, $\tau_{\psi} = 0.95 \cdot 10^{-1}$. We set $N_{iter} = 3000$.

In Figure 9, we plot the graphs of our numerical solution u_{θ_k} obtained at different time nodes $t_k = \frac{k}{N_t}$ ($1 \leq k \leq N_t$) with the numerical solution $\{U^k\}_{k=1}^{N_t}$ solved from the following time-implicit, finite difference scheme

$$\begin{aligned} \frac{U_i^k - U_i^{k-1}}{h_t} &= \epsilon_0 \frac{U_{i+1}^k - 2U_i^k + U_{i-1}^k}{h_x^2} - \frac{1}{\epsilon_0} (U_i^{k3} - U_i^k), \\ U_{-1}^k &= U_0^k, \quad U_{N_x+1}^k = U_{N_x}^k, \quad \forall 0 \leq i \leq N_x, \quad \text{for } 1 \leq k \leq N_t. \end{aligned} \tag{48}$$

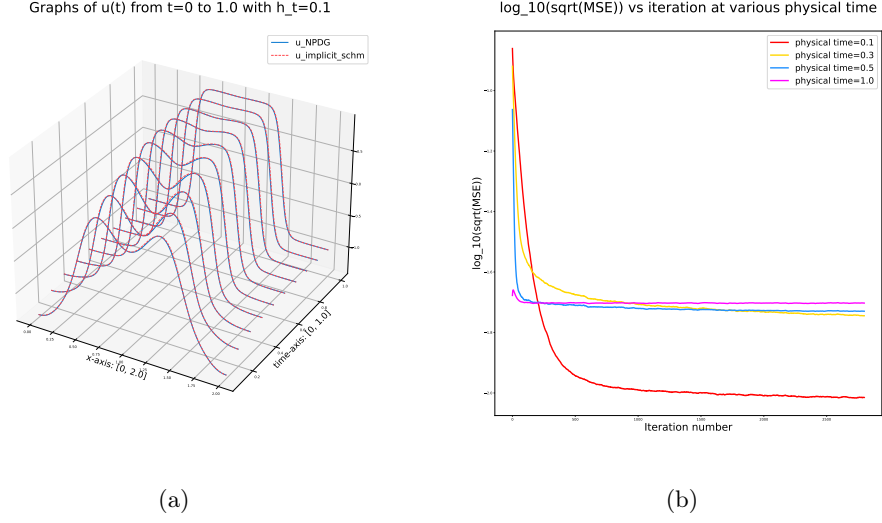


Figure 9: **Left:** The graph of $u_{\theta_k}(\cdot)$ (blue) obtained from the NPDG algorithm for $t_k = \frac{k}{N_t}$, $1 \leq k \leq N_t$ together with the benchmark solution (red, dashed line) solved via (48). **Right:** Semi-log plots of the $\sqrt{\text{MSE}}$ loss vs. computation time (seconds) at physical time 0.1, 0.3, 0.5, 1.0.

In the computation, we set $N_x = 400$, $h_x = 2/N_x$, $U_i^0 = u_0(\frac{2i}{N_x})$. We also plot the semi-log curve of $\sqrt{\frac{1}{N_x} \sum_{i=1}^{N_x} (u_{\theta_k}(x_i) - U_i^k)^2}$ vs. the computation time in the same figure.

2D example We further consider a 2D Allen-Cahn equation with $\Omega = [0, 2]^2$, $\epsilon_0 = 0.1$ and the initial condition

$$u_0(x) = \tanh\left(-\frac{\|x - x_0\| - R}{\nu}\right),$$

with $x_0 = (1, 1)^\top$, $R = 0.5$ and $\nu = 0.1$. We set $T = 1.5$ and $N_t = 15$. We keep the hyperparameters of the NPDG algorithm the same as the previous example except we set $N_{iter} = 1000$.

In Figure 10 and 11, we plot the graphs of the neural network solution u_{θ_k} together with the numerical solution $\{U_{ij}^k\}$ obtained via the time-implicit finite difference scheme. The semi-log curves for $\sqrt{\text{MSE}}$ loss versus training time is provided in Figure 10; The heatmaps of the error term $|u_{\theta_k}(\cdot) - U^k|$ are presented in Figure 11.

5.5 Monge-Ampère equation for the L^2 -Optimal Transport problem

In this section, we focus on the computation of the Monge-Ampère equation (27). A PINN solver for this equation is proposed in [80]. Deep learning algorithms from the optimal transport perspective are discussed in [44, 58, 26], among other references.

As discussed in section 2.5.1, solving the equation is equivalent to solving the L^2 -optimal transport problem. This can be further reduced to a sup-inf saddle point problem (30). In this research, we assume that the samples of μ_0, μ_1 are available. In order to evaluate the functional $\mathcal{E}(T_\theta, \varphi_\eta)$, we generate samples $\{\mathbf{X}_i\}_i^N \sim \mu_0 = \rho_0 dx$ and $\{\mathbf{Y}_i\}_{i=1}^N \sim \mu_1 = \rho_1 dy$ and apply the Monte-Carlo algorithm,

$$\mathcal{E}(T_\theta, \varphi_\eta) \approx \frac{1}{N} \sum_{i=1}^N \frac{1}{2} \|\mathbf{X}_i - T_\theta(\mathbf{X}_i)\|^2 + \varphi_\eta(T_\theta(\mathbf{X}_i)) - \varphi_\eta(\mathbf{Y}_i).$$

⁵That is, $\delta_x(E) = 1$ for any measurable set $E \subset \mathbb{R}$ that contains x , and $\delta_x(E) = 0$ for measurable sets that do not contain x .

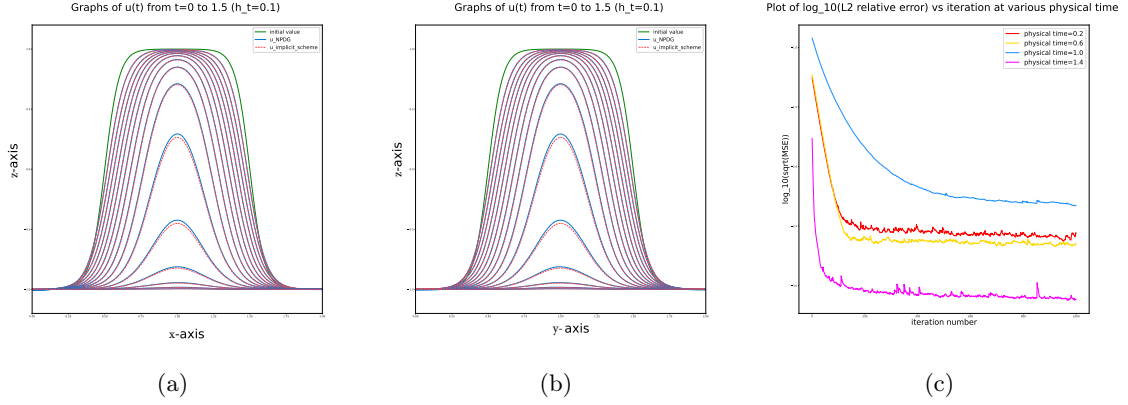


Figure 10: 10a & 10b: Comparison of neural network solution $u_{\theta}(\cdot)$ (blue) and finite difference solution U^k (red) along the x and y axis at time t_k , $1 \leq k \leq 15$. 10c: Semi-log plots of $\sqrt{\text{MSE}}$ loss vs computation time (seconds) at physical time 0.2, 0.6, 1.0, 1.4.

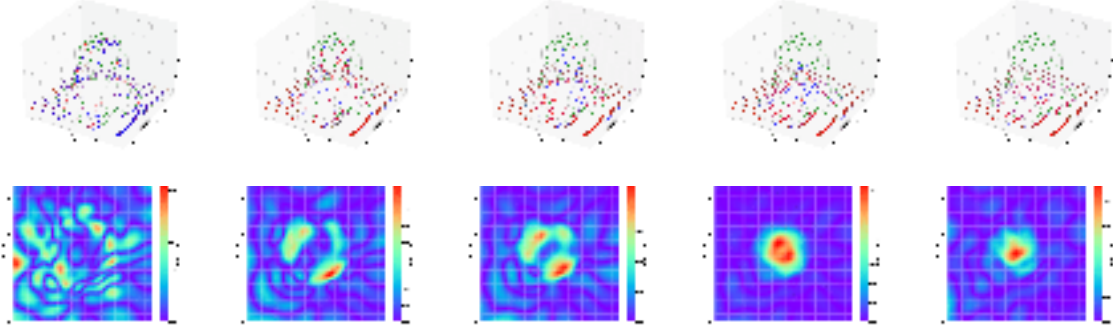


Figure 11: **Up row:** plots of u_{θ_k} (blue) together with the numerical solution $\{U_{ij}^k\}$ solved from implicit finite-difference scheme (red) on Ω at physical time 0.2, 0.6, 0.8, 1.0, 1.2. The initial function u_0 is marked with green color; **Down row:** heatmaps of the error term $|u_{\theta_k}(\cdot) - U^k|$ at physical time 0.2, 0.6, 0.8, 1.0, 1.2.

By applying Algorithm 1, we calculate the natural (preconditioned) gradients of $\mathcal{E}(T_{\theta}, \varphi_{\eta})$ with respect to θ, η . We then apply the NPDG algorithm 2 to solve the saddle point problem (30) for $T_*(\cdot)$ ($\nabla u(\cdot)$).

In experiments, we use the Primal-Dual algorithm with the Adam optimizer (PD-Adam) proposed in [26] as a benchmark for the proposed method. A brief description of this method, as well as its hyperparameters used in all tests, are provided in Appendix E. We test three numerical examples as a demonstration. The first two examples possess explicit formulas for the OT maps. In the third example, we compute the OT map from standard Gaussian to mixed Gaussian distributions embedded in 10D and 50D spaces. In the implementation, we set $T_{\theta}(\cdot)$, $\varphi_{\eta}(\cdot)$ as MLP with PReLU activation function

$$\text{PReLU}(x) = \begin{cases} x, & \text{if } x \geq 0 \\ ax, & \text{otherwise,} \end{cases}$$

where $a \in \mathbb{R}$ is a learnable parameter. The Input Convex Neural Networks (ICNN) architecture [3] advocated in [58] will be considered in future research.

5.5.1 1D Gaussian to mixed Gaussian

We set $\rho_0 = \mathcal{N}(0, 1)$, $\rho_1 = \sum_{k=1}^m \lambda_k \mathcal{N}(\mu_k, \sigma_k^2)$ with $\lambda_k > 0$, $\sum_{k=1}^m \lambda_k = 1$, $\mu_k \in \mathbb{R}$, $\sigma_k > 0$. The optimal transport map takes the explicit form,

$$T_*(x) = F_1^{-1}(F_0(x)), \quad F_0(x) = \sum_{k=1}^m \frac{\lambda_k}{2} (1 + \operatorname{erf}\left(\frac{x - \mu_k}{\sqrt{2}\sigma_k}\right)), \quad F_1^{-1}(y) = \operatorname{erf}^{-1}(2y - 1).$$

In the example, we consider $m = 2$, $\lambda_1 = \frac{2}{3}$, $\mu_1 = -1$, $\sigma_1 = 0.5$; $\lambda_2 = \frac{1}{3}$, $\mu_2 = 1$, $\sigma_2 = 0.5$. We set $T_\theta(\cdot)$ and φ_η as

$$T_\theta = \text{MLP}_{\text{PreLU}}(1, 50, 1, 3), \quad \varphi_\eta = \text{MLP}_{\text{PreLU}}(1, 50, 1, 3).$$

We set the sample size $N = 800$, $\omega = 1$, and $\tau_u = \tau_\varphi = 1.5 \cdot 10^{-1}$. We perform the NPDG algorithm for 6000 iterations. Figure 12a demonstrates the semi-log plots of the $L^2(\rho_0)$ error $\|T_\theta - T_*\|_{L^2(\rho_0)}$ versus the computation time. We make comparisons among the NPDG algorithms with different preconditioners ((31) and (35)), as well as the PD-Adam method.

5.5.2 5D Gaussian to Gaussian

For $\mu_0, \mu_1 \in \mathbb{R}^5$ and positive-definite symmetric matrices $\Sigma_0, \Sigma_1 \in \mathbb{R}^{5 \times 5}$, we set $\rho_0 = \mathcal{N}(\mu_0, \Sigma_0)$, $\rho_1 = \mathcal{N}(\mu_1, \Sigma_1)$. One can verify that the OT map takes the affine form $T_*(\mathbf{x}) = A\mathbf{x} + b$ with

$$A = \sqrt{\Sigma_0}^{-1} (\sqrt{\Sigma_0} \Sigma_1 \sqrt{\Sigma_0})^{1/2} \sqrt{\Sigma_0}^{-1}, \quad b = \mu_1 - A\mu_0.$$

For simplicity, we set $\mu_0 = \mu_1 = 0$ in the test example. The cases in which $\mu_0 \neq \mu_1$ can be readily handled by the pre-translating technique introduced in [46], which reduces the problem to the case in which $\mu_0 = \mu_1$. We define

$$\Sigma_0 = \begin{bmatrix} \frac{1}{4} & & & & \\ & 1 & & & \\ & & 1 & & \\ & & & 1 & \\ & & & & 1 \end{bmatrix}, \quad \Sigma_1 = \begin{bmatrix} 1 & & & & \\ & \frac{1}{4} & & & \\ & & 1 & & \\ & & & \frac{5}{8} & \frac{3}{8} \\ & & & \frac{3}{8} & \frac{5}{8} \end{bmatrix}.$$

Then the OT map is given by $T_*(x) = \sqrt{\Sigma_0^{-1} \Sigma_1} x$, with $\sqrt{\Sigma_0^{-1} \Sigma_1} = \begin{bmatrix} 2 & & & & \\ & \frac{1}{2} & & & \\ & & 1 & & \\ & & & \frac{3}{4} & \frac{1}{4} \\ & & & \frac{1}{4} & \frac{3}{4} \end{bmatrix}$. We set $T_\theta(\cdot)$ and φ_η as

$$T_\theta = \text{MLP}_{\text{PreLU}}(5, 80, 5, 4), \quad \varphi_\eta = \text{MLP}_{\text{PreLU}}(5, 80, 1, 4).$$

We set the sample size $N = 2000$, $\omega = 1$, and $\tau_u = 0.5 \cdot 10^{-1}$, $\tau_\varphi = 0.95 \cdot 10^{-1}$. We perform the NPDG algorithm for 20000 iterations. Similar to the previous example, we present the semi-log plots of $L^2(\rho_0)$ error vs computation time in Figure (12b). The plots of the computed transportation map $T_\theta(\cdot)$ together with $T_*(\cdot)$ are provided in Figure (12c) and (12d).

5.5.3 High dimensional Gaussian to mixed Gaussian (10D, 50D)

We consider the mixed-Gaussian distribution $\sum_{k=1}^8 \lambda_k \mathcal{N}(\mu_k, \sigma_k^2 I)$ defined on $\mathbb{R}^d$, where

$$\mu_k = \left(0, \dots, R \cos\left(\frac{k}{4}\pi\right), \dots, R \sin\left(\frac{k}{4}\pi\right), \dots, 0\right)^\top \quad \text{with } R = 3, \quad \sigma_k = \frac{4}{25}.$$

We assume that the two nonzero entries of μ_k are located in the i_0 and i_1 entries. We denote ρ_a as equal mixed-Gaussian

$$\rho_a = \sum_{k=1}^8 \lambda_k \mathcal{N}(\mu_k, \sigma_k^2 I), \quad \lambda_k = \frac{1}{8}, \quad 1 \leq k \leq 8; \quad (49)$$

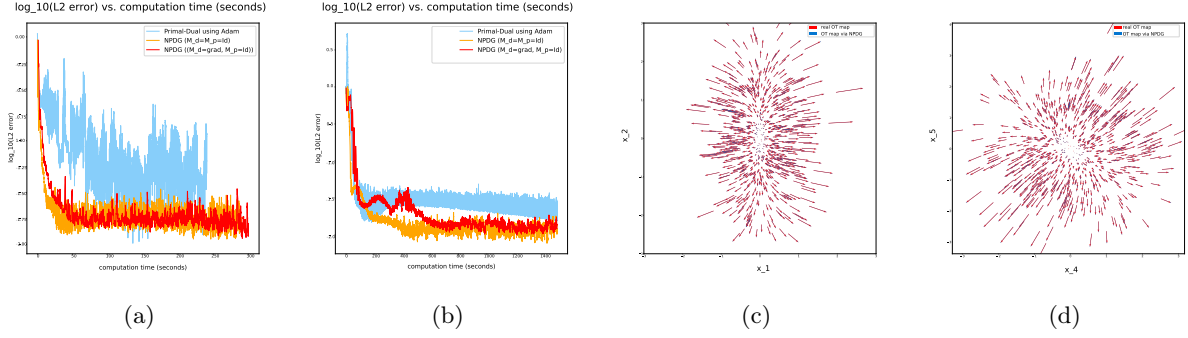


Figure 12: **OT problem (1D, 5D)**: 12a: Semi-log plots of $\|T_\theta - T_*\|_{L^2(\rho_0)}$ vs computation time (seconds) for the 1D problem discussed in section 5.5.1; 12b: Semi-log plots of $\|T_\theta - T_*\|_{L^2(\rho_0)}$ vs computation time (seconds) for the 5D problem discussed in section 5.5.2; 12c: Plot of the computed transport map $T_\theta(\cdot)$ (blue) with real OT map $T_*(\cdot)$ (red) on 1-2 plane; 12d: Plot of the computed transport map (blue) with real OT map (red) on 4-5 plane.

we denote ρ_b as a non-equally distributed mixed-Gaussian distribution with

$$\rho_b = \sum_{k=1}^8 \lambda_k \mathcal{N}(\mu_k, \sigma_k^2 I), \quad \lambda_k = \begin{cases} \frac{1}{5} & k \text{ is even,} \\ \frac{1}{20} & k \text{ is odd.} \end{cases}, \quad 1 \leq k \leq 8.$$

Consider $\rho_0 = \mathcal{N}(0, I)$. We compute the optimal transport from ρ_0 to ρ_a , as well as ρ_0 to ρ_b , by solving the sup-inf problem (30) using the NPDG algorithm. In the implementation, we always set

$$u_\theta(\cdot) = \text{MLP}_{\text{PReLU}}(d, 120, d, 6), \quad \varphi_\eta(\cdot) = \text{MLP}_{\text{PReLU}}(d, 120, 1, 6).$$

We first test the algorithm by setting $d = 10$, and $i_0 = 4, i_1 = 8$. We choose (31) as preconditioners for NPDG algorithm. We set $n_{\text{MINRES}} = 1000, \text{tol}_{\text{MINRES}} = 10^{-4}$; we choose the sample size $N = 2000, \omega = 1$, and $\tau_u = 0.5 \cdot 10^{-2}, \tau_\varphi = 0.95 \cdot 10^{-2}$; we perform the NPDG algorithm for 15000 iterations. We compute the optimal transport maps from ρ_0 to ρ_a and ρ_0 to ρ_b by applying the NPDG algorithm and the PD-Adam method. We compare the computational results in Figure 13. The pushforwarded distribution $T_{\theta\#}\rho_0$ of the proposed method outperforms PD-Adam in terms of homogeneity and shape of the mixed Gaussians.

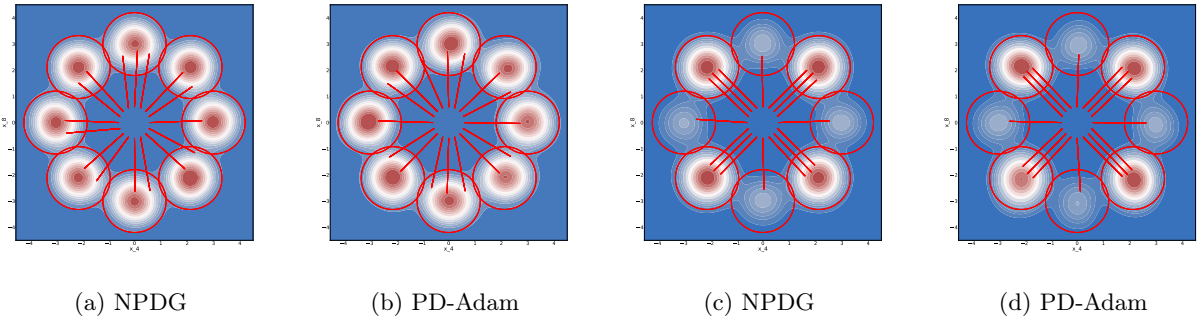


Figure 13: **OT problem (10D)**: Plots of the pushforwarded density $T_{\theta\#}\rho_0$ by using Kernel Density Estimation (KDE), together with the optimal transport map (red segments). **Left two figures**: OT from ρ_0 to ρ_a , 13a: Numerical result obtained by NPDG, 13b Numerical result obtained by PD-Adam; **Right two figures**: OT from ρ_0 to ρ_b , 13c: Numerical result obtained by NPDG, 13d: Numerical result obtained by PD-Adam. All figures are plotted on the 4 – 8 plane.

We further consider the OT problem with dimension $d = 50$ with $i_0 = 10, i_1 = 20$ in which the NPDG algorithm performs more robustly and achieves more accurate solutions compared to the PD-Adam algorithm. We set $n_{\text{MINRES}} = 1000$ and $\text{tol}_{\text{MINRES}} = 10^{-4}$. We choose the sample size $N = 2000$, the extrapolation coefficient $\omega = 5$ and stepsizes $\tau_u = \tau_\varphi = 0.5 \cdot 10^{-2}$. We perform the NPDG algorithm for 20000 iterations.

We first test the case of transporting ρ_0 to equally distributed mixed-Gaussian distribution ρ_a . We test the NPDG algorithm with various preconditioning (31), (35), as well as the PD-Adam method. The results are presented in Figure 14. It is worth mentioning that upon comparing the transport maps shown in Figure 14a and 14b, the more canonical precondition (35) yields solution with higher accuracy. We then test the case of transporting ρ_0 to non-equal mixed-Gaussian distribution ρ_b . The results are presented in Figure 15. Again, our NPDG algorithm with precondition (35) produces the transport map with better quality. Further plots on the numerical solutions can be found in Appendix 5.5.3. PD-Adam method does not behave as robustly as the NPDG algorithm in this 50D example.

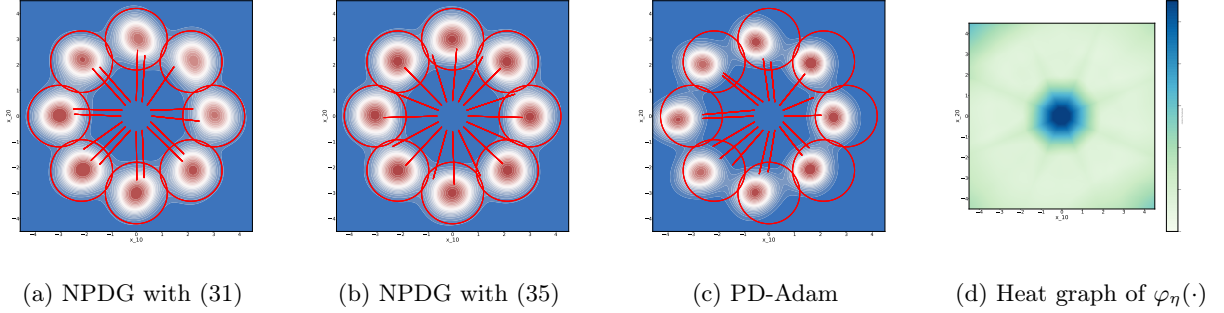


Figure 14: **OT problem from ρ_0 to ρ_a (50D)**: Plots of the pushforwarded density $T_{\theta\#}\rho_0$ by using Kernel Density Estimation (KDE). 14a-14c: Numerical results produced by NPDG method and PD-Adam method. 14d: heat graph of the Kantorovich dual function $\varphi_\eta(\cdot)$ learned from NPDG algorithm with precondition (35). All figures are plotted on the 10 – 20 coordinate plane.

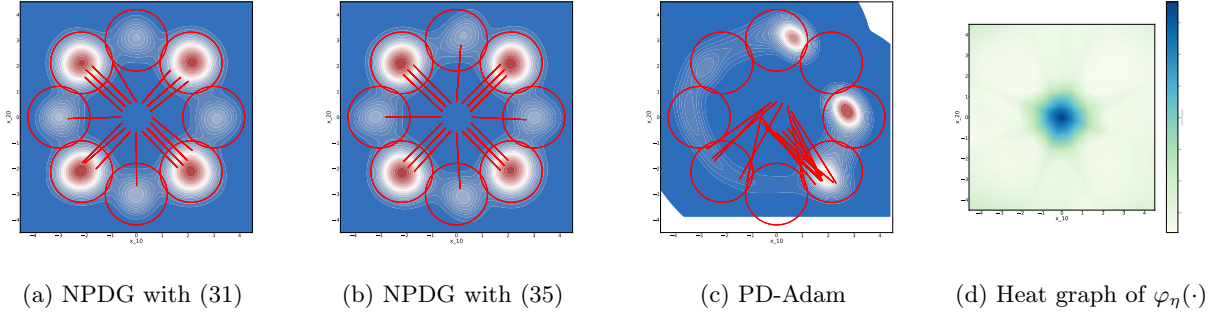


Figure 15: **OT problem from ρ_0 to ρ_b (50D)**: Plots of the pushforwarded density $T_{\theta\#}\rho_0$ by using Kernel Density Estimation (KDE). 15a-15c: Numerical results produced by NPDG method and PD-Adam method. 15d: heat graph of the Kantorovich dual function $\varphi_\eta(\cdot)$ learned from NPDG algorithm with precondition (35). All figures are plotted on the 10 – 20 coordinate plane.

6 Discussions

In this paper, we design a preconditioned adversarial training algorithm called Natural Primal Dual Hybrid Gradients (NPDHG) for solving various PDEs. We distill the information of the precondition operators

$\mathcal{M}_p, \mathcal{M}_d$, and construct them in the precondition matrices $M_p(\theta), M_d(\eta)$ for computing the natural gradients. Alternative gradient descent and ascent algorithms, together with suitable extrapolation, are utilized to update the primal and dual neural network parameters. A posterior linear convergence guarantee is established for the time-continuous version of the NPDHG algorithm. In practice, we apply the MINRES iterative solver to handle natural gradients efficiently. The algorithm performs stably and outperforms classical machine learning methods for (especially high-dimensional) PDEs, including PINN(Adam/LBFGS), DeepRitz method, and Weak Adversarial Network/Primal-Dual Adam algorithm.

Based on the numerical experiments, we also observe some critical questions about the proposed algorithm. We summarize some of them for future research directions.

- Convergence analysis for the time-discrete NPDG algorithm. What will be the optimal stepsize $\tau_u, \tau_\varphi, \tau_\psi$? Is it possible to improve the convergence speed by using adaptive stepsizes?
- Quantitative investigation of how the tolerance of MINRES tol_{MINRES} affects the convergence of the NPDG algorithm.
- Detailed analysis of coefficients α, β_1, β_2 (cf. (55), (56), (57)) for Multi-Layer Perceptrons.
- Further reduce the computational burden and improve the accuracy of the NPDG solver by considering a more meticulous way of evaluating natural gradients such as the Kronecker-factored Approximate Curvature and its variants [61, 30, 60, 19].
- Convergence analysis on the NPDG algorithm applied to different types of nonlinear PDE.
- The proposed research paves the way for the future application of natural gradient algorithms in adversarial training of neural networks, including Generative Adversarial Networks (GANs) [31, 5] and large-scale optimal transport problems [26, 45].
- Apply the approach to the time-dependent PDEs and the mean-field control or games from a temporal-space unified perspective.

Acknowledgement: S. Liu is partially supported by AFOSR YIP award No. FA9550-23-1-0087. S. Liu and S. Osher are partially funded by STROBE NSF STC DMR 1548924, AFOSR MURI FA9550-18-502, and ONR N00014-20-1-2787. W. Li is partially supported by AFOSR YIP award No. FA9550-23-1-0087, NSF DMS-2245097, and NSF RTG: 2038080.

References

- [1] Shun-Ichi Amari. Natural gradient works efficiently in learning. *Neural computation*, 10(2):251–276, 1998.
- [2] Shun-ichi Amari. *Information geometry and its applications*, volume 194. Springer, 2016.
- [3] Brandon Amos, Lei Xu, and J Zico Kolter. Input convex neural networks. In *International Conference on Machine Learning*, pages 146–155. PMLR, 2017.
- [4] Sokratis J Anagnostopoulos, Juan Diego Toscano, Nikolaos Stergiopoulos, and George Em Karniadakis. Residual-based attention and connection to information bottleneck theory in PINNs. *arXiv preprint arXiv:2307.00379*, 2023.
- [5] Martin Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein Generative Adversarial Networks. In *International Conference on Machine Learning*, pages 214–223. PMLR, 2017.
- [6] Yael Azulay and Eran Treister. Multigrid-augmented deep learning preconditioners for the Helmholtz equation. *SIAM Journal on Scientific Computing*, 45(3):S127–S151, 2022.

- [7] Gang Bao, Xiaojing Ye, Yaohua Zang, and Haomin Zhou. Numerical solution of inverse problems by weak adversarial networks. *Inverse Problems*, 36(11):115003, 2020.
- [8] Shamsulhaq Basir. Investigating and mitigating failure modes in physics-informed neural networks (PINNs). *arXiv preprint arXiv:2209.09988*, 2022.
- [9] Jean-David Benamou, Brittany D Froese, and Adam M Oberman. Two numerical methods for the elliptic Monge–Ampère equation. *ESAIM: Mathematical Modelling and Numerical Analysis*, 44(4):737–758, 2010.
- [10] Jean-David Benamou, Brittany D Froese, and Adam M Oberman. Numerical solution of the Optimal Transportation problem using the Monge–Ampère equation. *Journal of Computational Physics*, 260:107–126, 2014.
- [11] Léon Bottou and Olivier Bousquet. The Tradeoffs of Large Scale Learning. In J.C. Platt, D. Koller, Y. Singer, and S. Roweis, editors, *Advances in Neural Information Processing Systems 20 (NIPS 2007)*, pages 161–168. NIPS Foundation (<http://books.nips.cc>), 2008.
- [12] Peter N Brown and Youcef Saad. Hybrid Krylov methods for nonlinear systems of equations. *SIAM Journal on Scientific and Statistical Computing*, 11(3):450–481, 1990.
- [13] Peter N Brown and Youcef Saad. Convergence theory of nonlinear Newton–Krylov algorithms. *SIAM Journal on Optimization*, 4(2):297–330, 1994.
- [14] Joan Bruna, Benjamin Peherstorfer, and Eric Vanden-Eijnden. Neural Galerkin schemes with active learning for high-dimensional evolution equations. *Journal of Computational Physics*, 496:112588, 2024.
- [15] Russel E Caflisch. Monte Carlo and quasi-Monte Carlo methods. *Acta numerica*, 7:1–49, 1998.
- [16] Antonin Chambolle and Thomas Pock. A first-order primal-dual algorithm for convex problems with applications to imaging. *Journal of Mathematical Imaging and Vision*, 40:120–145, 2011.
- [17] Yifan Chen and Wuchen Li. Natural Gradient in Wasserstein Statistical Manifold. *arXiv:1805.08380 [cs, math]*, 2018.
- [18] Zhuo Chen, Jacob McCarran, Esteban Vizcaino, Marin Soljacic, and Di Luo. Teng: Time-evolving natural gradient for solving pdes with deep neural nets toward machine precision. In *Forty-first International Conference on Machine Learning*, 2024.
- [19] Felix Dangel, Johannes Müller, and Marius Zeinhofer. Kronecker-Factored Approximate Curvature for Physics-Informed Neural Networks. *arXiv preprint arXiv:2405.15603*, 2024.
- [20] Guido De Philippis and Alessio Figalli. The Monge–Ampère equation and its link to Optimal Transportation. *Bulletin of the American Mathematical Society*, 51(4):527–580, 2014.
- [21] Ron S Dembo, Stanley C Eisenstat, and Trond Steihaug. Inexact Newton methods. *SIAM Journal on Numerical analysis*, 19(2):400–408, 1982.
- [22] Suchuan Dong and Zongwei Li. Local extreme learning machines and domain decomposition for solving linear and nonlinear partial differential equations. *Computer Methods in Applied Mechanics and Engineering*, 387:114129, 2021.
- [23] Yifan Du and Tamer A Zaki. Evolutional deep neural network. *Physical Review E*, 104(4):045303, 2021.
- [24] John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12(7), 2011.

- [25] Stanley C Eisenstat and Homer F Walker. Globally convergent inexact Newton methods. *SIAM Journal on Optimization*, 4(2):393–422, 1994.
- [26] Jiaojiao Fan and Shu Liu. Neural Monge Map estimation and its applications. *Transactions on machine learning research*, 2023.
- [27] Roger Fletcher. *Practical methods of optimization*. John Wiley & Sons, 2000.
- [28] Brittany D Froese and Adam M Oberman. Convergent finite difference solvers for viscosity solutions of the elliptic Monge–Ampère equation in dimensions two and higher. *SIAM Journal on Numerical Analysis*, 49(4):1692–1714, 2011.
- [29] Nathan Gaby, Xiaojing Ye, and Haomin Zhou. Neural control of parametric solutions for high-dimensional evolution PDEs. *arXiv preprint arXiv:2302.00045*, 2023.
- [30] Thomas George, César Laurent, Xavier Bouthillier, Nicolas Ballas, and Pascal Vincent. Fast approximate natural gradient descent in a Kronecker factored eigenbasis. *Advances in Neural Information Processing Systems*, 31, 2018.
- [31] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks. *Communications of the ACM*, 63(11):139–144, 2020.
- [32] Jiequn Han, Arnulf Jentzen, and Weinan E. Solving high-dimensional partial differential equations using deep learning. *Proceedings of the National Academy of Sciences*, 115(34):8505–8510, 2018.
- [33] Jiequn Han, Arnulf Jentzen, et al. Deep learning-based numerical methods for high-dimensional parabolic partial differential equations and backward stochastic differential equations. *Communications in Mathematics and Statistics*, 5(4):349–380, 2017.
- [34] Wenrui Hao, Qingguo Hong, and Xianlin Jin. Gauss Newton method for solving variational problems of PDEs with neural network discretizations. *Journal of Scientific Computing*, 100(1):17, 2024.
- [35] Zheyuan Hu, Khemraj Shukla, George Em Karniadakis, and Kenji Kawaguchi. Tackling the curse of dimensionality with physics-informed neural networks. *Neural Networks*, 176:106369, 2024.
- [36] Xiaokai Huo and Hailiang Liu. Inf-Sup neural networks for high-dimensional elliptic PDE problems. *Journal of Computational Physics*, page 113188, 2024.
- [37] Martin Hutzenthaler, Arnulf Jentzen, Thomas Kruse, et al. Multilevel Picard iterations for solving smooth semilinear parabolic heat equations. *Partial Differential Equations and Applications*, 2(6):1–31, 2021.
- [38] Matt Jacobs and Flavien Léger. A fast approach to optimal transport: The back-and-forth method. *Numerische Mathematik*, 146(3):513–544, 2020.
- [39] Matt Jacobs, Flavien Léger, Wuchen Li, and Stanley Osher. Solving large-scale optimization problems with a convergence rate independent of grid size. *SIAM Journal on Numerical Analysis*, 57(3):1100–1123, 2019.
- [40] Yijie Jin, Shu Liu, Hao Wu, Xiaojing Ye, and Haomin Zhou. Parameterized Wasserstein Gradient Flow. *arXiv preprint arXiv:2404.19133*, 2024.
- [41] Diederik P Kingma. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [42] Dana A Knoll and David E Keyes. Jacobian-free Newton–Krylov methods: a survey of approaches and applications. *Journal of Computational Physics*, 193(2):357–397, 2004.

- [43] Alena Kopaničáková, Hardik Kothari, George E Karniadakis, and Rolf Krause. Enhancing training of physics-informed neural networks using domain decomposition–based preconditioning strategies. *SIAM Journal on Scientific Computing*, pages S46–S67, 2024.
- [44] Alexander Korotin, Vage Egiazarian, Arip Asadulaev, Alexander Safin, and Evgeny Burnaev. Wasserstein-2 Generative Networks. *arXiv preprint arXiv:1909.13082*, 2019.
- [45] Alexander Korotin, Daniil Selikhanovych, and Evgeny Burnaev. Neural optimal transport. *arXiv preprint arXiv:2201.12220*, 2022.
- [46] Max Kuang and Esteban G Tabak. Preconditioning of optimal transport. *SIAM Journal on Scientific Computing*, 39(4):A1793–A1810, 2017.
- [47] Devadatta Kulkarni, Darrell Schmidt, and Sze-Kai Tsui. Eigenvalues of tridiagonal pseudo-Toeplitz matrices. *Linear Algebra and its Applications*, 297:63–80, 1999.
- [48] Wuchen Li and Guido Montufar. Natural Gradient via Optimal Transport. *arXiv:1803.07033 [cs, math]*, 2018.
- [49] Dong C Liu and Jorge Nocedal. On the limited memory BFGS method for large scale optimization. *Mathematical programming*, 45(1):503–528, 1989.
- [50] Min Liu, Zhiqiang Cai, and Karthik Ramani. Deep Ritz method with adaptive quadrature for linear elasticity. *Computer Methods in Applied Mechanics and Engineering*, 415:116229, 2023.
- [51] Shu Liu, Wuchen Li, Hongyuan Zha, and Haomin Zhou. Neural Parametric Fokker–Planck Equation. *SIAM Journal on Numerical Analysis*, 60(3):1385–1449, 2022.
- [52] Shu Liu, Siting Liu, Stanley Osher, and Wuchen Li. A first-order computational algorithm for reaction-diffusion type equations via primal-dual hybrid gradient method. *Journal of Computational Physics*, 500:112753, 2024.
- [53] Shu Liu, Xinzhe Zuo, Stanley Osher, and Wuchen Li. Numerical analysis of a first-order computational algorithm for reaction-diffusion equations via the primal-dual hybrid gradient method. *arXiv preprint arXiv:2401.14602*, 2024.
- [54] Siting Liu, Stanley Osher, Wuchen Li, and Chi-Wang Shu. A primal-dual approach for solving conservation laws with implicit in time approximations. *Journal of Computational Physics*, 472, 2023.
- [55] Songming Liu, Chang Su, Jiachen Yao, Zhongkai Hao, Hang Su, Youjia Wu, and Jun Zhu. Preconditioning for physics-informed neural networks. *arXiv preprint arXiv:2402.00531*, 2024.
- [56] Lu Lu, Xuhui Meng, Zhiping Mao, and George Em Karniadakis. DeepXDE: A deep learning library for solving differential equations. *SIAM review*, 63(1):208–228, 2021.
- [57] Yulong Lu, Jianfeng Lu, and Min Wang. A Priori Generalization Analysis of the Deep Ritz Method for Solving High Dimensional Elliptic Partial Differential Equations. In Mikhail Belkin and Samory Kpotufe, editors, *Proceedings of Thirty Fourth Conference on Learning Theory*, volume 134 of *Proceedings of Machine Learning Research*, pages 3196–3241. PMLR, 15–19 Aug 2021.
- [58] Ashok Makkuva, Amirhossein Taghvaei, Sewoong Oh, and Jason Lee. Optimal transport mapping via input convex neural networks. In *International Conference on Machine Learning*, pages 6672–6681. PMLR, 2020.
- [59] James Martens. Deep learning via Hessian-free optimization . In *ICML*, volume 27, pages 735–742, 2010.
- [60] James Martens. New insights and perspectives on the natural gradient method. *Journal of Machine Learning Research*, 21(146):1–76, 2020.

- [61] James Martens and Roger Grosse. Optimizing neural networks with Kronecker-factored approximate curvature. In *International Conference on Machine Learning*, pages 2408–2417. PMLR, 2015.
- [62] Levi McClenny and Ulisses Braga-Neto. Self-adaptive physics-informed neural networks using a soft attention mechanism. *arXiv preprint arXiv:2009.04544*, 2020.
- [63] Levi D McClenny and Ulisses M Braga-Neto. Self-adaptive physics-informed neural networks. *Journal of Computational Physics*, 474:111722, 2023.
- [64] Tingwei Meng, Wenbo Hao, Siting Liu, Stanley J Osher, and Wuchen Li. Primal-dual hybrid gradient algorithms for computing time-implicit Hamilton-Jacobi equations. *arXiv preprint arXiv:2310.01605*, 2023.
- [65] Johannes Müller and Marius Zeinhofer. Achieving high accuracy with PINNs via energy natural gradient descent. In *International Conference on Machine Learning*, pages 25471–25485. PMLR, 2023.
- [66] Michael Neilan, Abner J Salgado, and Wujun Zhang. The Monge–Ampère equation. In *Handbook of Numerical Analysis*, volume 21, pages 105–219. Elsevier, 2020.
- [67] Y. Nesterov. A method for solving the convex programming problem with convergence rate $O\left(\frac{1}{k^2}\right)$. *Doklady Akademii Nauk SSSR*, 269:543–547, 1983.
- [68] Naxian Ni and Suchuan Dong. Numerical computation of partial differential equations by hidden-layer concatenated extreme learning machine. *Journal of Scientific Computing*, 95(2):35, 2023.
- [69] Levon Nurbekyan, Wanzhou Lei, and Yunan Yang. Efficient natural gradient descent methods for large-scale PDE-based optimization problems. *SIAM Journal on Scientific Computing*, 45(4):A1621–A1655, 2023.
- [70] Christopher C Paige and Michael A Saunders. Solution of sparse indefinite systems of linear equations. *SIAM journal on numerical analysis*, 12(4):617–629, 1975.
- [71] Yesom Park, Changhoon Song, and Myungjoo Kang. Beyond Derivative Pathology of PINNs: Variable Splitting Strategy with Convergence Analysis. *arXiv preprint arXiv:2409.20383*, 2024.
- [72] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems 32*, pages 8024–8035. Curran Associates, Inc., 2019.
- [73] Maziar Raissi, Paris Perdikaris, and George E Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.
- [74] Pratik Rathore, Weimu Lei, Zachary Frangella, Lu Lu, and Madeleine Udell. Challenges in training PINNs: A loss landscape perspective. *arXiv preprint arXiv:2402.01868*, 2024.
- [75] Herbert Robbins and Sutton Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, pages 400–407, 1951.
- [76] Fred Roosta, Yang Liu, Peng Xu, and Michael W Mahoney. Newton-MR: Inexact Newton method with minimum residual sub-problem solver. *EURO Journal on Computational Optimization*, 10:100035, 2022.
- [77] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

- [78] David Saad. Online algorithms and stochastic approximations. *Online Learning*, 5(3):6, 1998.
- [79] Zebang Shen, Zhenfu Wang, Alejandro Ribeiro, and Hamed Hassani. Sinkhorn natural gradient for generative models. *Advances in Neural Information Processing Systems*, 33:1646–1656, 2020.
- [80] Amanpreet Singh, Martin Bauer, and Sarang Joshi. Physics informed convex artificial neural networks (PICANNs) for optimal transport based density estimation. *arXiv preprint arXiv:2104.01194*, 2021.
- [81] Justin Sirignano and Konstantinos Spiliopoulos. DGM: A deep learning algorithm for solving partial differential equations. *Journal of Computational Physics*, 375:1339–1364, 2018.
- [82] Yang Song, Jiaming Song, and Stefano Ermon. Accelerating natural gradient with higher-order invariance. In *International Conference on Machine Learning*, pages 4713–4722. PMLR, 2018.
- [83] Weijie Su, Stephen Boyd, and Emmanuel J Candes. A differential equation for modeling Nesterov’s accelerated gradient method: Theory and insights. *Journal of Machine Learning Research*, 17(153):1–43, 2016.
- [84] Kejun Tang, Xiaoliang Wan, and Qifeng Liao. Adaptive deep density approximation for Fokker-Planck equations. *Journal of Computational Physics*, 457:111080, 2022.
- [85] Kejun Tang, Xiaoliang Wan, and Chao Yang. DAS-PINNs: A deep adaptive sampling method for solving high-dimensional partial differential equations. *Journal of Computational Physics*, 476:111868, 2023.
- [86] Philip Thomas, Bruno Castro Silva, Christoph Dann, and Emma Brunskill. Energetic natural gradient descent. In *International Conference on Machine Learning*, pages 2887–2895. PMLR, 2016.
- [87] Tijmen Tieleman and Geoffrey Hinton. Lecture 6.5-rmsprop, coursera: Neural networks for machine learning. *University of Toronto, Technical Report*, 6, 2012.
- [88] Cédric Villani. *Topics in Optimal Transportation*, volume 58. American Mathematical Soc., 2021.
- [89] Cédric Villani et al. *Optimal Transport: Old and New*, volume 338. Springer, 2009.
- [90] Sifan Wang, Xinling Yu, and Paris Perdikaris. When and why PINNs fail to train: A neural tangent kernel perspective. *Journal of Computational Physics*, 449:110768, 2022.
- [91] Yifei Wang and Wuchen Li. Information Newton’s flow: second-order optimization method in probability space. *arXiv preprint arXiv:2001.04341*, 2020.
- [92] Yiran Wang and Suchuan Dong. An extreme learning machine-based method for computational PDEs in higher dimensions. *Computer Methods in Applied Mechanics and Engineering*, 418:116578, 2024.
- [93] Colby L Wight and Jia Zhao. Solving Allen-Cahn and Cahn-Hilliard equations using the adaptive physics informed neural networks. *arXiv preprint arXiv:2007.04542*, 2020.
- [94] Hao Wu, Shu Liu, Xiaojing Ye, and Haomin Zhou. Parameterized Wasserstein Hamiltonian flow. *arXiv preprint arXiv:2306.00191*, 2023.
- [95] Lexing Ying. Natural gradient for combined loss using wavelets. *Journal of Scientific Computing*, 86(2):26, 2021.
- [96] Bing Yu et al. The deep Ritz method: a deep learning-based numerical algorithm for solving variational problems. *Communications in Mathematics and Statistics*, 6(1):1–12, 2018.
- [97] Yaohua Zang, Gang Bao, Xiaojing Ye, and Haomin Zhou. Weak adversarial networks for high-dimensional partial differential equations. *Journal of Computational Physics*, 411:109409, 2020.

- [98] Matthew D Zeiler. ADADELTA: an adaptive learning rate method. *arXiv preprint arXiv:1212.5701*, 2012.
- [99] Qi Zeng, Yash Kothari, Spencer H Bryngelson, and Florian Schäfer. Competitive physics informed networks. *arXiv preprint arXiv:2204.11144*, 2022.
- [100] Mingqiang Zhu and Tony Chan. An efficient primal-dual hybrid gradient algorithm for total variation image restoration. *UCLA CAM Report*, 34:8–34, 2008.
- [101] Xinzhe Zuo, Jiaxi Zhao, Shu Liu, Stanley Osher, and Wuchen Li. Numerical Analysis on Neural Network Projected Schemes for Approximating One Dimensional Wasserstein Gradient Flows. *arXiv preprint arXiv:2402.16821*, 2024.

A Multiple Layer Perceptron (MLP)

In this research, we denote a Multiple Layer Perceptron (MLP) with activation function f , input dimension d_{in} , hidden dimension d_h , output dimension d_{out} , and number of layers n_l as $\text{MLP}_f(d_{in}, d_h, d_{out}, n_l)$. Such MLP takes the form

$$\text{MLP}_f(d_{in}, d_h, d_{out}, n_l)(x) = h_{n_l} \circ \cdots \circ h_2 \circ h_1(x),$$

where each $h_k(\cdot)$ is defined as

$$h_k(x) = \begin{cases} f(W_1 x + b_1) & \text{here } W_1 \in \mathbb{R}^{d_h \times d_{in}}, b \in \mathbb{R}^{d_{in}} \quad \text{if } k = 1 \\ f(W_k x + b_k) & \text{here } W_k \in \mathbb{R}^{d_h \times d_h}, b \in \mathbb{R}^{d_h} \quad \text{if } 2 \leq k \leq n_l - 1 \\ W_{n_l} x + b_{n_l} & \text{here } W_{n_l} \in \mathbb{R}^{d_{out} \times d_h}, b_{n_l} \in \mathbb{R}^{d_{out}} \quad \text{if } k = n_l \end{cases}.$$

The parameters of the MLP are $(W_{n_l}, b_{n_l}, \dots, W_1, b_0)$. The number of the parameters equals $(d_{out} + 1)d_h + (n_l - 2) \cdot d_h(d_h + 1) + (d_h + 1)d_{in}$. The activation function f of the MLP is usually chosen as a nonlinear function such as $\text{ReLU}(\cdot)$, $\tanh(\cdot)$, etc⁶.

B Proof of Lemma 1 and Theorem 2

In this section, we present the proof to Lemma 1 and Theorem 2. We first prove Lemma 1.

Proof of Lemma 1. We first prove that $\nabla_\theta F(\theta) \in \text{Ran}(M(\theta))$. We can first calculate

$$\nabla_\theta F(\theta) = \left\langle D_u \mathcal{F}(u_\theta), \frac{\partial u_\theta}{\partial \theta} \right\rangle_{\mathbb{X}}.$$

By decomposing $D_u \mathcal{F}(u_\theta)$ as

$$D_u \mathcal{F}(u_\theta) = \Pi_{\partial u_\theta} [D_u \mathcal{F}(u_\theta)] + \Pi_{\partial u_\theta^\perp} [D_u \mathcal{F}(u_\theta)].$$

The first term can be written as the linear combination of $\{\frac{\partial u_\theta}{\partial \theta_k}\}_{k=1}^m$, i.e. $\Pi_{\partial u_\theta} [\mathcal{F}(u_\theta)] = \frac{\partial u_\theta}{\partial \theta} \mathbf{u}$ for certain $\mathbf{u} \in \mathbb{R}^m$. The inner product between $\Pi_{\partial u_\theta^\perp} [D_u \mathcal{F}(u_\theta)]$ and $\frac{\partial u_\theta}{\partial \theta}$ equals 0. As a result, we have

$$\nabla_\theta F(\theta) = \left\langle \frac{\partial u_\theta}{\partial \theta} \mathbf{u}, \frac{\partial u_\theta}{\partial \theta} \right\rangle_{\mathbb{X}} = M(\theta) \mathbf{u} \in \text{Ran}(M(\theta)).$$

On the other hand, we write

$$f(\zeta) = \left\| D_u \mathcal{F}(u_\theta) - \frac{\partial u_\theta}{\partial \theta} \zeta \right\|_{\mathbb{X}}^2 = \zeta^\top M(\theta) \zeta - 2\zeta^\top \nabla_\theta F(\theta) + \text{Const.}$$

⁶ $\text{ReLU}(x) = \max\{x, 0\}$, $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$.

Recall that $M(\theta)$ is a Gram matrix, it is non-negative positive definite, thus $f(\zeta)$ is a convex function. Thus, $\mathbf{v}$ is a minima of $f(\zeta)$ iff $\nabla f(\zeta) = 0$, which is equivalent to $M(\theta)\mathbf{v} = \nabla_\theta F(\theta)$.

To show the orthogonality, consider arbitrary $\mathbf{w} \in \mathbb{R}^m$, for any $s \in \mathbb{R}$, $f(\mathbf{v} + s\mathbf{w}) \geq f(\mathbf{v})$. This yields

$$0 = \frac{d}{ds} f(\mathbf{v} + s\mathbf{w}) \Big|_{s=0} = \left\langle D_u \mathcal{F}(u_\theta) - \frac{\partial u_\theta}{\partial \theta} \mathbf{v}, \frac{\partial u_\theta}{\partial \theta} \mathbf{w} \right\rangle_{\mathbb{X}} \quad \text{for any } \mathbf{w} \in \mathbb{R}^m.$$

This verifies the fact that $D_u \mathcal{F}(u_\theta) - \frac{\partial u_\theta}{\partial \theta} \mathbf{v}$ is orthogonal to the subspace $\text{span}\{\frac{\partial u_\theta}{\partial \theta_1}, \dots, \frac{\partial u_\theta}{\partial \theta_m}\}$. $\square$

We then prove Theorem 2.

Proof of Theorem 2. We first recall the functional $\mathcal{E} : \mathbb{H} \times \mathbb{K}^{dual} \times \mathbb{K}_{\partial\Omega}^{dual} \rightarrow \mathbb{R}$ defined in (15),

$$\begin{aligned} \mathcal{E}(u, \varphi, \psi) &= \langle \mathcal{L}u - f, \varphi \rangle_{L^2(\Omega)} + \lambda \langle \mathcal{B}u - g, \psi \rangle_{L^2(\partial\Omega)} - \frac{\epsilon}{2} (\|\mathcal{M}_d \varphi\|_{L^2(\Omega; \mathbb{R}^r)}^2 + \|\psi\|_{L^2(\partial\Omega)}^2) \\ &= \left\langle \mathcal{M}_d^* \tilde{\mathcal{L}} \mathcal{M}_p(u - u_*), \varphi \right\rangle_{L^2(\Omega)} + \lambda \left\langle \mathcal{B}(u - u_*), \psi \right\rangle_{L^2(\partial\Omega)} - \frac{\epsilon}{2} (\|\mathcal{M}_d \varphi\|_{L^2(\Omega; \mathbb{R}^r)}^2 + \|\psi\|_{L^2(\partial\Omega)}^2) \\ &= \left\langle \tilde{\mathcal{L}} \mathcal{M}_p(u - u_*), \mathcal{M}_d \varphi \right\rangle_{L^2(\Omega; \mathbb{R}^r)} + \left\langle \sqrt{\lambda} \mathcal{B}(u - u_*), \sqrt{\lambda} \psi \right\rangle_{L^2(\partial\Omega)} - \frac{\epsilon}{2} (\|\mathcal{M}_d \varphi\|_{L^2(\Omega; \mathbb{R}^r)}^2 + \|\sqrt{\lambda} \psi\|_{L^2(\partial\Omega)}^2) \\ &= \left\langle \begin{pmatrix} \tilde{\mathcal{L}} & \\ & \text{Id} \end{pmatrix} \begin{pmatrix} \mathcal{M}_p(u - u_*) \\ \sqrt{\lambda} \mathcal{B}(u - u_*) \end{pmatrix}, \begin{pmatrix} \mathcal{M}_d \varphi \\ \sqrt{\lambda} \psi \end{pmatrix} \right\rangle_{\mathbb{L}^2} - \frac{\epsilon}{2} \left\| \begin{pmatrix} \mathcal{M}_d \varphi \\ \sqrt{\lambda} \psi \end{pmatrix} \right\|_{\mathbb{L}^2}^2. \end{aligned}$$

We now substitute u, φ, ψ with parametrized functions $u_\theta, \varphi_\eta, \psi_\xi$, with $\theta \in \Theta_\theta \subseteq \mathbb{R}^{m_\theta}, \eta \in \Theta_\eta \subseteq \mathbb{R}^{m_\eta}, \xi \in \Theta_\xi \subseteq \mathbb{R}^{m_\xi}$. Recall that we define as $\widehat{E}(\theta; \eta, \xi) = \mathcal{E}(u_\theta; \varphi_\eta, \psi_\xi)$. In our discussion, we assume that $\mathcal{M}_p(u_\theta - u_*), \mathcal{B}(u_\theta - u_*), \mathcal{M}_d \varphi_\eta$ and ψ_ξ are differentiable w.r.t. parameters θ, η, ξ ; and $\frac{\partial}{\partial \theta}(\mathcal{M}_p(u_\theta - u_*)) \in \widetilde{\mathbb{H}}$, $\frac{\partial}{\partial \eta}(\mathcal{M}_d \varphi_\eta) \in \widetilde{\mathbb{K}}^{dual}$, and $\frac{\partial}{\partial \xi}(\sqrt{\lambda} \psi_\xi) \in \mathbb{K}_{\partial\Omega}^{dual}$ for arbitrary $\theta \in \Theta_\theta, \eta \in \Theta_\eta, \xi \in \Theta_\xi$.

Now recall the preconditioning matrices introduced in (21), (25) and (24), they can be formulated as:

$$\begin{aligned} (M_p(\theta))_{ij} &= \left\langle \frac{\partial}{\partial \theta_i} \begin{pmatrix} \mathcal{M}_p(u_\theta - u_*) \\ \sqrt{\lambda} \mathcal{B}(u_\theta - u_*) \end{pmatrix}, \frac{\partial}{\partial \theta_j} \begin{pmatrix} \mathcal{M}_p(u_\theta - u_*) \\ \sqrt{\lambda} \mathcal{B}(u_\theta - u_*) \end{pmatrix} \right\rangle_{\mathbb{L}^2} \\ (M_d(\eta))_{ij} &= \left\langle \frac{\partial}{\partial \eta_i} (\mathcal{M}_d \varphi_\eta), \frac{\partial}{\partial \eta_j} (\mathcal{M}_d \varphi_\eta) \right\rangle_{L^2(\Omega; \mathbb{R}^r)} \\ (M_{bdd}(\xi))_{ij} &= \left\langle \frac{\partial}{\partial \xi_i} (\sqrt{\lambda} \psi_\xi), \frac{\partial}{\partial \xi_j} (\sqrt{\lambda} \psi_\xi) \right\rangle_{L^2(\partial\Omega)}. \end{aligned}$$

To alleviate our notation, we denote $M_{d,bdd}(\eta, \xi) = M_d(\eta) \oplus M_{bdd}(\xi) = \begin{pmatrix} M_d(\eta) & \\ & M_{bdd}(\xi) \end{pmatrix}$. We further denote

$$\mathbf{U}_\theta = \begin{pmatrix} \mathcal{M}_p(u_\theta - u_*) \\ \sqrt{\lambda} \mathcal{B}(u_\theta - u_*) \end{pmatrix} \in \widetilde{\mathbb{H}} \times \mathbb{K}_{\partial\Omega} \subseteq \mathbb{L}^2, \quad \Phi_{\eta, \xi} = \begin{pmatrix} \mathcal{M}_d \varphi_\eta \\ \sqrt{\lambda} \psi_\xi \end{pmatrix} \in \widetilde{\mathbb{K}}^{dual} \times \mathbb{K}_{\partial\Omega}^{dual} \subseteq \mathbb{L}^2.$$

By slightly abusing the notation, we denote $\tilde{\mathcal{E}} : \mathbb{L}^2 \times \mathbb{L}^2 \rightarrow \mathbb{R}$ as

$$\mathcal{E}(\mathbf{U}_\theta, \Phi_{\eta, \xi}) = \left\langle (\tilde{\mathcal{L}} \oplus \text{Id}) \mathbf{U}_\theta, \Phi_{\eta, \xi} \right\rangle_{\mathbb{L}^2} - \frac{\epsilon}{2} \|\Phi_{\eta, \xi}\|_{\mathbb{L}^2}^2,$$

which equals to the previous functional $\mathcal{E}(u_\theta, \varphi_\eta, \psi_\xi)$.

Notice that (37) is denoted as $\Phi_{\eta_t, \xi_t} + \gamma \dot{\Phi}_{\eta_t, \xi_t}$ by using our new notation, the NPDHG flow (36) can be formulated as

$$\begin{aligned} (\eta_t, \xi_t)^\top &= M_{d,bdd}(\eta_t, \xi_t)^\dagger \nabla_{\eta, \xi} \mathcal{E}(\mathbf{U}_{\theta_t}, \Phi_{\eta_t, \xi_t}) \\ \dot{\theta}_t &= -M_p(\theta_t)^\dagger \nabla_\theta \mathcal{E}(\mathbf{U}_{\theta_t}, \Phi_{\eta_t, \xi_t} + \gamma \dot{\Phi}_{\eta_t, \xi_t}). \end{aligned} \tag{50}$$

Now suppose $(\theta_t, \eta_t, \xi_t)$ solves (50), we compute

$$\dot{\Phi}_{\eta_t, \xi_t} = \frac{\partial \Phi_{\eta_t, \xi_t}}{\partial(\eta, \xi)} M_{d, bdd}(\eta_t, \xi_t)^\dagger \nabla_{\eta, \xi} \mathcal{E}(U_{\theta_t}, \Phi_{\eta_t, \xi_t}). \quad (51)$$

By treating $\mathbb{X} = \mathbb{L}^2$ and $\mathcal{F}(\cdot)$ as $\mathcal{E}(U_\theta, \cdot)$ in Lemma 1, the right-hand side of (51) is nothing but the orthogonal projection of $D_{\Phi} \mathcal{E}(U_{\theta_t}, \Phi_{\eta_t, \xi_t}) = (\tilde{\mathcal{L}} \oplus \text{Id})U_{\theta_t} - \epsilon \Phi_{\eta_t, \xi_t}$ onto the tangent space $\partial_{\eta, \xi} \Phi_{\eta_t, \xi_t}$, that is,

$$\frac{\partial \Phi_{\eta_t, \xi_t}}{\partial(\eta, \xi)} M_{d, bdd}(\eta_t, \xi_t)^\dagger \nabla_{\eta, \xi} \mathcal{E}(U_{\theta_t}, \Phi_{\eta_t, \xi_t}) = \Pi_{\partial_{\eta, \xi} \Phi_{\eta_t, \xi_t}} [D_{\Phi} \mathcal{E}(U_{\theta_t}, \Phi_{\eta_t, \xi_t})] = \Pi_{\partial_{\eta, \xi} \Phi_{\eta_t, \xi_t}} [(\tilde{\mathcal{L}} \oplus \text{Id})U_{\theta_t} - \epsilon \Phi_{\eta_t, \xi_t}].$$

Similarly

$$\dot{U}_{\theta_t} = -\frac{\partial U_{\theta_t}}{\partial \theta} M_{d, bdd}(\eta_t, \xi_t)^\dagger \nabla_{\eta, \xi} \mathcal{E}(U_{\theta_t}, \Phi_{\eta_t, \xi_t} + \gamma \dot{\Phi}_{\eta_t, \xi_t}). \quad (52)$$

By denoting $\tilde{\mathcal{L}}^*$ as the adjoint operator⁷ of $\tilde{\mathcal{L}}$, we have

$$\mathcal{E}(U, \Phi) = \left\langle (\tilde{\mathcal{L}} \oplus \text{Id})U, \Phi \right\rangle_{\mathbb{L}^2} - \frac{\epsilon}{2} \|\Phi\|_{\mathbb{L}^2}^2 = \left\langle U, (\tilde{\mathcal{L}}^* \oplus \text{Id})\Phi \right\rangle_{\mathbb{L}^2} - \frac{\epsilon}{2} \|\Phi\|_{\mathbb{L}^2}^2.$$

the right-hand side of (52) equals

$$-\Pi_{\partial_\theta U_{\theta_t}} [D_U \mathcal{E}(U_{\theta_t}, \Phi_{\eta_t, \xi_t} + \gamma \dot{\Phi}_{\eta_t, \xi_t})] = -\Pi_{\partial_\theta U_{\theta_t}} [(\tilde{\mathcal{L}}^* \oplus \text{Id})(\Phi_{\eta_t, \xi_t} + \gamma \dot{\Phi}_{\eta_t, \xi_t})],$$

Thus the corresponding dynamic of (50) in the functional space can be formulated as

$$\begin{aligned} \dot{\Phi}_{\eta_t, \xi_t} &= \Pi_{\partial_{\eta, \xi} \Phi_{\eta_t, \xi_t}} [(\tilde{\mathcal{L}} \oplus \text{Id})U_{\theta_t} - \epsilon \Phi_{\eta_t, \xi_t}], \\ \dot{U}_{\theta_t} &= -\Pi_{\partial_\theta U_{\theta_t}} [(\tilde{\mathcal{L}}^* \oplus \text{Id})(\Phi_{\eta_t, \xi_t} + \gamma \dot{\Phi}_{\eta_t, \xi_t})]. \end{aligned}$$

We now consider the Lyapunov functional

$$\begin{aligned} \mathcal{I}(U, \Phi) &= \frac{1}{2} (\|\mathcal{M}_p(u - u_*)\|_{L^2(\Omega; \mathbb{R}^r)}^2 + \lambda \|\mathcal{B}(u - u_*)\|_{L^2(\partial\Omega)}^2 + \|\mathcal{M}_d \varphi\|_{L^2(\Omega; \mathbb{R}^r)}^2 + \lambda \|\psi\|_{L^2(\partial\Omega)}^2) \\ &= \frac{1}{2} \|U\|_{\mathbb{L}^2}^2 + \frac{1}{2} \|\Phi\|_{\mathbb{L}^2}^2. \end{aligned} \quad (53)$$

We shall study the decay of this Lyapunov functional along $\{(U_{\theta_t}, \Phi_{\eta_t, \xi_t})\}$. We calculate

$$\begin{aligned} \frac{d}{dt} \mathcal{I}(U_{\theta_t}, \Phi_{\eta_t, \xi_t}) &= \left\langle U_{\theta_t}, \dot{U}_{\theta_t} \right\rangle_{\mathbb{L}^2} + \left\langle \Phi_{\eta_t, \xi_t}, \dot{\Phi}_{\eta_t, \xi_t} \right\rangle_{\mathbb{L}^2} \\ &= \underbrace{\left\langle U_{\theta_t}, -\Pi_{\partial_\theta U_{\theta_t}} [(\tilde{\mathcal{L}}^* \oplus \text{Id})(\Phi_{\eta_t, \xi_t} + \gamma \dot{\Phi}_{\eta_t, \xi_t})] \right\rangle_{\mathbb{L}^2}}_{(1)} \\ &\quad + \underbrace{\left\langle \Phi_{\eta_t, \xi_t}, \Pi_{\partial_{\eta, \xi} \Phi_{\eta_t, \xi_t}} [(\tilde{\mathcal{L}} \oplus \text{Id})U_{\theta_t} - \epsilon \Phi_{\eta_t, \xi_t}] \right\rangle_{\mathbb{L}^2}}_{(2)} \end{aligned}$$

⁷In the sense that

$$\left\langle \tilde{\mathcal{L}}v, w \right\rangle_{L^2(\Omega; \mathbb{R}^r)} = \left\langle v, \tilde{\mathcal{L}}^*w \right\rangle_{L^2(\Omega; \mathbb{R}^r)}, \quad \forall v \in \tilde{\mathbb{H}}, w \in \tilde{\mathbb{K}}^{dual}.$$

We further compute (I) as:

$$\begin{aligned}
(1) &= - \left\langle \mathbf{U}_{\theta_t}, \quad \Pi_{\partial \mathbf{U}_{\theta_t}} [(\tilde{\mathcal{L}}^* \oplus \text{Id})(\Phi_{\eta_t, \xi_t} + \gamma \Pi_{\partial \Phi_{\eta_t, \xi_t}} [(\tilde{\mathcal{L}} \oplus \text{Id}) \mathbf{U}_{\theta_t} - \epsilon \Phi_{\eta_t, \xi_t}]) \right\rangle_{\mathbb{L}^2} \\
&= - \left\langle \Pi_{\partial \mathbf{U}_{\theta_t}} [\mathbf{U}_{\theta_t}], \quad (\tilde{\mathcal{L}}^* \oplus \text{Id})(\Phi_{\eta_t, \xi_t} + \gamma(\tilde{\mathcal{L}} \oplus \text{Id}) \mathbf{U}_{\theta_t} - \gamma \epsilon \Phi_{\eta_t, \xi_t}) \right\rangle_{\mathbb{L}^2} \\
&\quad + \left\langle \Pi_{\partial \mathbf{U}_{\theta_t}} [\mathbf{U}_{\theta_t}], \quad \gamma(\tilde{\mathcal{L}}^* \oplus \text{Id}) \Pi_{\partial \Phi_{\eta_t, \xi_t}^\perp} ((\tilde{\mathcal{L}} \oplus \text{Id}) \mathbf{U}_{\theta_t} - \epsilon \Phi_{\eta_t, \xi_t}) \right\rangle_{\mathbb{L}^2} \\
&= - \underbrace{\left\langle \mathbf{U}_{\theta_t}, \quad (\tilde{\mathcal{L}}^* \oplus \text{Id})((1 - \gamma \epsilon) \Phi_{\eta_t, \xi_t} + \gamma(\tilde{\mathcal{L}} \oplus \text{Id}) \mathbf{U}_{\theta_t}) \right\rangle_{\mathbb{L}^2}}_{(A)} \\
&\quad + \underbrace{\left\langle \Pi_{\partial \mathbf{U}_{\theta_t}^\perp} [\mathbf{U}_{\theta_t}], \quad (\tilde{\mathcal{L}}^* \oplus \text{Id})((1 - \gamma \epsilon) \Phi_{\eta_t, \xi_t} + \gamma(\tilde{\mathcal{L}} \oplus \text{Id}) \mathbf{U}_{\theta_t}) \right\rangle_{\mathbb{L}^2}}_{(R1)} \\
&\quad + \gamma \underbrace{\left\langle (\tilde{\mathcal{L}} \oplus \text{Id}) \Pi_{\partial \mathbf{U}_{\theta_t}} [\mathbf{U}_{\theta_t}], \quad \Pi_{\partial \Phi_{\eta_t, \xi_t}^\perp} [(\tilde{\mathcal{L}} \oplus \text{Id}) \mathbf{U}_{\theta_t} - \epsilon \Phi_{\eta_t, \xi_t}] \right\rangle_{\mathbb{L}^2}}_{(R2)}.
\end{aligned}$$

For the second equality, we use the fact that the orthogonal projection $\Pi_{\partial \mathbf{U}_{\theta_t}}$ is self-adjoint on $\mathbb{L}^2$. Furthermore, the term (II) equals

$$\begin{aligned}
(2) &= \left\langle \Phi_{\eta_t, \xi_t}, \quad (\tilde{\mathcal{L}} \oplus \text{Id}) \mathbf{U}_{\theta_t} - \epsilon \Phi_{\eta_t, \xi_t} \right\rangle_{\mathbb{L}^2} + \left\langle \Phi_{\eta_t, \xi_t}, \quad \Pi_{\partial \Phi_{\eta_t, \xi_t}^\perp} [(\tilde{\mathcal{L}} \oplus \text{Id}) \mathbf{U}_{\theta_t} - \epsilon \Phi_{\eta_t, \xi_t}] \right\rangle_{\mathbb{L}^2} \\
&= \underbrace{\left\langle \Phi_{\eta_t, \xi_t}, \quad (\tilde{\mathcal{L}} \oplus \text{Id}) \mathbf{U}_{\theta_t} - \epsilon \Phi_{\eta_t, \xi_t} \right\rangle_{\mathbb{L}^2}}_{(B)} + \underbrace{\left\langle \Pi_{\partial \Phi_{\eta_t, \xi_t}^\perp} [\Phi_{\eta_t, \xi_t}], \quad (\tilde{\mathcal{L}} \oplus \text{Id}) \mathbf{U}_{\theta_t} - \epsilon \Phi_{\eta_t, \xi_t} \right\rangle_{\mathbb{L}^2}}_{(R3)}
\end{aligned}$$

Then one can calculate

$$\begin{aligned}
&(A) + (B) \\
&= - \left\langle \mathbf{U}_{\theta_t}, (\tilde{\mathcal{L}}^* \oplus \text{Id})((1 - \gamma \epsilon) \Phi_{\eta_t, \xi_t} + \gamma(\tilde{\mathcal{L}} \oplus \text{Id}) \mathbf{U}_{\theta_t}) \right\rangle_{\mathbb{L}^2} + \left\langle \Phi_{\eta_t, \xi_t}, (\tilde{\mathcal{L}} \oplus \text{Id}) \mathbf{U}_{\theta_t} - \epsilon \Phi_{\eta_t, \xi_t} \right\rangle_{\mathbb{L}^2} \\
&= -\gamma \left\langle \mathbf{U}_{\theta_t}, (\tilde{\mathcal{L}}^* \oplus \text{Id})(\tilde{\mathcal{L}} \oplus \text{Id}) \mathbf{U}_{\theta_t} \right\rangle_{\mathbb{L}^2} + \gamma \epsilon \left\langle \Phi_{\eta_t, \xi_t}, (\tilde{\mathcal{L}} \oplus \text{Id}) \mathbf{U}_{\theta_t} \right\rangle_{\mathbb{L}^2} - \epsilon \left\langle \Phi_{\eta_t, \xi_t}, \Phi_{\eta_t, \xi_t} \right\rangle_{\mathbb{L}^2} \quad (54)
\end{aligned}$$

Recall the assumption (38), we have:

$$\begin{aligned}
\|(\tilde{\mathcal{L}} \oplus \text{Id}) \mathbf{U}\|_{\mathbb{L}^2}^2 &= \|\tilde{\mathcal{L}} \mathbf{u}\|_{L^2(\Omega; \mathbb{R}^r)}^2 + \|w\|_{L^2(\partial \Omega)}^2 \leq L_1^2 \|\mathbf{u}\|_{L^2(\Omega; \mathbb{R}^r)}^2 + \|w\|_{L^2(\partial \Omega)}^2 \\
&\leq (L_1^2 \vee 1) \cdot (\|\mathbf{u}\|_{L^2(\Omega; \mathbb{R}^r)}^2 + \|w\|_{L^2(\partial \Omega)}^2) = (L_1^2 \vee 1) \cdot \|\mathbf{U}\|_{\mathbb{L}^2}^2.
\end{aligned}$$

That is, $\|(\tilde{\mathcal{L}} \oplus \text{Id}) \mathbf{U}\|_{\mathbb{L}^2} \leq (L_1 \vee 1) \cdot \|\mathbf{U}\|_{\mathbb{L}^2}$. Similarly, we have $\|(\tilde{\mathcal{L}} \oplus \text{Id}) \mathbf{U}\|_{\mathbb{L}^2} \geq (L_0 \wedge 1) \|\mathbf{U}\|_{\mathbb{L}^2}$.

We can verify that (54) yields

$$(A) + (B) \leq -\gamma(L_0 \wedge 1)^2 \|\mathbf{U}_{\theta_t}\|_{\mathbb{L}^2} + \gamma \epsilon (L_1 \vee 1) \|\Phi_{\eta_t, \xi_t}\|_{\mathbb{L}^2} \cdot \|\mathbf{U}_{\theta_t}\|_{\mathbb{L}^2} - \epsilon \|\Phi_{\eta_t, \xi_t}\|_{\mathbb{L}^2}^2.$$

Moreover, by Cauchy-Schwarz inequality, we estimate the remainder terms (R1), (R2), (R3) as

$$\begin{aligned}
(R1) &\leq \|\Pi_{\partial \mathbf{U}_{\theta_t}^\perp} [\mathbf{U}_{\theta_t}]\|_{\mathbb{L}^2} \cdot ((L_1 \vee 1) |1 - \gamma \epsilon| \|\Phi_{\eta_t, \xi_t}\|_{\mathbb{L}^2} + \gamma (L_1 \vee 1)^2 \|\mathbf{U}_{\theta_t}\|_{\mathbb{L}^2}) \\
&\leq \alpha \|\mathbf{U}_{\theta_t}\|_{\mathbb{L}^2} \cdot ((L_1 \vee 1) |1 - \gamma \epsilon| \|\Phi_{\eta_t, \xi_t}\|_{\mathbb{L}^2} + \gamma (L_1 \vee 1)^2 \|\mathbf{U}_{\theta_t}\|_{\mathbb{L}^2}) \\
&= \alpha \cdot (L_1 \vee 1) \cdot |1 - \gamma \epsilon| \cdot \|\mathbf{U}_{\theta_t}\|_{\mathbb{L}^2} \cdot \|\Phi_{\eta_t, \xi_t}\|_{\mathbb{L}^2} + \alpha \cdot \gamma \cdot (L_1 \vee 1)^2 \cdot \|\mathbf{U}_{\theta_t}\|_{\mathbb{L}^2}^2. \\
(R2) &\leq \gamma \cdot (L_1 \vee 1) \|\Pi_{\partial \mathbf{U}_{\theta_t}} [\mathbf{U}_{\theta_t}]\|_{\mathbb{L}^2} \cdot \|\Pi_{\partial \Phi_{\eta_t, \xi_t}^\perp} [(\tilde{\mathcal{L}} \oplus \text{Id}) \mathbf{U}_{\theta_t} - \epsilon \Phi_{\eta_t, \xi_t}]\|_{\mathbb{L}^2} \\
&\leq \gamma \cdot (L_1 \vee 1) \cdot \|\mathbf{U}_{\theta_t}\|_{\mathbb{L}^2} \cdot (\|\Pi_{\partial \Phi_{\eta_t, \xi_t}^\perp} [(\tilde{\mathcal{L}} \oplus \text{Id}) \mathbf{U}_{\theta_t}]\|_{\mathbb{L}^2} + \epsilon \|\Pi_{\partial \Phi_{\eta_t, \xi_t}} [\Phi_{\eta_t, \xi_t}]\|_{\mathbb{L}^2}) \\
&\leq \gamma \cdot (L_1 \vee 1) \cdot \|\mathbf{U}_{\theta_t}\|_{\mathbb{L}^2} \cdot (\beta_1 \|(\tilde{\mathcal{L}} \oplus \text{Id}) \mathbf{U}_{\theta_t}\|_{\mathbb{L}^2} + \epsilon \beta_2 \|\Phi_{\eta_t, \xi_t}\|_{\mathbb{L}^2}) \\
&\leq \gamma \cdot (L_1 \vee 1)^2 \cdot \beta_1 \|\mathbf{U}_{\theta_t}\|_{\mathbb{L}^2}^2 + \gamma \epsilon \cdot (L_1 \vee 1) \cdot \beta_2 \cdot \|\mathbf{U}_{\theta_t}\|_{\mathbb{L}^2} \cdot \|\Phi_{\eta_t, \xi_t}\|_{\mathbb{L}^2}.
\end{aligned}$$

$$\begin{aligned}
(R3) &\leq \|\Pi_{\partial\Phi_{\eta_t,\xi_t}^\perp} [\Phi_{\eta_t,\xi_t}]\|_{\mathbb{L}^2} \cdot \|(\tilde{\mathcal{L}} \oplus \text{Id})\mathbf{U}_{\theta_t} - \epsilon\Phi_{\eta_t,\xi_t}\|_{\mathbb{L}^2} \\
&\leq \beta_2 \cdot \|\Phi_{\eta_t,\xi_t}\|_{\mathbb{L}^2} \cdot ((L_1 \vee 1)\|\mathbf{U}_{\theta_t}\|_{\mathbb{L}^2} + \epsilon\|\Phi_{\eta_t,\xi_t}\|_{\mathbb{L}^2}) \\
&= \beta_2 \cdot (L_1 \vee 1) \cdot \|\mathbf{U}_{\theta_t}\|_{\mathbb{L}^2} \cdot \|\Phi_{\eta_t,\xi_t}\|_{\mathbb{L}^2} + \beta_2 \cdot \epsilon \cdot \|\Phi_{\eta_t,\xi_t}\|_{\mathbb{L}^2}^2.
\end{aligned}$$

Here, we denote

$$\alpha = \max_{t \in [0, T]} \frac{\|\Pi_{\partial\mathbf{U}_{\theta_t}^\perp} [\mathbf{U}_{\theta_t}]\|_{\mathbb{L}^2}}{\|\mathbf{U}_{\theta_t}\|_{\mathbb{L}^2}}; \quad (55)$$

$$\beta_1 = \max_{t \in [0, T]} \frac{\|\Pi_{\partial\Phi_{\eta_t,\xi_t}^\perp} [(\tilde{\mathcal{L}} \oplus \text{Id})\mathbf{U}_{\theta_t}]\|_{\mathbb{L}^2}}{\|(\tilde{\mathcal{L}} \oplus \text{Id})\mathbf{U}_{\theta_t}\|_{\mathbb{L}^2}}; \quad (56)$$

$$\beta_2 = \max_{t \in [0, T]} \frac{\|\Pi_{\partial\Phi_{\eta_t,\xi_t}^\perp} [\Phi_{\eta_t,\xi_t}]\|_{\mathbb{L}^2}}{\|\Phi_{\eta_t,\xi_t}\|_{\mathbb{L}^2}}. \quad (57)$$

It is not hard to tell that $0 \leq \alpha, \beta_1, \beta_2 \leq 1$.

Now, recall (54) and (54), together with the estimates on the remainder terms (R1), (R2), (R3) we obtain

$$\begin{aligned}
\frac{d}{dt} \mathcal{I}(\mathbf{U}_{\theta_t}, \Phi_{\eta_t,\xi_t}) &\leq -\gamma \cdot ((L_0 \wedge 1)^2 - (L_1 \vee 1)^2(\alpha + \beta_1)) \cdot \|\mathbf{U}_{\theta_t}\|_{\mathbb{L}^2}^2 \\
&\quad + (L_1 \vee 1) \cdot ((1 + \beta_1)\gamma\epsilon + \beta_2 + \alpha|1 - \gamma\epsilon|) \cdot \|\Phi_{\eta_t,\xi_t}\|_{\mathbb{L}^2} \cdot \|\mathbf{U}_{\theta_t}\|_{\mathbb{L}^2} \\
&\quad - \epsilon \cdot (1 - \beta_2) \cdot \|\Phi_{\eta_t,\xi_t}\|_{\mathbb{L}^2}^2. \\
&\leq -[\|\mathbf{U}_{\theta_t}\|_{\mathbb{L}^2}, \|\Phi_{\eta_t,\xi_t}\|_{\mathbb{L}^2}] \underbrace{\begin{bmatrix} \Gamma_{\mathbf{U}\mathbf{U}} & \Gamma_{\Phi\mathbf{U}}/2 \\ \Gamma_{\Phi\mathbf{U}}/2 & \Gamma_{\Phi\Phi} \end{bmatrix}}_{\Gamma} \begin{bmatrix} \|\mathbf{U}_{\theta_t}\|_{\mathbb{L}^2} \\ \|\Phi_{\eta_t,\xi_t}\|_{\mathbb{L}^2} \end{bmatrix}.
\end{aligned}$$

Here we denote

$$\begin{aligned}
\Gamma_{\mathbf{U}\mathbf{U}} &= \gamma \cdot ((L_0 \wedge 1)^2 - (L_1 \vee 1)^2(\alpha + \beta_1)), \quad \Gamma_{\Phi\Phi} = \epsilon(1 - \beta_2), \\
\Gamma_{\Phi\mathbf{U}} &= -(L_1 \vee 1) \cdot ((1 + \beta_1)\gamma\epsilon + \beta_2 + \alpha|1 - \gamma\epsilon|).
\end{aligned}$$

Since we assumed that $\frac{1}{\kappa^2} > \alpha + \beta_1$, this yields $\Gamma_{\mathbf{U}\mathbf{U}} > 0$; and $\beta_2 < 1$ yields $\Gamma_{\Phi\Phi} > 0$; moreover, (40) is equivalent to $\det(\Gamma) = \Gamma_{\mathbf{U}\mathbf{U}}\Gamma_{\Phi\Phi} - \frac{1}{4}\Gamma_{\Phi\mathbf{U}}^2 > 0$. In conclusion, these lead to the fact that Γ is positive definite. Further, we denote the smaller eigenvalue of Γ as

$$r = \frac{1}{2} \left(\Gamma_{\mathbf{U}\mathbf{U}} + \Gamma_{\Phi\Phi} - \sqrt{(\Gamma_{\mathbf{U}\mathbf{U}} - \Gamma_{\Phi\Phi})^2 + \Gamma_{\Phi\mathbf{U}}^2} \right). \quad (58)$$

Thus, $r > 0$, and we obtain

$$\frac{d}{dt} \mathcal{I}(\mathbf{U}_{\theta_t}, \Phi_{\eta_t,\xi_t}) \leq -r \cdot \mathcal{I}(\mathbf{U}_{\theta_t}, \Phi_{\eta_t,\xi_t}), \quad t \in [0, T].$$

Applying the Gröwall's inequality yields

$$\mathcal{I}(\mathbf{U}_{\theta_t}, \Phi_{\eta_t,\xi_t}) \leq \exp(-rt) \cdot \mathcal{I}(\mathbf{U}_{\theta_0}, \Phi_{\eta_0,\xi_0}),$$

for $t \in [0, T]$. Recall definition (53), we have proven the theorem

$$\|\mathcal{M}_p(u_{\theta_t} - u_*)\|_{L^2(\Omega; \mathbb{R}^r)}^2 + \lambda \|\mathcal{B}(u_{\theta_t} - u_*)\|_{L^2(\partial\Omega)}^2 \leq 2 \exp(-rt) \cdot \mathcal{I}(\mathbf{U}_{\theta_0}, \Phi_{\eta_0,\xi_0}), \quad 0 \leq t \leq T.$$

□

C Basic settings for the methods tested in section 5

We provide the loss function, as well as the hyperparameters of the three methods PINN, Deep Ritz, and WAN tested In experiments in the following Table 2. In the following Table 3, we summarize the real solutions and their norms for equation (43), (44) and 45 tested in our experiments.

		PINN	Deep Ritz	WAN/Primal-Dual using Adam
Poisson (43) ($d = 10, 50$)	loss function	$\int_{\Omega} -\Delta u_{\theta} - f ^2 dx + \lambda \int_{\partial\Omega} u_{\theta} - g ^2 d\sigma$	$\int_{\Omega} \frac{1}{2} \ \nabla u_{\theta}\ ^2 - f u_{\theta} dx + \lambda \int_{\partial\Omega} u_{\theta} - g ^2 d\sigma$	$\log(\int_{\Omega} \nabla u_{\theta} \cdot \nabla \varphi_{\eta} - f \varphi_{\eta} dx ^2) - \log(\int_{\Omega} \varphi_{\eta}^2 dx) + \lambda \int_{\partial\Omega} u_{\theta} - g ^2 d\sigma$
	λ	10^4	10^4	10^4
	lr	$lr = 10^{-4}$	$lr = 10^{-4}$	$\tau_{\theta} = 0.5 \cdot 10^{-3}$ $\tau_{\eta} = 0.5 \cdot 10^{-2}$
	N_{iter}	Iterate till GPU time reaches 200s ($d = 10$)/8000s ($d = 50$)		
	(N_{in}, N_{bdd})	(2000, 80d)	(2000, 80d)	(10000, 60d)
	NN	$u_{\theta} = \text{MLP}_{\tanh}(d, 256, 1, 4), \quad \varphi_{\eta} = \text{MLP}_{\tanh}(d, 256, 1, 4) \cdot \zeta$		
VarCoeff (44) ($d = 10, 20, 50$)	loss function	$\int_{\Omega} -\nabla \cdot (\kappa \nabla u_{\theta}) - f ^2 dx + \lambda \int_{\partial\Omega} u_{\theta} - g ^2 d\sigma$	$\int_{\Omega} \kappa \ \nabla u_{\theta}\ ^2 dx + \lambda \int_{\partial\Omega} u_{\theta} - g ^2 d\sigma$	$\log(\int_{\Omega} \kappa \nabla u_{\theta} \cdot \nabla \varphi_{\eta} dx ^2) - \log(\int_{\Omega} \varphi_{\eta}^2 dx) + \lambda \int_{\partial\Omega} u_{\theta} - g ^2 d\sigma$
	λ	10^4	10^3	10^4
	lr	$lr = 10^{-4}$	$lr = 0.5 \cdot 10^{-3}$	$(d = 10) \quad \tau_{\theta} = 0.5 \cdot 10^{-2}$ $\tau_{\eta} = 0.5 \cdot 10^{-1}$ $(d = 20, 50) \quad \tau_{\theta} = 0.5 \cdot 10^{-3}$ $\tau_{\eta} = 0.5 \cdot 10^{-2}$
	N_{iter}	Iterate till GPU time reaches 500s ($d = 10$)/1500s ($d = 20$)		
	(N_{in}, N_{bdd})	14000 ($d = 50$) (4000, 80d)	10000 ($d = 50$) (4000, 80d)	12000 ($d = 50$) (4000, 80d)
	NN	$u_{\theta} = \text{MLP}_{\text{softplus}}(d, 256, 1, 4), \quad \varphi_{\eta} = \text{MLP}_{\text{softplus}}(d, 256, 1, 4) \cdot \zeta$ for $d = 10, 20$ $u_{\theta} = \text{MLP}_{\text{softplus}}(d, 256, 1, 6), \quad \varphi_{\eta} = \text{MLP}_{\text{softplus}}(d, 256, 1, 6) \cdot \zeta$ for $d = 50$		
Nonlinear Elliptic (45) $d = 5$	loss function	$\int_{\Omega} \frac{1}{2} \ \nabla u_{\theta}\ ^2 + V - \Delta u_{\theta} ^2 dx + \lambda \int_{\partial\Omega} u_{\theta}^2 d\sigma$	N.A.	$\log(\int_{\Omega} \nabla u_{\theta} \cdot \nabla \varphi_{\eta} + \frac{1}{2} \ \nabla u_{\theta}\ ^2 \varphi_{\eta} + V \varphi_{\eta} dx ^2) - \log(\int_{\Omega} \varphi_{\eta}^2 dx) + \lambda \int_{\partial\Omega} u_{\theta}^2 d\sigma$
	λ	10^4	N.A.	10^3
	lr	10^{-4}	N.A.	$0.5 \cdot 10^{-3}, 0.5 \cdot 10^{-2}$
	N_{iter}	200000	N.A.	200000
	(N_{in}, N_{bdd})	(4000, 40d)	N.A.	(4000, 40d)
	NN	$u_{\theta} = \text{MLP}_{\tanh}(d, 256, 1, 4), \quad \varphi_{\eta} = \text{MLP}_{\tanh}(d, 256, 1, 4) \cdot \zeta$		

Table 2: Loss functions and hyperparameters of the different methods tested In experiments. For Poisson equation (43), we perform each method for 200s; For the equation with variable coefficient (44) in 10d and 20d, we perform each method for 500s and 1500s respectively.

	Domain Ω	Solution u_*	$\ u_*\ _{L^2(\mu)} = \ u_*\ _{L^2(\Omega)} / \sqrt{ \Omega }$	$\ \nabla u_*\ _{L^2(\mu)} = \ \nabla u_*\ _{L^2(\Omega)} / \sqrt{ \Omega }$
Poisson (43)	$[0, 1]^d$ $ \Omega = 1$	$\sum_{k=1}^d \sin(\frac{\pi}{2} x_k)$	$\sqrt{\frac{4d(d-1)}{\pi^2} + \frac{d}{2}}$	$\sqrt{\frac{\pi^2 d}{8}}$
VarCoeff (44)	$[-1, 1]^d$ $ \Omega = 2^d$	$\frac{1}{2} x^{\top} \Lambda^{-1} x$	$\sqrt{\frac{1}{\lambda_0^2} + \frac{1}{\lambda_1^2}}$	$\sqrt{(\frac{1}{\lambda_0} + \frac{1}{\lambda_1})^{\frac{d}{6}}}$
Nonlinear Elliptic (45)	$B_{d,3}$ $ \Omega = \frac{\pi^{\frac{d}{2}} 3^d}{\Gamma(\frac{d}{2} + 1)}$	$\cos(\frac{\pi}{2} \ x\)$	$\sqrt{\frac{d^2}{144} + \frac{d}{90}} + \frac{d^2}{72\lambda_0\lambda_1}$ $5d : \sqrt{\frac{1}{2} - \frac{10}{9\pi^2} + \frac{20}{27\pi^4}} \approx 0.6285$	$5d : \sqrt{\frac{\pi^2}{8} (1 + \frac{20}{9\pi^2} - \frac{40}{27\pi^4})} \approx 1.2218$

Table 3: Solutions and their norms to some of the PDEs tested In experiments.

D Comparison among different methods

In the following Table 4, we test four different methods with various step sizes on different equations. The step sizes used for each method are summarized below.

- **NPDG** ($\tau_u, \tau_{\varphi}, \tau_{\psi}$): A. $(1.5 \cdot 10^{-1}, 1.5 \cdot 10^{-1}, 1.5 \cdot 10^{-1})$, B. $(10^{-1}, 10^{-1}, 10^{-1})$, C. $(0.5 \cdot 10^{-1}, 0.95 \cdot 10^{-1}, 0.95 \cdot 10^{-1})$, D. $(0.5 \cdot 10^{-1}, 0.5 \cdot 10^{-1}, 0.5 \cdot 10^{-1})$;
We fix $tol_{\text{MINRES}} = 10^{-3}$ for $d = 5, 10, 20$, and $tol_{\text{MINRES}} = 10^{-4}$ for $d = 50$.

- **PINN(Adam)** (lr): A. $(0.5 \cdot 10^{-2})$ B. (10^{-3}) C. $(0.5 \cdot 10^{-3})$ D. (10^{-4}) E. $(0.5 \cdot 10^{-4})$;
- **DeepRitz** (lr): A. $(0.5 \cdot 10^{-2})$ B. (10^{-3}) C. $(0.5 \cdot 10^{-3})$ D. (10^{-4}) E. $(0.5 \cdot 10^{-4})$;
- **WAN** (τ_θ, τ_η): A. $(0.5 \cdot 10^{-2}, 0.5 \cdot 10^{-1})$, B. $(10^{-3}, 10^{-2})$, C. $(0.5 \cdot 10^{-3}, 0.5 \cdot 10^{-2})$, D. $(10^{-4}, 10^{-3})$, E. $(0.5 \cdot 10^{-4}, 0.5 \cdot 10^{-3})$.

We record the computation time (seconds) spent by each method to achieve accuracy δ in Table 4, we only present the time for the most efficient step size(s).

Equ	δ^*	d	PINN(Adam)	Deep Ritz	WAN	NPDG
Poisson	0.005	5D	26.22 (A)	25.11 (A)	51.14 (B)	68.87 (A)
		10D	44.83 (A)	43.45 (B)	51.65 (C)	40.98 (B)
		20D	160.82 (C)	183.49 (B)	460.12 (D)	110.42 (B)
		50D	1989.06 (C)	1452.29 (B)	2117.24 (D)	821.24 (C)
VarCoeff	0.01	10D	—	105.2 (C)	—	238.34 (C)
		20D	—	228.55 (C)	—	795.32 (C)
		50D	774.70 (D)	—	—	10250.21 (C)
	0.005	10D	—	—	—	281.26 (C)
		20D	—	—	—	998.09 (C)
		50D	—	—	—	13731.33 (C)
Nonlinear Elliptic	0.1	5D	2805.92 (B)	N.A.	1130.76 (C)	1086.35 (C)
	0.05	5D	—	N.A.	—	1894.89 (C)

Table 4: GPU time (seconds) spent by different methods upon achieving the designated accuracy δ . The uppercase letters inside each parenthesis indicate the optimal learning rate(s) used in the algorithm. We apply the Monte-Carlo method with sample size 10^5 to evaluate the relative L^2 error of u_θ . “—” denotes that the method does not achieve the designated accuracy in a given time.

* Relative L^2 error is used for Poisson and Variable coefficient equations; Average L^2 error is used for nonlinear elliptic equation.

E Primal-Dual algorithm using Adam optimizer for Optimal Transport problem

In this section, we briefly describe the PD-Adam algorithm tested in section 5.5. Recall the loss functional $\mathcal{L}(T, \varphi)$ defined in (29), we parametrize both the map T and the dual function φ by neural networks T_θ, φ_η . We aim at solving the following saddle point problem

$$\begin{aligned}
\max_{\eta} \min_{\theta} \mathcal{L}(T_\theta, \varphi_\eta) &:= \int_{\mathbb{R}^d} \frac{1}{2} \|x - T(x)\|^2 \rho_0(x) dx + \int_{\mathbb{R}^d} \varphi(T(x)) \rho_0(x) dx - \int_{\mathbb{R}^d} \varphi(y) \rho_1(y) dy \\
&\approx \frac{1}{N} \sum_{i=1}^N \frac{1}{2} \|\mathbf{X}_i - T_\theta(\mathbf{X}_i)\|^2 - \varphi_\eta(T_\theta(\mathbf{X}_i)) + \varphi_\eta(\mathbf{Y}_i),
\end{aligned} \tag{59}$$

where N is the size of the datasets, $\{\mathbf{X}_i\}_{i=1}^N, \{\mathbf{Y}_i\}_{i=1}^N$ are samples drawn by ρ_0 and ρ_1 . The PD-Adam algorithm is summarized in Algorithm 3.

In all tests, we always set $K_1 = K_2 = 1$. We summarize all the other hyperparameters of the PD-Adam algorithm in section 5.5 in Table 5.

F Further numerical results regarding section 5.5.3

For the OT problem from ρ_0 to ρ_a , we sample $\{\mathbf{X}_i\}_i^N \sim \rho_0$, and plot the pushforwarded samples $\{T_\theta(\mathbf{X}_i)\}$ in Figure 16. We use the T_θ obtained at the end of each algorithm.

Algorithm 3 Computing optimal Monge map from ρ_a to ρ_b

Input: Marginal distributions ρ_0 and ρ_1 , learning rate lr_u, lr_φ of the Adam algorithm; Batch size N , total iteration number N_{iter} .

Initialize T_θ, φ_η .

for $iter = 1$ to N_{iter} **do**

 Sample $\{\mathbf{X}_i\}_{i=1}^N \sim \rho_a$. Sample $\{\mathbf{Y}_i\}_{i=1}^N \sim \rho_b$.

 Update θ to decrease (59) by using Adam algorithm with learning rate lr_u for K_1 steps.

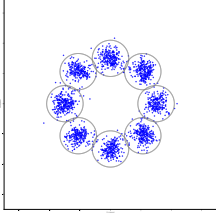
 Update η to increase (59) by using Adam algorithm with learning rate lr_φ for K_2 steps.

end for

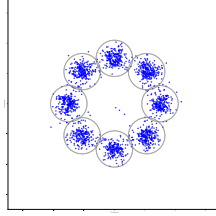
Output: The transport map T_θ .

		lr_u, lr_φ	N_{iter}	N	NN architecture
5.5.1 (1D)		$0.5 \cdot 10^{-3}, 0.5 \cdot 10^{-3}$	40000	800	$\text{MLP}_{\text{PReLU}}(1, 50, 1, 3)$
5.5.2 (5D)		$0.5 \cdot 10^{-4}, 0.5 \cdot 10^{-4}$	200000	2000	$\text{MLP}_{\text{PReLU}}(5, 80, 5, 4)$
5.5.3	(10D)	$0.5 \cdot 10^{-4}, 0.5 \cdot 10^{-4}$	100000	2000	$\text{MLP}_{\text{PReLU}}(10, 120, 10, 6)$
	(50D)	$10^{-5}, 10^{-5}$	300000	2000	$\text{MLP}_{\text{PReLU}}(50, 120, 50, 6)$

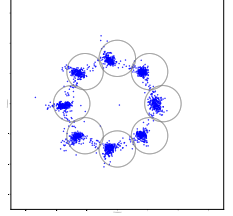
Table 5: Some hyperparameters used in the PD-Adam algorithm tested in section 5.5.



(a) NPDG with (31)



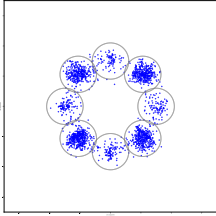
(b) NPDG with (35)



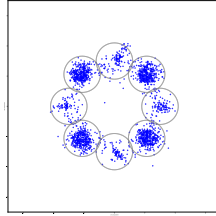
(c) PD-Adam

Figure 16: OT problem from ρ_0 to ρ_a : Plots of samples pushforwarded by the computed map T_θ . All figures are plotted on the 10 – 20 coordinate plane.

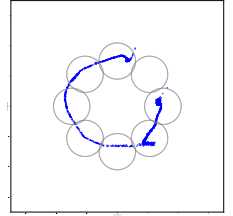
For the OT problem from ρ_0 to ρ_b , we plot the pushforwarded samples $\{T_\theta(\mathbf{X}_i)\}$ in Figure 17. We use the T_θ obtained at the end of each algorithm. Furthermore, we provide the intermediate results obtained by the NPDG algorithms as well as the PD-Adam algorithms in Figure 18. The PD-Adam method behaves unstable in this example, while the proposed NPDG method performs robustly for both preconditions.



(a) NPDG with (31)

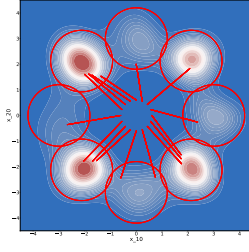


(b) NPDG with (35)

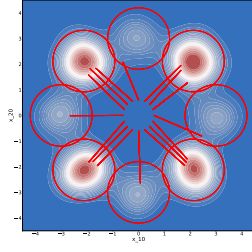


(c) PD-Adam

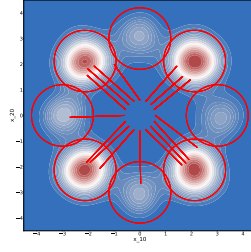
Figure 17: OT problem from ρ_0 to ρ_b : Plots of samples pushforwarded by the computed map T_θ . All figures are plotted on the 10 – 20 coordinate plane.



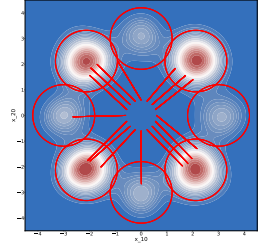
(a) NPDG with (31) at iteration 5000



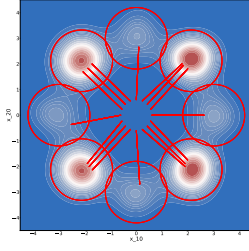
(b) NPDG with (31) at iteration 10000



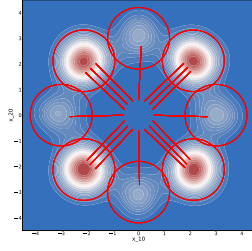
(c) NPDG with (31) at iteration 15000



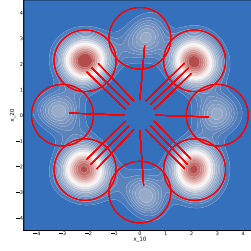
(d) NPDG with (31) at iteration 20000



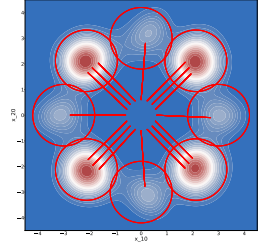
(e) NPDG with (35) at iteration 5000



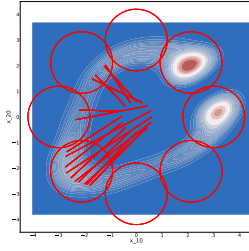
(f) NPDG with (35) at iteration 10000



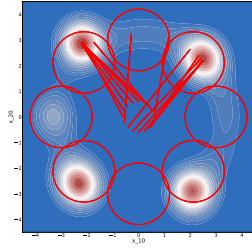
(g) NPDG with (35) at iteration 15000



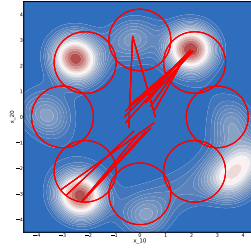
(h) NPDG with (35) at iteration 20000



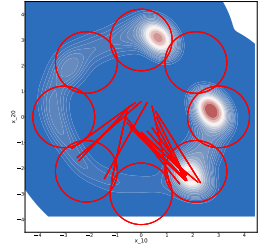
(i) PD-Adam at iteration 150000



(j) PD-Adam at iteration 200000



(k) PD-Adam at iteration 250000



(l) PD-Adam at iteration 300000

Figure 18: OT problem from ρ_0 to ρ_b : Plots of the pushforwarded densities $T_{\theta\#}\rho_0$ of the computed T_θ obtained by NPDG method (1st row & 2nd row) and PD-Adam method (3rd row). All figures are plotted on the 10 – 20 coordinate plane.